

Servlet: Web Application Basics. Architecture and challenges of Web Application. Introduction to servlet life cycle Developing and Deploying Servlets Exploring Deployment Descriptor (web.xml). Handling Request and Response Initializing a Servlet Accessing Database Servlet Chaining Session Tracking & Management Dealing with cookies Transferring Request Accessing Web Context Passing INIT and CONTEXT Parameter Sharing information using scope object Controlling concurrent access User Authentication Filtering Request and Response Programming Filter Filter Mapping Servlet Listeners.

Servlets

Servlet technology is used to create a web application (resides at server side and generates a dynamic web page).

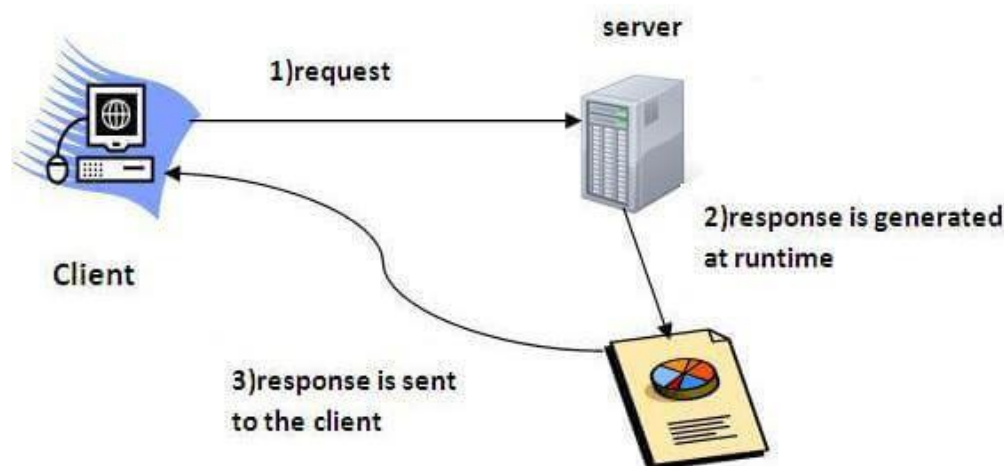
Servlet technology is robust and scalable because of java language. Before Servlet, CGI (Common Gateway Interface) scripting language was common as a server-side programming language. However, there were many disadvantages to this technology. We have discussed these disadvantages below.

There are many interfaces and classes in the Servlet API such as Servlet, GenericServlet, HttpServlet, ServletRequest, ServletResponse, etc.

What is a Servlet?

Servlet can be described in many ways, depending on the context.

- Servlet is a technology which is used to create a web application.
- Servlet is an API that provides many interfaces and classes including documentation.
- Servlet is an interface that must be implemented for creating any Servlet.
- Servlet is a class that extends the capabilities of the servers and responds to the incoming requests. It can respond to any requests.
- Servlet is a web component that is deployed on the server to create a dynamic web page.

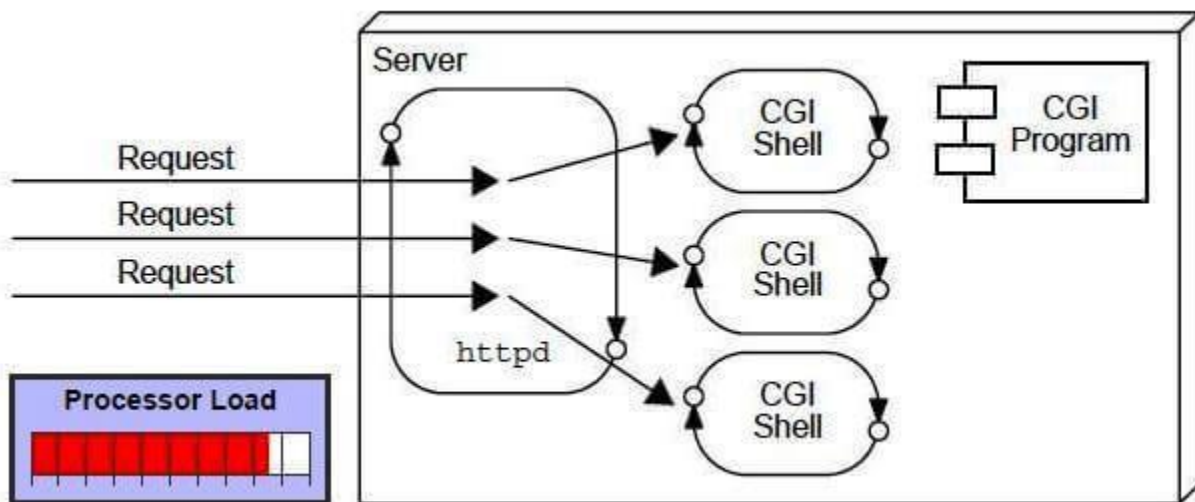


What is a web application?

A web application is an application accessible from the web. A web application is composed of web components like Servlet, JSP, Filter, etc. and other elements such as HTML, CSS, and JavaScript. The web components typically execute in Web Server and respond to the HTTP request.

CGI (Common Gateway Interface)

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.

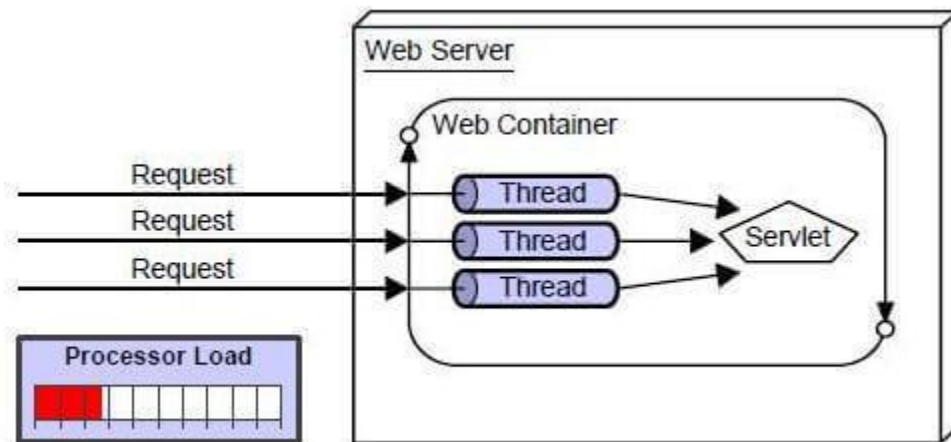


Disadvantages of CGI

There are many problems in CGI technology:

1. If the number of clients increases, it takes more time for sending the response.
 2. For each request, it starts a process, and the web server is limited to start processes.
 3. It uses platform dependent language e.g. [C](#), [C++](#), [perl](#).
-

Advantages of Servlet



There are many advantages of Servlet over CGI. The web container creates threads for handling the multiple requests to the Servlet. Threads have many benefits over the Processes such as they share a common memory area, lightweight, cost of communication between the threads are low. The advantages of Servlet are as follows:

1. **Better performance:** because it creates a thread for each request, not process.
2. **Portability:** because it uses Java language.
3. **Robust:** [JVM](#) manages Servlets, so we don't need to worry about the memory leak, [garbage collection](#), etc.
4. **Secure:** because it uses java language.

Web Terminology

Servlet Terminology	Description
Website: static vs dynamic	It is a collection of related web pages that may contain text, images, audio and video.
HTTP	It is the data communication protocol used to establish communication between client and server.
HTTP Requests	It is the request send by the computer to a web server that contains all sorts of potentially interesting information.
Get vs Post	It gives the difference between GET and POST request.
Container	It is used in java for dynamically generating the web pages on the server side.

Server: Web Application vs	It is used to manage the network resources and for running the program or software that provides services.
Content Type	It is HTTP header that provides the description about what are you sending to the browser.

Website

Website is a collection of related web pages that may contain text, images, audio and video. The first page of a website is called home page. Each website has specific internet address (URL) that you need to enter in your browser to access a website.

Website is hosted on one or more servers and can be accessed by visiting its homepage using a computer network. A website is managed by its owner that can be an individual, company or an organization.



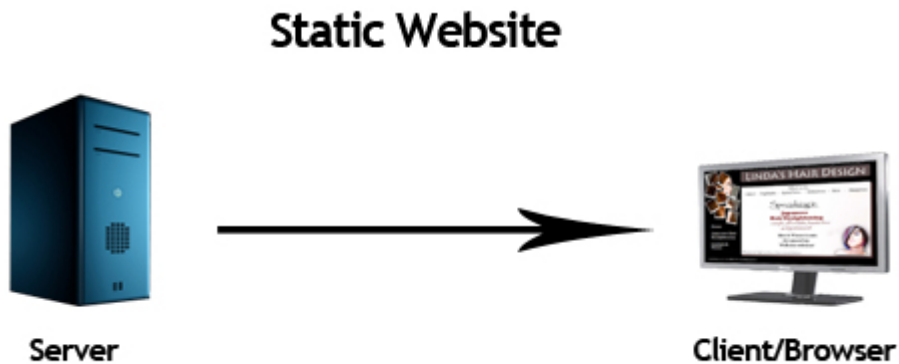
A website can be of two types:

- Static Website
 - Dynamic Website
-

Static website

Static website is the basic type of website that is easy to create. You don't need the knowledge of web programming and database design to create a static website. Its web pages are coded in HTML.

The codes are fixed for each page so the information contained in the page does not change and it looks like a printed page.



Dynamic website

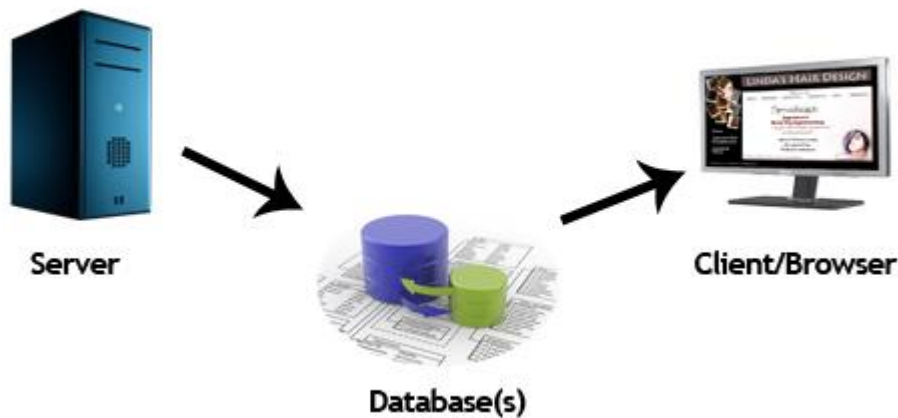
Dynamic website is a collection of dynamic web pages whose content changes dynamically. It accesses content from a database or Content Management System (CMS). Therefore, when you alter or update the content of the database, the content of the website is also altered or updated.

Dynamic website uses client-side scripting or server-side scripting, or both to generate dynamic content.

Client side scripting generates content at the client computer on the basis of user input. The web browser downloads the web page from the server and processes the code within the page to render information to the user.

In server side scripting, the software runs on the server and processing is completed in the server then plain pages are sent to the user.

Dynamic Website



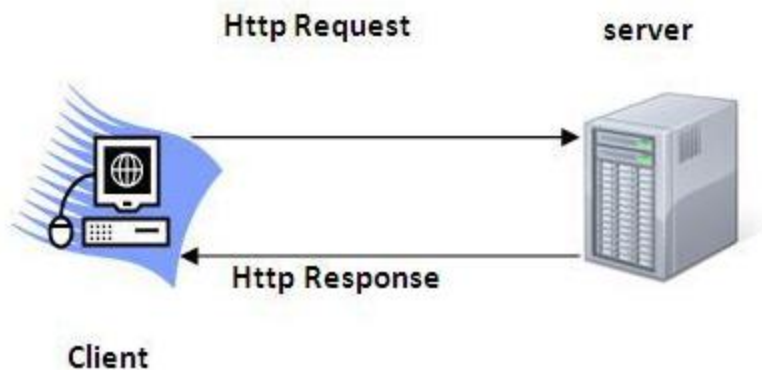
Static vs Dynamic website

Static Website	Dynamic Website
Prebuilt content is same every time the page is loaded.	Content is generated quickly and changes regularly.
It uses the HTML code for developing a website.	It uses the server side languages such as PHP, SERVLET, JSP, and ASP.NET etc. for developing a website.
It sends exactly the same response for every request.	It may generate different HTML for each of the request.
The content is only changed when someone publishes and updates the file (sends it to the web server).	The page contains "server-side" code which allows the server to generate the unique content when the page is loaded.
Flexibility is the main advantage of static website.	Content Management System (CMS) is the main advantage of dynamic website.

HTTP (Hyper Text Transfer Protocol)

The Hypertext Transfer Protocol (HTTP) is application-level protocol for collaborative, distributed, hypermedia information systems. It is the data communication protocol used to establish communication between client and server.

HTTP is TCP/IP based communication protocol, which is used to deliver the data like image files, query results, HTML files etc on the World Wide Web (WWW) with the default port is TCP 80. It provides the standardized way for computers to communicate with each other.



The Basic Characteristics of HTTP (Hyper Text Transfer Protocol):

- It is the protocol that allows web servers and browsers to exchange data over the web.
- It is a request response protocol.
- It uses the reliable TCP connections by default on TCP port 80.
- It is stateless means each request is considered as the new request. In other words, server doesn't recognize the user by default.

The Basic Features of HTTP (Hyper Text Transfer Protocol):

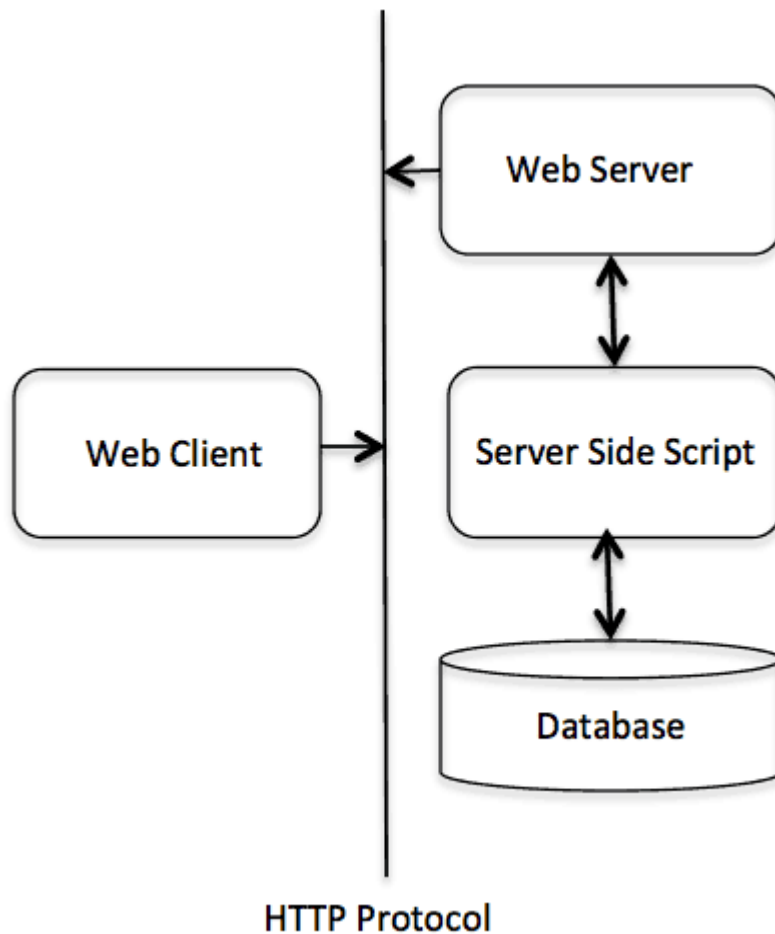
There are three fundamental features that make the HTTP a simple and powerful protocol used for communication:

- **HTTP is media independent:** It specifies that any type of media content can be sent by HTTP as long as both the server and the client can handle the data content.
- **HTTP is connectionless:** It is a connectionless approach in which HTTP client i.e., a browser initiates the HTTP request and after the request is sent the client disconnects from server and waits for the response.

- **HTTP is stateless:** The client and server are aware of each other during a current request only. Afterwards, both of them forget each other. Due to the stateless nature of protocol, neither the client nor the server can retain the information about different request across the web pages.

The Basic Architecture of HTTP (Hyper Text Transfer Protocol):

The below diagram represents the basic architecture of web application and depicts where HTTP stands:



HTTP is request/response protocol which is based on client/server based architecture. In this protocol, web browser, search engines, etc. behave as HTTP clients and the Web server like Servlet behaves as a server

HTTP Requests

The request sent by the computer to a web server, contains all sorts of potentially interesting information; it is known as HTTP requests.

The HTTP client sends the request to the server in the form of request message which includes following information:

- The Request-line
- The analysis of source IP address, proxy and port
- The analysis of destination IP address, protocol, port and host
- The Requested URI (Uniform Resource Identifier)
- The Request method and Content
- The User-Agent header
- The Connection control header
- The Cache control header



The HTTP request method indicates the method to be performed on the resource identified by the **Requested URI (Uniform Resource Identifier)**. This method is case-sensitive and should be used in uppercase.

The HTTP request methods are:

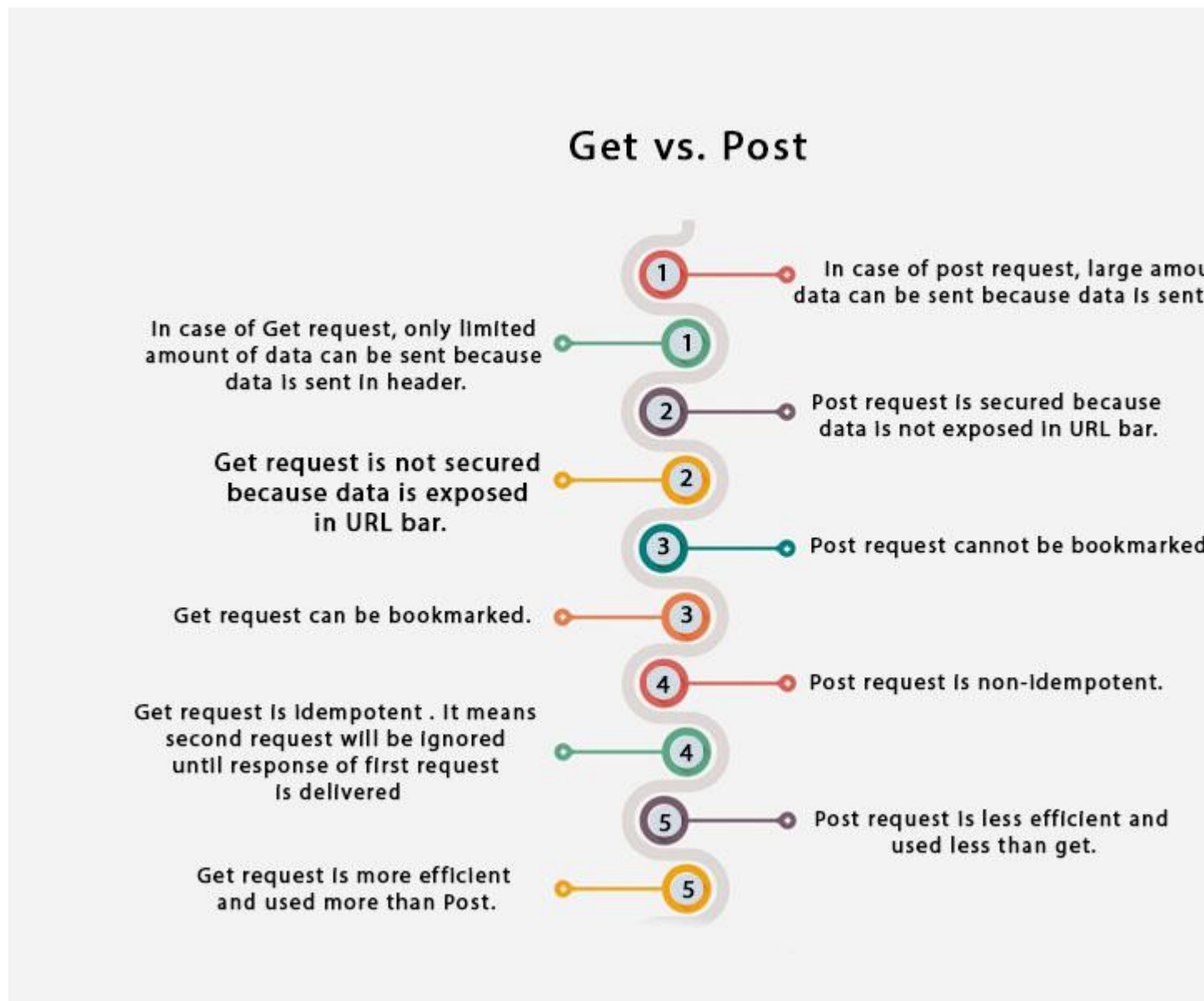
HTTP Request	Description
GET	Asks to get the resource at the requested URL.
POST	Asks the server to accept the body info attached. It is like GET request with extra info sent with the request.
HEAD	Asks for only the header part of whatever a GET would return. Just like GET but with no body.
TRACE	Asks for the loopback of the request message, for testing or troubleshooting.
PUT	Says to put the enclosed info (the body) at the requested URL.
DELETE	Says to delete the resource at the requested URL.
OPTIONS	Asks for a list of the HTTP methods to which the thing at the request URL can respond

Get vs. Post

There are many differences between the Get and Post request. Let's see these differences:

GET	POST
1) In case of Get request, only limited amount of data can be sent because data is sent in header.	In case of post request, large amount of data can be sent because data is sent in body.
2) Get request is not secured because data is exposed in URL bar.	Post request is secured because data is not exposed in URL bar.
3) Get request can be bookmarked .	Post request cannot be bookmarked .

4) Get request is idempotent . It means second request will be ignored until response of first request is delivered	Post request is non-idempotent .
5) Get request is more efficient and used more than Post.	Post request is less efficient and used less than get.



GET and POST

Two common methods for the request-response between a server and client are:

- **GET**- It requests the data from a specified resource
- **POST**- It submits the processed data to a specified resource

Anatomy of Get Request

The query string (name/value pairs) is sent inside the URL of a GET request:

1. GET/RegisterDao.jsp?name1=value1&name2=value2

As we know that data is sent in request header in case of get request. It is the default request type. Let's see what information is sent to the server.



Some other features of GET requests are:

- It remains in the browser history
- It can be bookmarked
- It can be cached
- It have length restrictions
- It should never be used when dealing with sensitive data
- It should only be used for retrieving the data

Anatomy of Post Request

The query string (name/value pairs) is sent in HTTP message body for a POST request:

1. POST/RegisterDao.jsp HTTP/1.1
2. Host: www.javatpoint.com
3. `name1=value1&name2=value2`

As we know, in case of post request original data is sent in message body. Let's see how information is passed to the server in case of post request.

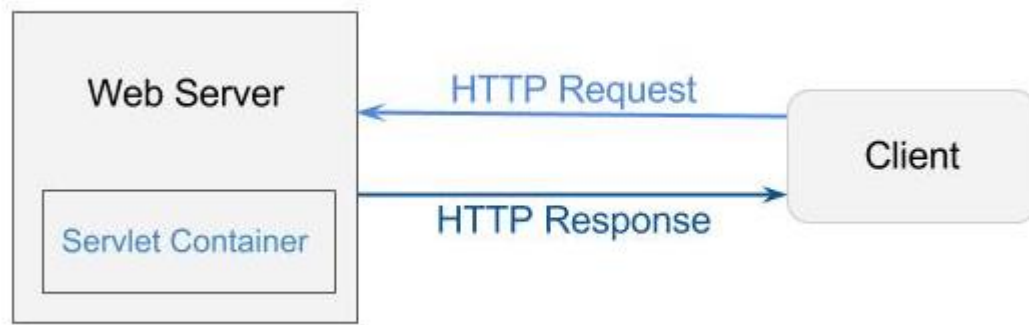
The HTTP Method	Path to the source on Web Server	Protocol Version Browser supports
The Request Headers	Post /RegisterDao.jsp HTTP/1.1	
	Host: www.javatpoint.com	
	User-Agent: Mozilla/5.0	
	Accept: text/xml,text/html,text/plain,image/jpeg	
	Accept-Language: en-us,en	
	Accept-Encoding: gzip,deflate	
	Accept-Charset: ISO-8859-1,utf-8	
	Keep-Alive:300	
	Connection:keep-alive	
	User=ravi&pass=java } Message body	

Some other features of POST requests are:

- This requests cannot be bookmarked
- This requests have no restrictions on length of data
- This requests are never cached
- This requests do not retain in the browser history

- ## Servlet Container

- It provides the runtime environment for JavaEE (j2ee) applications. The client/user can request only a static WebPages from the server. If the user wants to read the web pages as per input then the servlet container is used in java.
- The servlet container is the part of web server which can be run in a separate process. We can classify the servlet container states in three types:



-
- **Servlet Container States**
- The servlet container is the part of web server which can be run in a separate process. We can classify the servlet container states in three types:
- **Standalone:** It is typical Java-based servers in which the servlet container and the web servers are the integral part of a single program. For example:- Tomcat running by itself
- **In-process:** It is separated from the web server, because a different program runs within the address space of the main server as a plug-in. For example:- Tomcat running inside the JBoss.
- **Out-of-process:** The web server and servlet container are different programs which are run in a different process. For performing the communications between them, web server uses the plug-in provided by the servlet container.

The Servlet Container performs many operations that are given below:

- Life Cycle Management
- Multithreaded support
- Object Pooling
- Security etc.

Server: Web vs. Application

Server is a device or a computer program that accepts and responds to the request made by other program, known as client. It is used to manage the network resources and for running the program or software that provides services.

There are two types of servers:

1. Web Server
2. Application Server

Web Server

Web server contains only web or servlet container. It can be used for servlet, jsp, struts, jsf etc. It can't be used for EJB.

It is a computer where the web content can be stored. In general web server can be used to host the web sites but there also used some other web servers also such as FTP, email, storage, gaming etc.

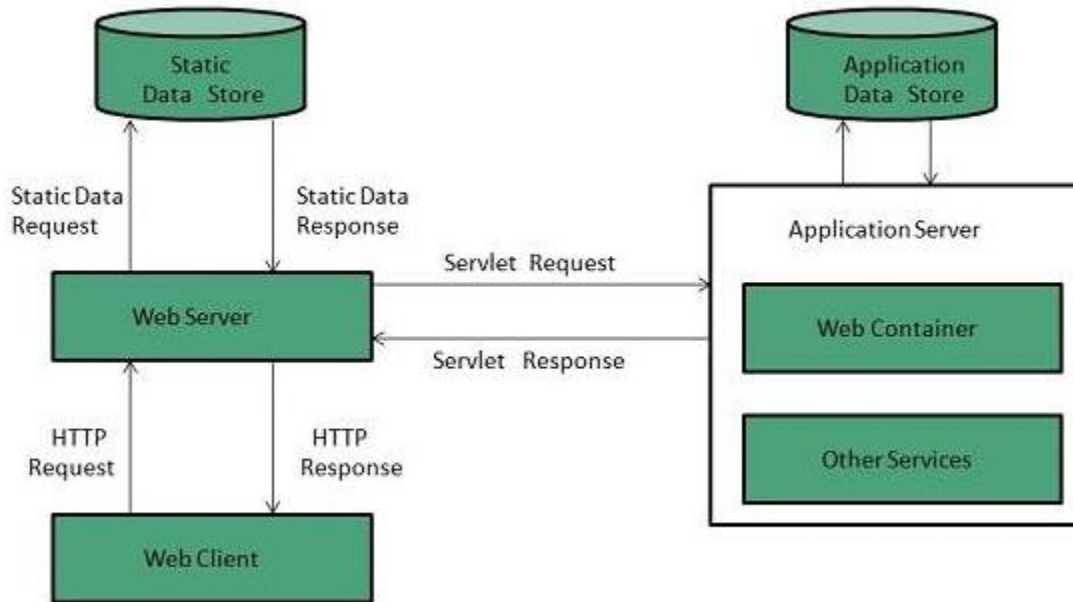
Examples of Web Servers are: **Apache Tomcat** and **Resin**.

Web Server Working

It can respond to the client request in either of the following two possible ways:

- Generating response by using the script and communicating with database.
- Sending file to the client associated with the requested URL.

The block diagram representation of Web Server is shown below:



Important points

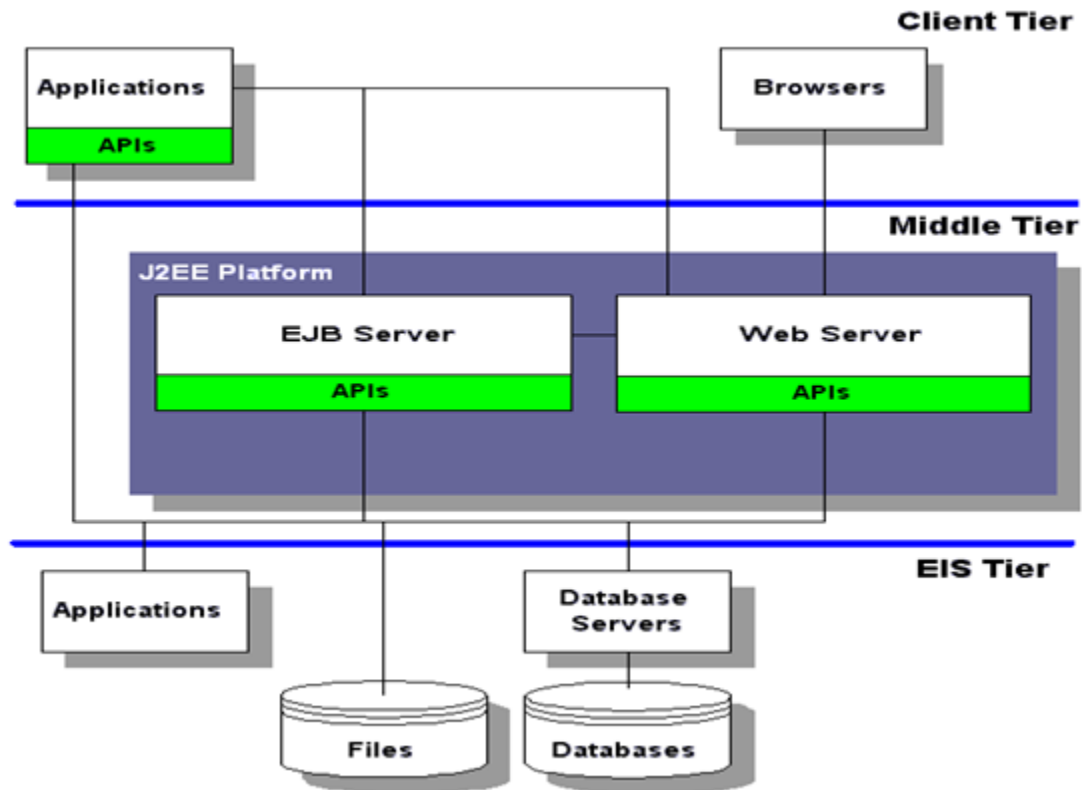
- If the requested web page at the client side is not found, then web server will send the HTTP response: Error 404 Not found.
- When the web server searches the requested page, if the requested page is found, then it will send to the client with an HTTP response.
- If the client requests some other resources, then the web server will contact the application server and use the data store for constructing the HTTP response.

Application Server

Application server contains Web and EJB containers. It can be used for servlet, jsp, struts, jsf, ejb etc. It is a component-based product that lies in the middle-tier of a server-centric architecture.

It provides the middleware services for state maintenance and security, along with persistence and data access. It is a type of server designed to install, operate and host associated services and applications for the IT services, end users and organizations.

The block diagram representation of Application Server is shown below:



The Example of Application Servers are:

1. **JBoss**: Open-source server from JBoss community.
2. **Glassfish**: Provided by Sun Microsystems. Now acquired by Oracle.
3. **Weblogic**: Provided by Oracle. It more secured.
4. **WebSphere**: Provided by IBM.

Content Type

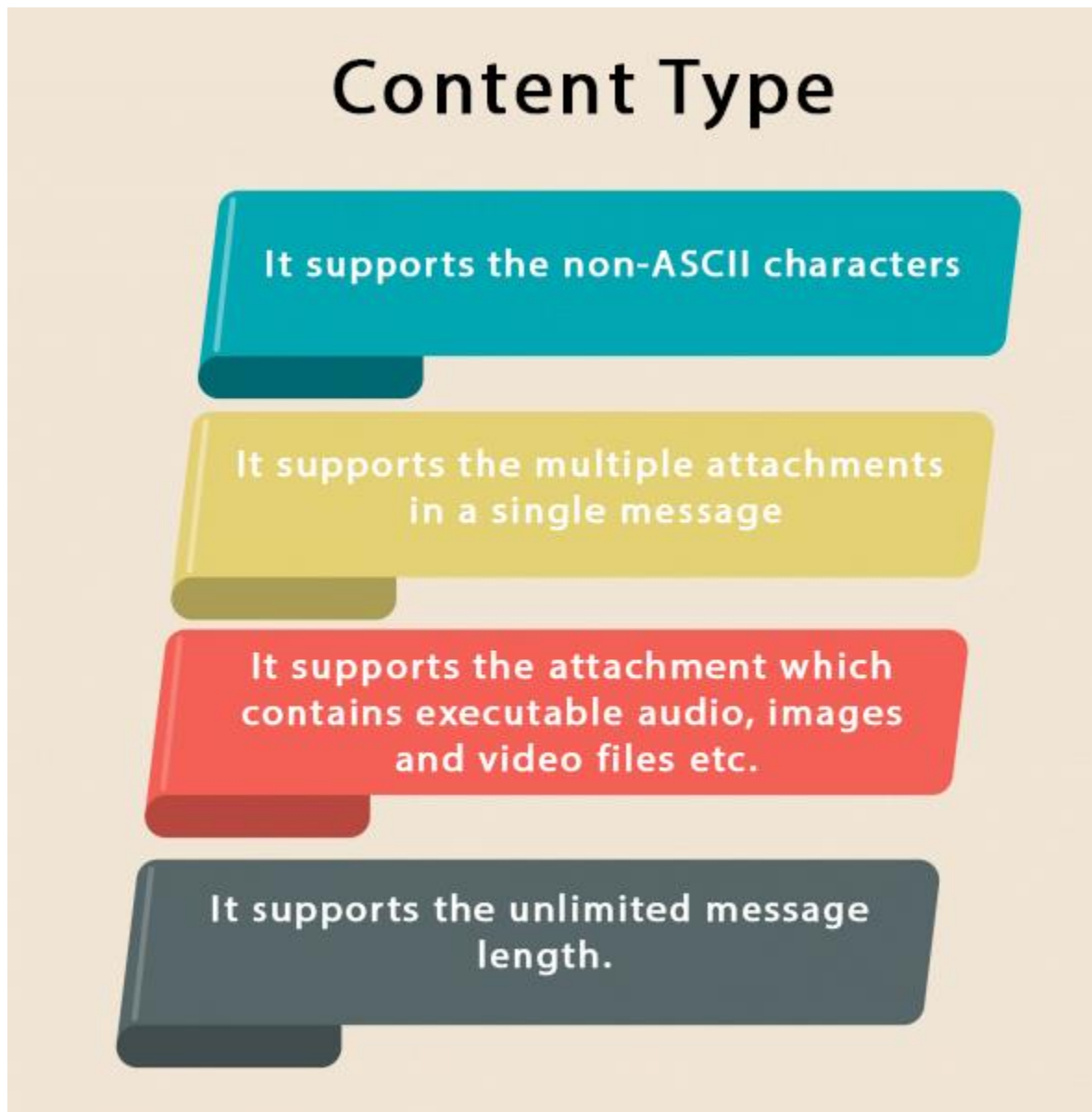
Content Type is also known as **MIME (Multipurpose internet Mail Extension)**Type. It is a **HTTP header** that provides the description about what are you sending to the browser.

MIME is an internet standard that is used for extending the limited capabilities of email by allowing the insertion of sounds, images and text in a message.

The features provided by MIME to the email services are as given below:

- It supports the non-ASCII characters
- It supports the multiple attachments in a single message
- It supports the attachment which contains executable audio, images and video files etc.

- It supports the unlimited message length.



List of Content Types

There are many content types. The commonly used content types are given below:

- text/html
- text/plain
- application/msword
- application/vnd.ms-excel
- application/jar

- application/pdf
- application/octet-stream
- application/x-zip
- images/jpeg
- images/png
- images/gif
- audio/mp3
- video/mp4
- video/quicktime etc.

Servlet API

The `javax.servlet` and `javax.servlet.http` packages represent interfaces and classes for servlet api.

The **`javax.servlet`** package contains many interfaces and classes that are used by the servlet or web container. These are not specific to any protocol.

The **`javax.servlet.http`** package contains interfaces and classes that are responsible for http requests only.

Let's see what are the interfaces of `javax.servlet` package.

Interfaces in `javax.servlet` package

There are many interfaces in `javax.servlet` package. They are as follows:

1. `Servlet`
2. `ServletRequest`
3. `ServletResponse`
4. `RequestDispatcher`
5. `ServletConfig`
6. `ServletContext`
7. `SingleThreadModel`
8. `Filter`
9. `FilterConfig`
10. `FilterChain`
11. `ServletRequestListener`

12. ServletRequestAttributeListener
13. ServletContextListener
14. ServletContextAttributeListener

Classes in javax.servlet package

There are many classes in javax.servlet package. They are as follows:

1. GenericServlet
2. ServletInputStream
3. ServletOutputStream
4. ServletRequestWrapper
5. ServletResponseWrapper
6. ServletRequestEvent
7. ServletContextEvent
8. ServletRequestAttributeEvent
9. ServletContextAttributeEvent
10. ServletException
11. UnavailableException

Interfaces in javax.servlet.http package

There are many interfaces in javax.servlet.http package. They are as follows:

1. HttpServletRequest
2. HttpServletResponse
3. HttpSession
4. HttpSessionListener
5. HttpSessionAttributeListener
6. HttpSessionBindingListener
7. HttpSessionActivationListener
8. HttpSessionContext (deprecated now)

Classes in javax.servlet.http package

There are many classes in javax.servlet.http package. They are as follows:

1. HttpServlet
2. Cookie
3. HttpServletRequestWrapper
4. HttpServletResponseWrapper
5. HttpSessionEvent
6. HttpSessionBindingEvent
7. HttpUtils (deprecated now)

Servlet Interface

Servlet interface provides common behavior to all the servlets. Servlet interface defines methods that all servlets must implement.

Servlet interface needs to be implemented for creating any servlet (either directly or indirectly). It provides 3 life cycle methods that are used to initialize the servlet, to service the requests, and to destroy the servlet and 2 non-life cycle methods.

Methods of Servlet interface

There are 5 methods in Servlet interface. The init, service and destroy are the life cycle methods of servlet. These are invoked by the web container.

Method	Description
public void init(ServletConfig config)	initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.
public void service(ServletRequest request, ServletResponse response)	provides response for the incoming request. It is invoked at each request by the web container.
public void destroy()	is invoked only once and indicates that servlet is being destroyed.
public ServletConfig getServletConfig()	returns the object of ServletConfig.
public String getServletInfo()	returns information about servlet such as writer,

Servlet Example by implementing Servlet interface

Let's see the simple example of servlet by implementing the servlet interface.

File: First.java

```
1. import java.io.*;
2. import javax.servlet.*;
3.
4. public class First implements Servlet{
5.     ServletConfig config=null;
6.
7.     public void init(ServletConfig config){
8.         this.config=config;
9.         System.out.println("servlet is initialized");
10.    }
11.
12.    public void service(ServletRequest req,ServletResponse res)
13.    throws IOException,ServletException{
14.
15.        res.setContentType("text/html");
16.
17.        PrintWriter out=res.getWriter();
18.        out.print("<html><body>");
19.        out.print("<b>hello simple servlet</b>");
20.        out.print("</body></html>");
21.
22.    }
23.    public void destroy(){System.out.println("servlet is destroyed");}
24.    public ServletConfig getServletConfig(){return config;}
25.    public String getServletInfo(){return "copyright 2007-1010";}
26.
27. }
```

GenericServlet class

GenericServlet class implements **Servlet**, **ServletConfig** and **Serializable** interfaces. It provides the implementation of all the methods of these interfaces except the service method.

GenericServlet class can handle any type of request so it is protocol-independent.

You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

Methods of GenericServlet class

There are many methods in GenericServlet class. They are as follows:

1. **public void init(ServletConfig config)** is used to initialize the servlet.
2. **public abstract void service(ServletRequest request, ServletResponse response)** provides service for the incoming request. It is invoked at each time when user requests for a servlet.
3. **public void destroy()** is invoked only once throughout the life cycle and indicates that servlet is being destroyed.
4. **public ServletConfig getServletConfig()** returns the object of ServletConfig.
5. **public String getServletInfo()** returns information about servlet such as writer, copyright, version etc.
6. **public void init()** it is a convenient method for the servlet programmers, now there is no need to call super.init(config)
7. **public ServletContext getServletContext()** returns the object of ServletContext.
8. **public String getInitParameter(String name)** returns the parameter value for the given parameter name.
9. **public Enumeration getInitParameterNames()** returns all the parameters defined in the web.xml file.
10. **public String getServletName()** returns the name of the servlet object.
11. **public void log(String msg)** writes the given message in the servlet log file.
12. **public void log(String msg, Throwable t)** writes the explanatory message in the servlet log file and a stack trace.

Servlet Example by inheriting the GenericServlet class

Let's see the simple example of servlet by inheriting the GenericServlet class.

File: First.java

1. **import** java.io.*;

```
2. import javax.servlet.*;
3.
4. public class First extends GenericServlet{
5.     public void service(ServletRequest req,ServletResponse res)
6.     throws IOException,ServletException{
7.
8.         res.setContentType("text/html");
9.
10.        PrintWriter out=res.getWriter();
11.        out.print("<html><body>");
12.        out.print("<b>hello generic servlet</b>");
13.        out.print("</body> </html>");
14.
15.    }
16. }
```

HttpServlet class

The HttpServlet class extends the GenericServlet class and implements Serializable interface. It provides http specific methods such as doGet, doPost, doHead, doTrace etc.

Methods of HttpServlet class

There are many methods in HttpServlet class. They are as follows:

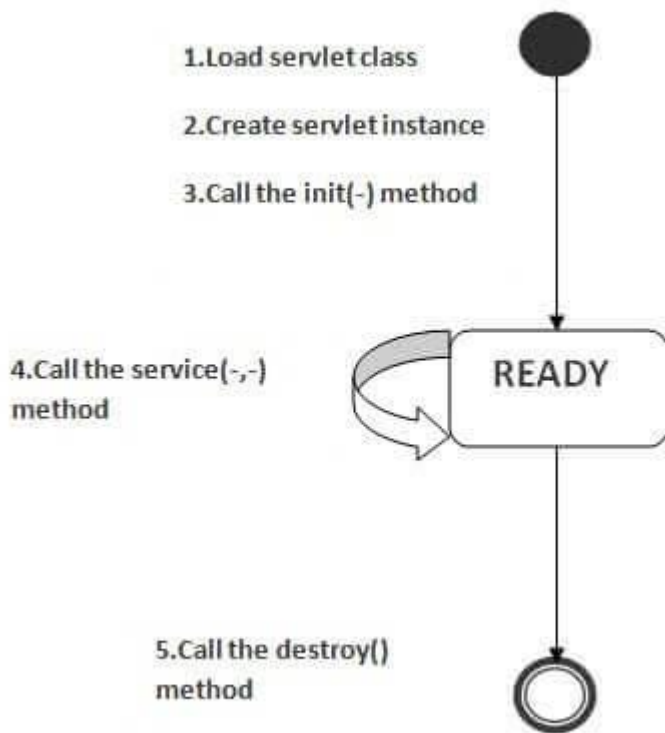
1. **public void service(ServletRequest req,ServletResponse res)** dispatches the request to the protected service method by converting the request and response object into http type.
2. **protected void service(HttpServletRequest req, HttpServletResponse res)** receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.
3. **protected void doGet(HttpServletRequest req, HttpServletResponse res)** handles the GET request. It is invoked by the web container.
4. **protected void doPost(HttpServletRequest req, HttpServletResponse res)** handles the POST request. It is invoked by the web container.
5. **protected void doHead(HttpServletRequest req, HttpServletResponse res)** handles the HEAD request. It is invoked by the web container.

6. **protected void doOptions(HttpServletRequest req, HttpServletResponse res)** handles the OPTIONS request. It is invoked by the web container.
7. **protected void doPut(HttpServletRequest req, HttpServletResponse res)** handles the PUT request. It is invoked by the web container.
8. **protected void doTrace(HttpServletRequest req, HttpServletResponse res)** handles the TRACE request. It is invoked by the web container.
9. **protected void doDelete(HttpServletRequest req, HttpServletResponse res)** handles the DELETE request. It is invoked by the web container.
10. **protected long getLastModified(HttpServletRequest req)** returns the time when HttpServletRequest was last modified since midnight January 1, 1970 GMT.

Life Cycle of a Servlet (Servlet Life Cycle)

The web container maintains the life cycle of a servlet instance. Let's see the life cycle of the servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



As displayed in the above diagram, there are three states of a servlet: new, ready and end. The servlet is in new state if servlet instance is created. After invoking the `init()` method, Servlet comes in the ready state. In the ready state, servlet performs all the tasks. When the web container invokes the `destroy()` method, it shifts to the end state.

1) Servlet class is loaded

The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

2) Servlet instance is created

The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

3) init method is invoked

The web container calls the `init` method only once after creating the servlet instance. The `init` method is used to initialize the servlet. It is the life cycle method of the `javax.servlet.Servlet` interface. Syntax of the

init method is given below:

1. **public void** init(ServletConfig config) **throws** ServletException
-

4) service method is invoked

The web container calls the service method each time when request for the servlet is received. If servlet is not initialized, it follows the first three steps as described above then calls the service method. If servlet is initialized, it calls the service method. Notice that servlet is initialized only once. The syntax of the service method of the Servlet interface is given below:

1. **public void** service(ServletRequest request, ServletResponse response)
 2. **throws** ServletException, IOException
-

5) destroy method is invoked

The web container calls the destroy method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc. The syntax of the destroy method of the Servlet interface is given below:

1. **public void** destroy()

Steps to create a servlet example

There are given 6 steps to create a **servlet example**. These steps are required for all the servers.

The servlet example can be created by three ways:

1. By implementing Servlet interface,
2. By inheriting GenericServlet class, (or)
3. By inheriting HttpServlet class

The mostly used approach is by extending HttpServlet because it provides http request specific method such as doGet(), doPost(), doHead() etc.

Here, we are going to use **apache tomcat server** in this example. The steps are as follows:

1. Create a directory structure
2. Create a Servlet
3. Compile the Servlet

4. Create a deployment descriptor
5. Start the server and deploy the project
6. Access the servlet

[download this example of servlet](#)

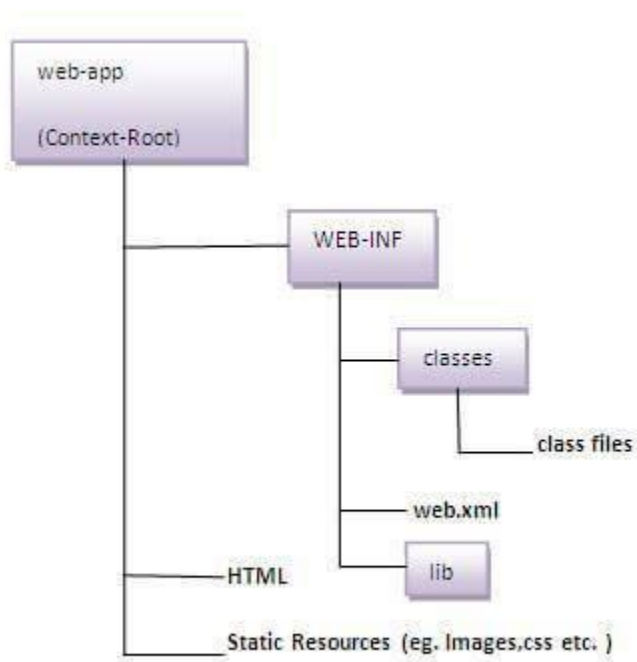
[download example of servlet by extending GenericServlet](#)

[download example of servlet by implementing Servlet interface](#)

1)Create a directory structures

The **directory structure** defines that where to put the different types of files so that web container may get the information and respond to the client.

The Sun Microsystems defines a unique standard to be followed by all the server vendors. Let's see the directory structure that must be followed to create the servlet.



As you can see that the servlet class file must be in the classes folder. The web.xml file must be under the WEB-INF folder.

2)Create a Servlet

There are three ways to create the servlet.

1. By implementing the Servlet interface
2. By inheriting the GenericServlet class
3. By inheriting the HttpServlet class

The HttpServlet class is widely used to create the servlet because it provides methods to handle http requests such as doGet(), doPost, doHead() etc.

In this example we are going to create a servlet that extends the HttpServlet class. In this example, we are inheriting the HttpServlet class and providing the implementation of the doGet() method. Notice that get request is the default request.

DemoServlet.java

```
1. import javax.servlet.http.*;
2. import javax.servlet.*;
3. import java.io.*;
4. public class DemoServlet extends HttpServlet{
5.     public void doGet(HttpServletRequest req,HttpServletResponse res)
6.     throws ServletException,IOException
7.     {
8.         res.setContentType("text/html");//setting the content type
9.         PrintWriter pw=res.getWriter();//get the stream to write the data
10.
11.         //writing html in the stream
12.         pw.println("<html> <body>");
13.         pw.println("Welcome to servlet");
14.         pw.println("</body> </html>");
15.
16.         pw.close();//closing the stream
17. }}
```

3)Compile the servlet

For compiling the Servlet, jar file is required to be loaded. Different Servers provide different jar files:

Jar file	Server
1) servlet-api.jar	Apache Tomcat
2) weblogic.jar	Weblogic
3) javaee.jar	Glassfish
4) javaee.jar	JBoss

Two ways to load the jar file

1. set classpath
2. paste the jar file in JRE/lib/ext folder

Put the java file in any folder. After compiling the java file, paste the class file of servlet in **WEB-INF/classes** directory.

4) Create the deployment descriptor (web.xml file)

The **deployment descriptor** is an xml file, from which Web Container gets the information about the servlet to be invoked.

The web container uses the Parser to get the information from the web.xml file. There are many xml parsers such as SAX, DOM and Pull.

There are many elements in the web.xml file. Here is given some necessary elements to run the simple servlet program.

web.xml file

1. **<web-app>**
- 2.
3. **<servlet>**
4. **<servlet-name>**sonoojaiswal**</servlet-name>**

5. `<servlet-class>DemoServlet</servlet-class>`
6. `</servlet>`
- 7.
8. `<servlet-mapping>`
9. `<servlet-name>sonoojaiswal</servlet-name>`
10. `<url-pattern>/welcome</url-pattern>`
11. `</servlet-mapping>`
- 12.
13. `</web-app>`

Description of the elements of web.xml file

There are too many elements in the web.xml file. Here is the illustration of some elements that is used in the above web.xml file. The elements are as follows:

<web-app> represents the whole application.

<servlet> is sub element of <web-app> and represents the servlet.

<servlet-name> is sub element of <servlet> represents the name of the servlet.

<servlet-class> is sub element of <servlet> represents the class of the servlet.

<servlet-mapping> is sub element of <web-app>. It is used to map the servlet.

<url-pattern> is sub element of <servlet-mapping>. This pattern is used at client side to invoke the servlet.

5)Start the Server and deploy the project

To start Apache Tomcat server, double click on the startup.bat file under apache-tomcat/bin directory.

One Time Configuration for Apache Tomcat Server

You need to perform 2 tasks:

1. set JAVA_HOME or JRE_HOME in environment variable (It is required to start server).
2. Change the port number of tomcat (optional). It is required if another server is running on same port (8080).

1) How to set JAVA_HOME in environment variable?

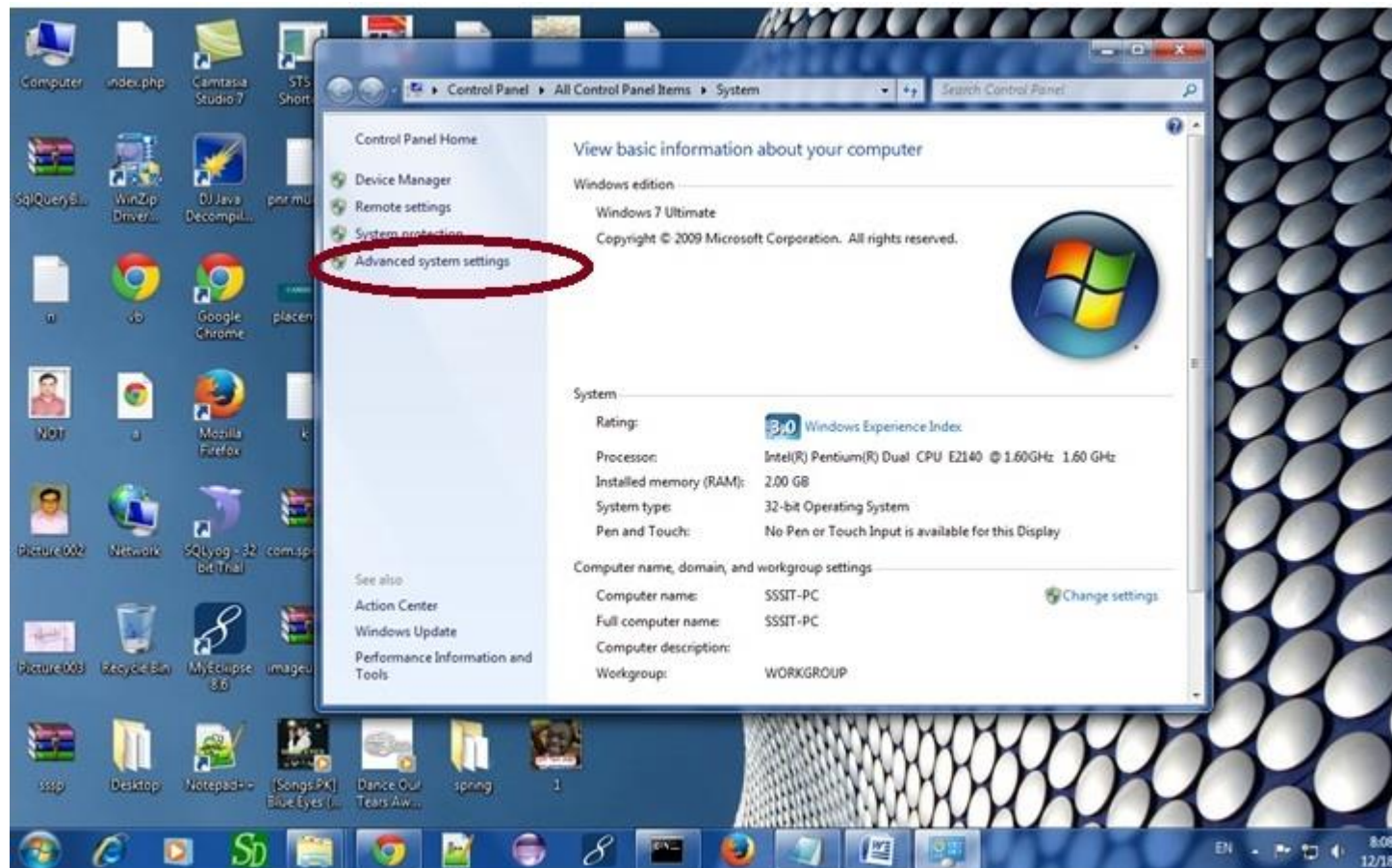
To start Apache Tomcat server JAVA_HOME and JRE_HOME must be set in Environment variables.

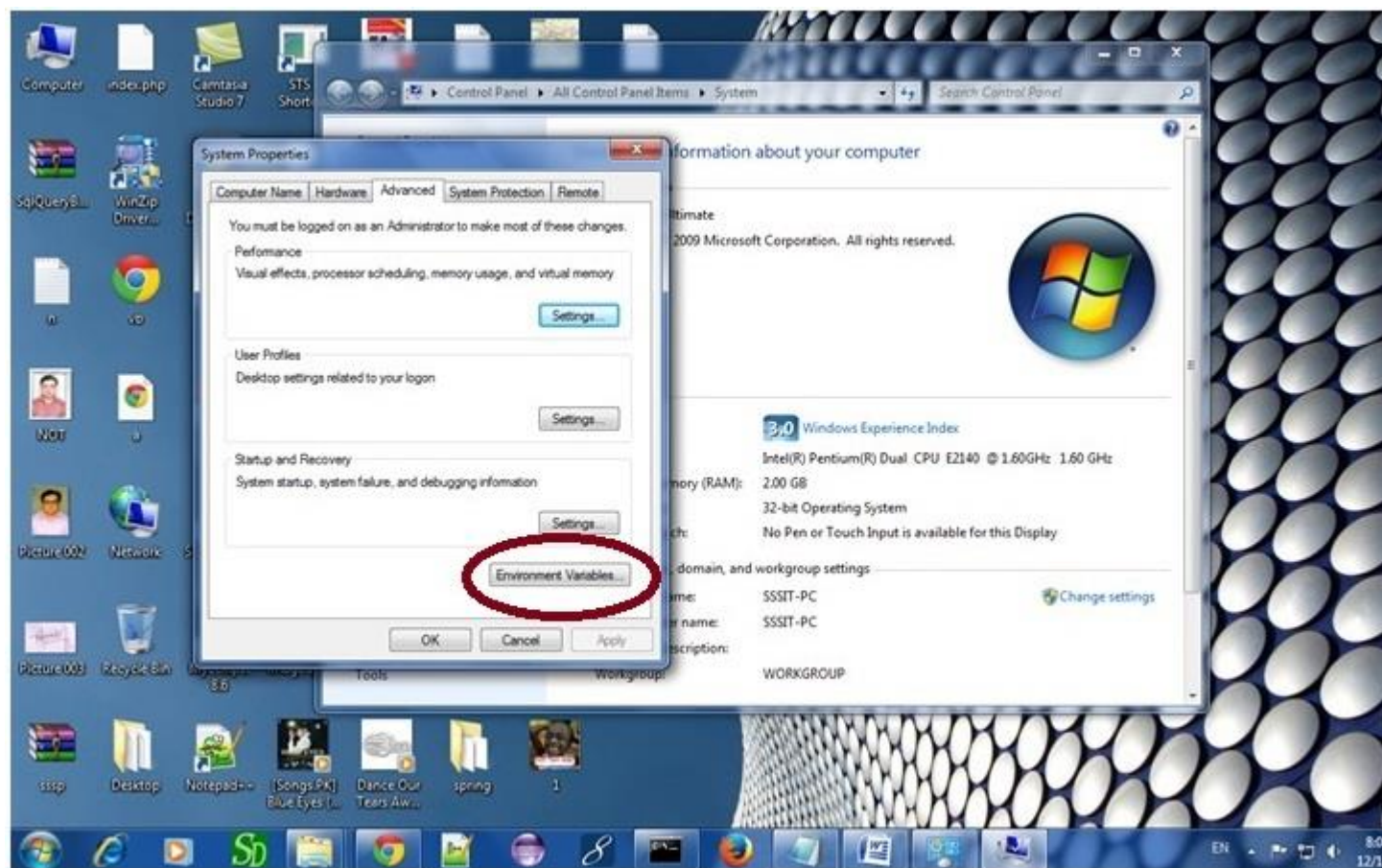
Go to My Computer properties -> Click on advanced tab then environment variables -> Click on the new tab of user variable -> Write JAVA_HOME in variable name and paste the path of jdk folder in variable value -> ok -> ok -> ok.

Go to My Computer properties:

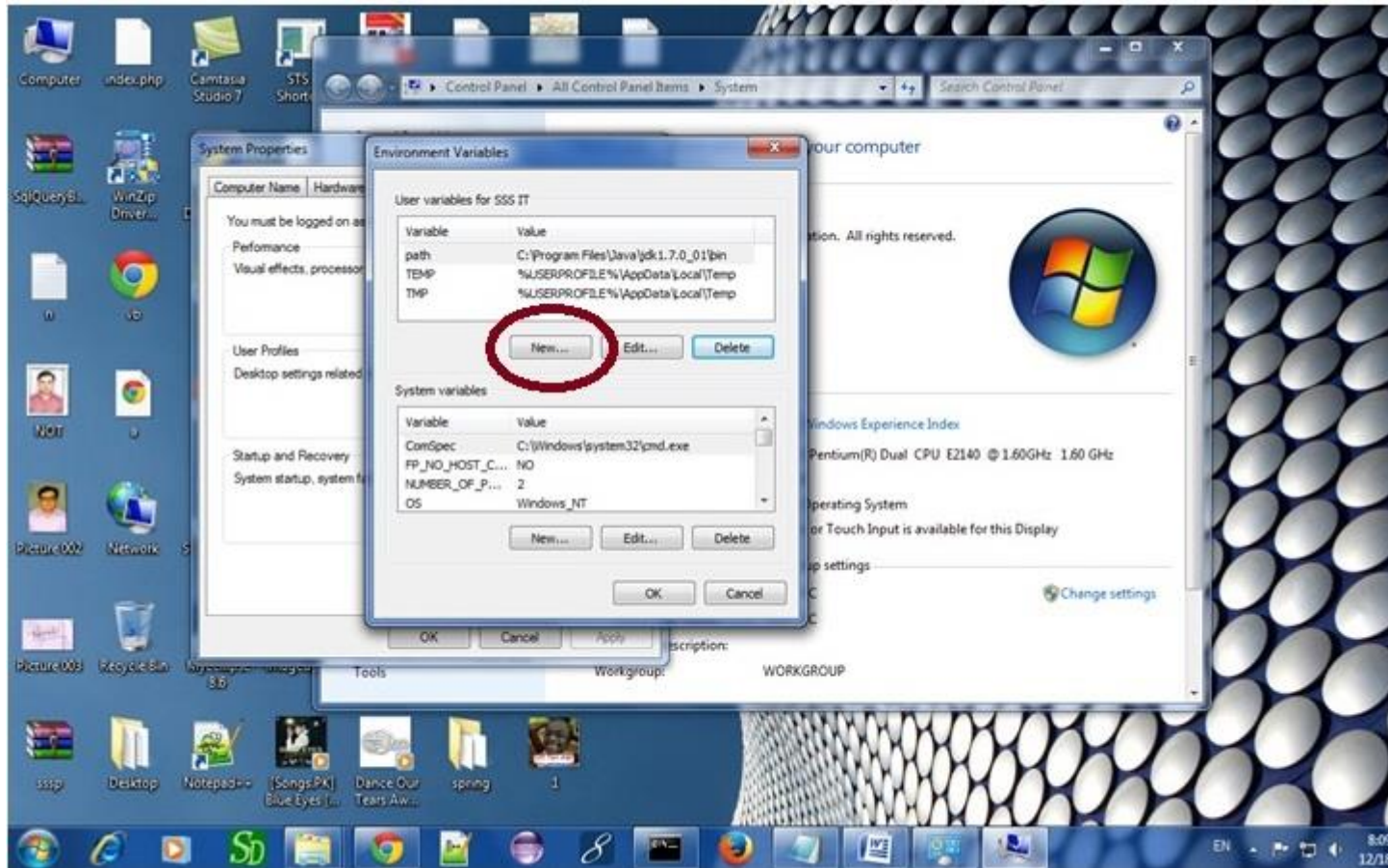


Click on advanced system settings tab then environment variables:

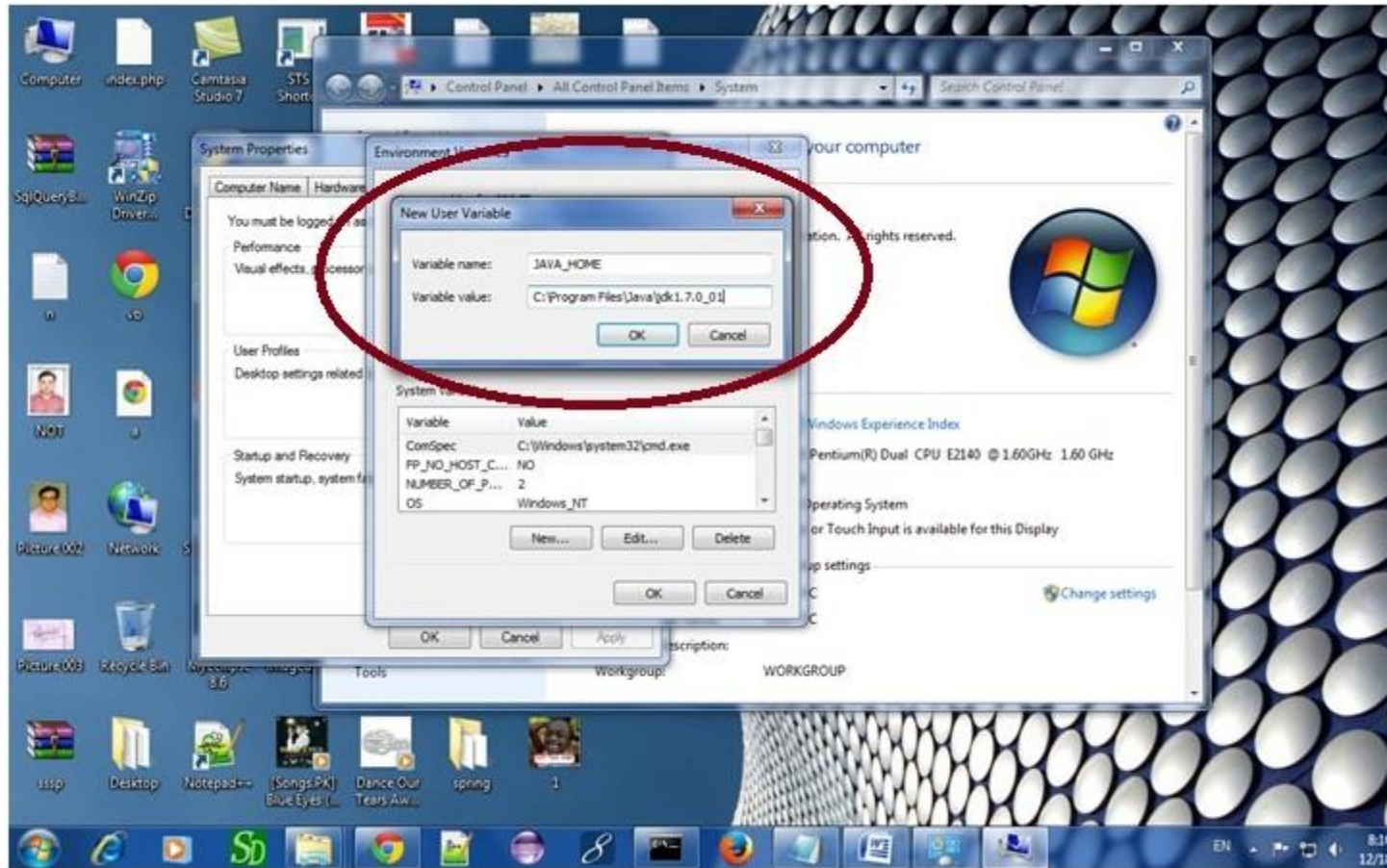




Click on the new tab of user variable or system variable:



Write JAVA_HOME in variable name and paste the path of jdk folder in variable value:



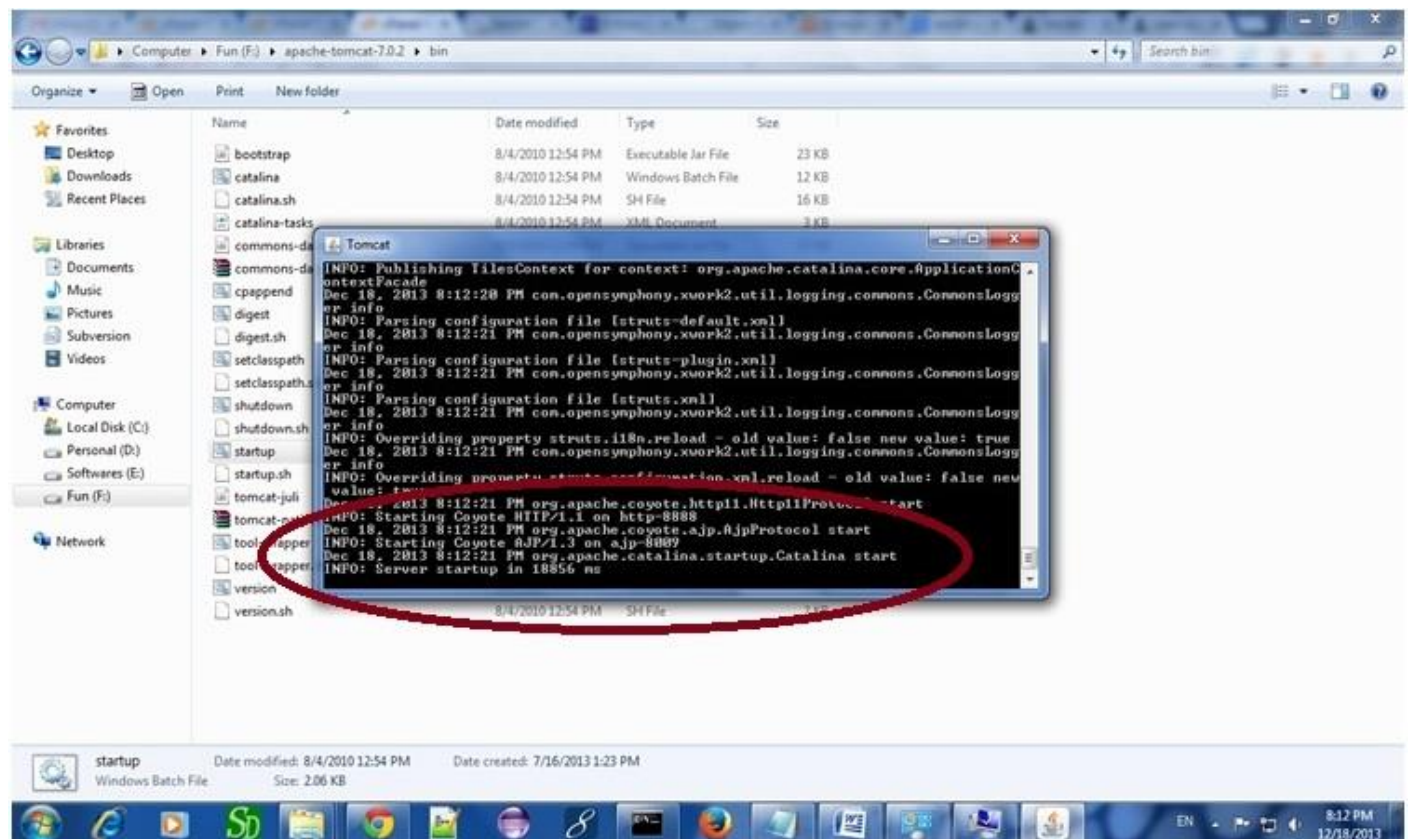
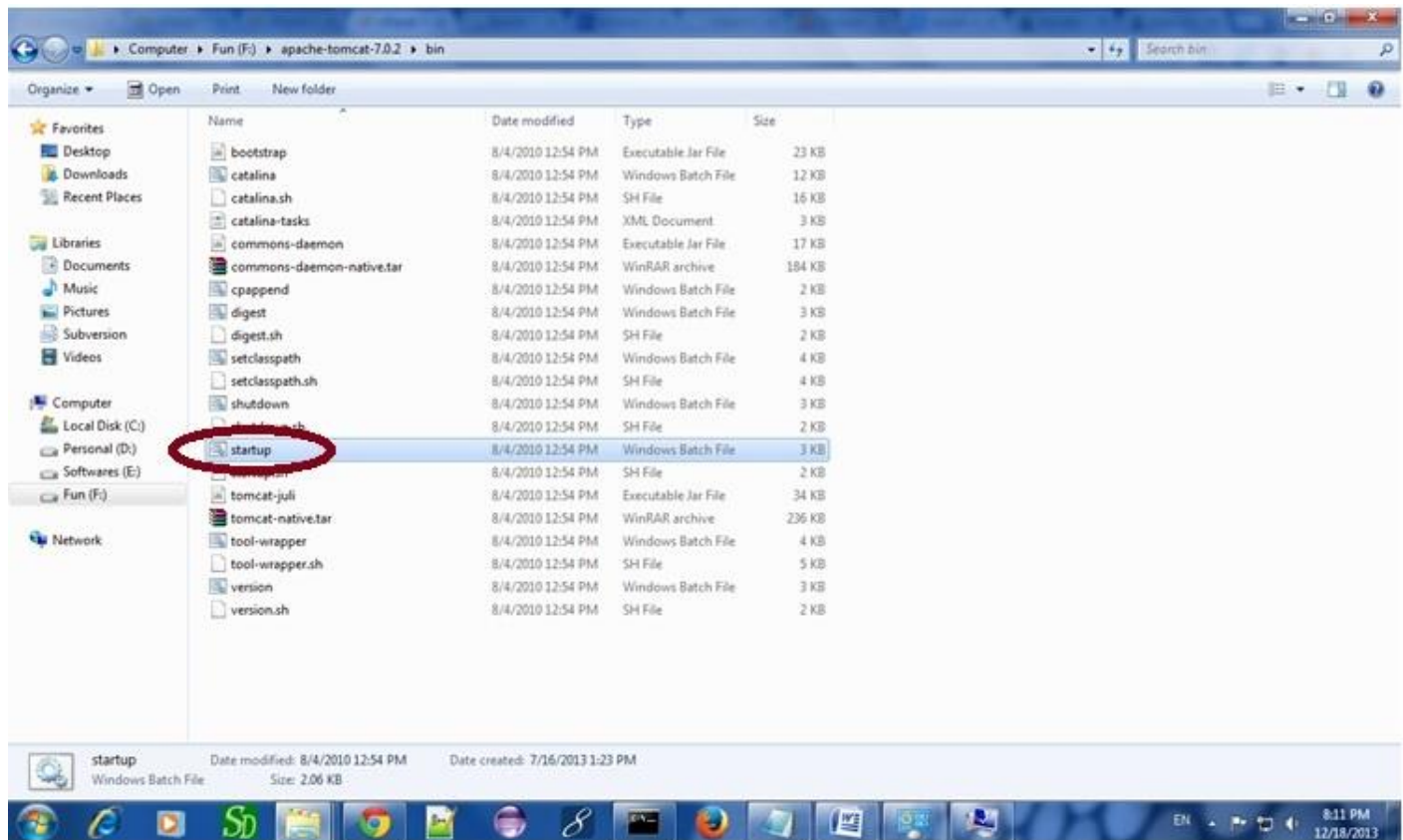
There must not be semicolon (;) at the end of the path.

After setting the JAVA_HOME double click on the startup.bat file in apache tomcat/bin.

Note: There are two types of tomcat available:

1. Apache tomcat that needs to extract only (no need to install)
2. Apache tomcat that needs to install

It is the example of apache tomcat that needs to extract only.



Now server is started successfully.

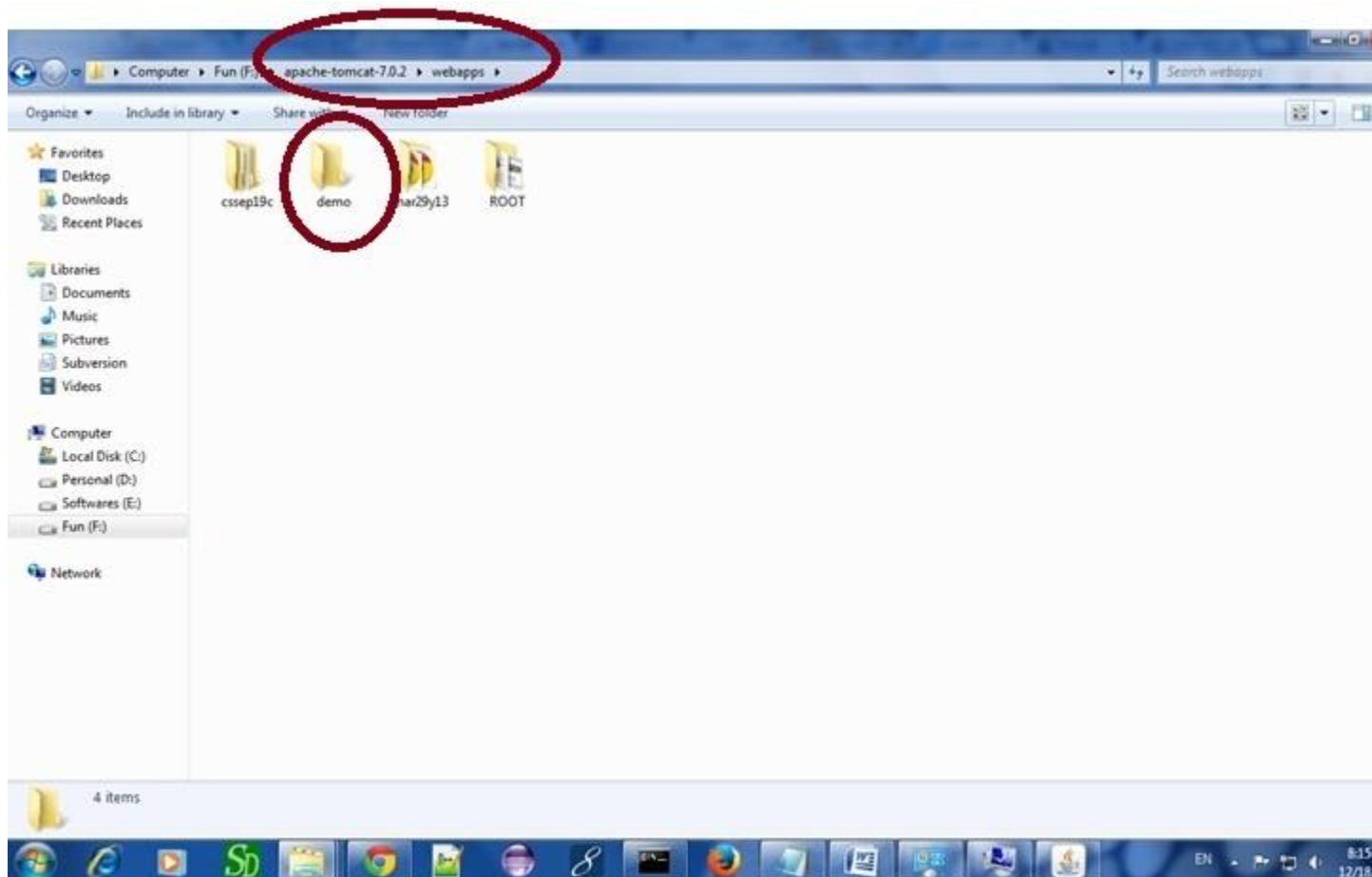
2) How to change port number of apache tomcat

Changing the port number is required if there is another server running on the same system with same port number. Suppose you have installed oracle, you need to change the port number of apache tomcat because both have the default port number 8080.

Open **server.xml** file in notepad. It is located inside the **apache-tomcat/conf** directory . Change the Connector port = 8080 and replace 8080 by any four digit number instead of 8080. Let us replace it by 9999 and save this file.

5) How to deploy the servlet project

Copy the project and paste it in the webapps folder under apache tomcat.



But there are several ways to deploy the project. They are as follows:

- By copying the context(project) folder into the webapps directory
- By copying the war folder into the webapps directory
- By selecting the folder path from the server
- By selecting the war file from the server

Here, we are using the first approach.

You can also create war file, and paste it inside the webapps directory. To do so, you need to use jar tool to create the war file. Go inside the project directory (before the WEB-INF), then write:

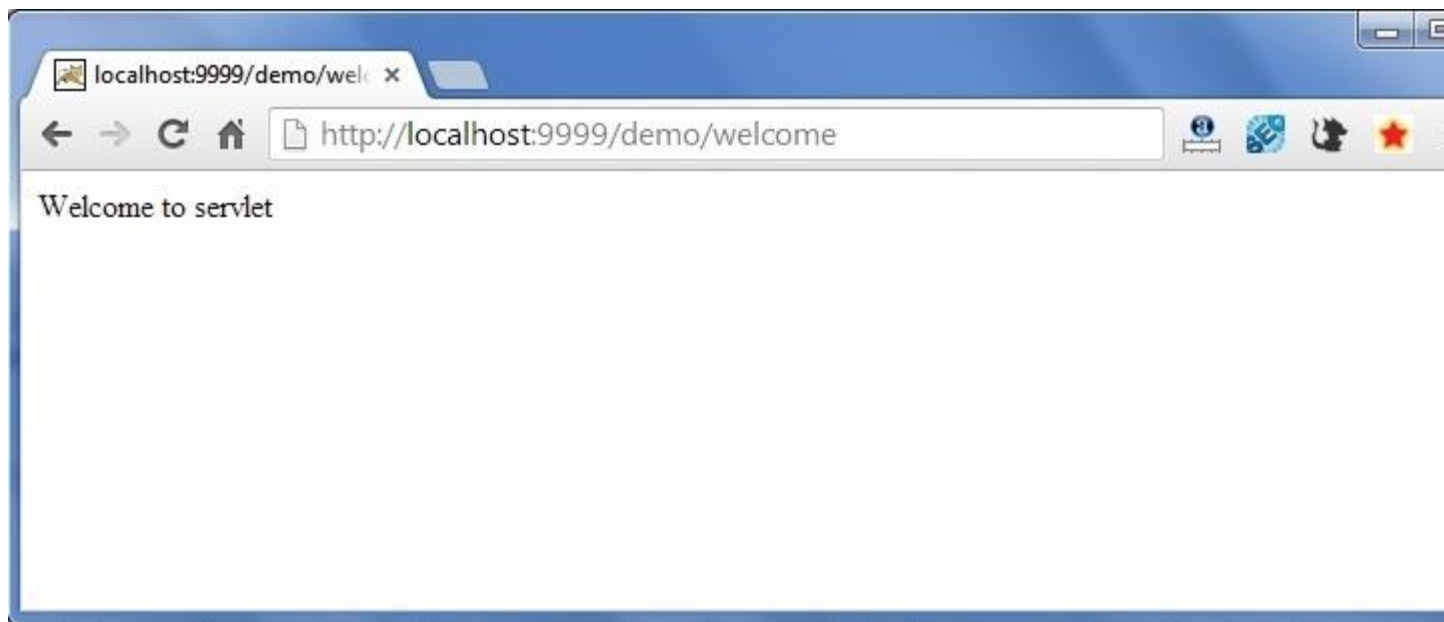
1. `projectfolder> jar cvf myproject.war *`

Creating war file has an advantage that moving the project from one location to another takes less time.

6) How to access the servlet

Open browser and write `http://hostname:portno/contextroot/urlpatternofservlet`. For example:

1. `http://localhost:9999/demo/welcome`



[download this example of servlet \(using notepad\)](#)
[download example of servlet by extending GenericServlet](#)
[download example of servlet by implementing Servlet interface](#)

How Servlet works?

It is important to learn how servlet works for understanding the servlet well. Here, we are going to get the internal detail about the first servlet program.

The server checks if the servlet is requested **for the first time**.

If yes, web container does the following tasks:

- loads the servlet class.
- instantiates the servlet class.
- calls the init method passing the ServletConfig object

else

- calls the service method passing request and response objects

The web container calls the destroy method when it needs to remove the servlet such as at time of stopping server or undeploying the project.

How web container handles the servlet request?

The web container is responsible to handle the request. Let's see how it handles the request.

- maps the request with the servlet in the web.xml file.
- creates request and response objects for this request
- calls the service method on the thread
- The public service method internally calls the protected service method
- The protected service method calls the doGet method depending on the type of request.
- The doGet method generates the response and it is passed to the client.
- After sending the response, the web container deletes the request and response objects. The thread is contained in the thread pool or deleted depends on the server implementation.

What is written inside the public service method?

The public service method converts the ServletRequest object into the HttpServletRequest type and ServletResponse object into the HttpServletResponse type. Then, calls the service method passing these objects. Let's see the internal code:

```
1. public void service(ServletRequest req, ServletResponse res)
2.     throws ServletException, IOException
3. {
4.     HttpServletRequest request;
5.     HttpServletResponse response;
6.     try
7.     {
8.         request = (HttpServletRequest)req;
9.         response = (HttpServletResponse)res;
10.    }
11.    catch(ClassCastException e)
12.    {
13.        throw new ServletException("non-HTTP request or response");
14.    }
15.    service(request, response);
16. }
```

What is written inside the protected service method?

The protected service method checks the type of request, if request type is get, it calls doGet method, if request type is post, it calls doPost method, so on. Let's see the internal code:

```
1. protected void service(HttpServletRequest req, HttpServletResponse resp)
2.     throws ServletException, IOException
3. {
4.     String method = req.getMethod();
5.     if(method.equals("GET"))
6.     {
7.         long lastModified = getLastModified(req);
8.         if(lastModified == -1L)
9.         {
10.            doGet(req, resp);
11.        }
```

12.
13. `//rest of the code`
14. `}`
15. `}`

War File

A **war (web archive) File** contains files of a web project. It may have servlet, xml, jsp, image, html, css, js etc. files.

Here, we will discuss what is war file, how to create war file, how to deploy war file and how to extract war file.

What is war file?

web archive (war) file contains all the contents of a web application. It reduces the time duration for transferring file.

Advantage of war file

saves time: The war file combines all the files into a single unit. So it takes less time while transferring file from client to server.

How to create war file?

To create war file, you need to use **jar tool** of JDK. You need to use **-c** switch of jar, to create the war file.

Go inside the project directory of your project (outside the WEB-INF), then write the following command:

1. `jar -cvf projectname.war *`

Here, **-c** is used to create file, **-v** to generate the verbose output and **-f** to specify the archive file name.

The ***** (**asterisk**) **symbol** signifies that all the files of this directory (including sub directory).

How to deploy the war file?

There are two ways to deploy the war file.

1. By server console panel
2. By manually having the war file in specific folder of server.

If you want to deploy the war file in **apache tomcat** server manually, go to the **webapps** directory of apache tomcat and paste the war file here.

Now, you are able to access the web project through browser.

How to extract war file manually?

To extract the war file, you need to use **-x switch** of **jar tool** of JDK. Let's see the command to extract the war file.

1. `jar -xvf projectname.war`

welcome-file-list in web.xml

The **welcome-file-list** element of **web-app**, is used to define a list of welcome files. Its sub element is **welcome-file** that is used to define the welcome file.

A **welcome file** is the file that is invoked automatically by the server, if you don't specify any file name.

By default server looks for the welcome file in following order:

1. welcome-file-list in web.xml
2. index.html
3. index.htm
4. index.jsp

If none of these files are found, server renders 404 error.

welcome-file-list in web.xml



If you have specified welcome-file in web.xml, and all the files index.html, index.htm and index.jsp exists, priority goes to welcome-file.

If welcome-file-list entry doesn't exist in web.xml file, priority goes to index.html file then index.htm and at last index.jsp file.

Let's see the web.xml file that defines the welcome files.

web.xml

1. **<web-app>**
2.
- 3.
4. **<welcome-file-list>**

5. `<welcome-file>home.html</welcome-file>`
6. `<welcome-file>default.html</welcome-file>`
7. `</welcome-file-list>`
8. `</web-app>`

Now, home.html and default.html will be the welcome files.

If you have the welcome file, you can directly invoke the project as given below:

1. `http://localhost:8888/myproject`

As you can see, we have not specified any file name after the project.

load on startup in web.xml

The **load-on-startup** element of **web-app** loads the servlet at the time of deployment or server start if value is positive. It is also known as **pre initialization of servlet**.

You can pass positive and negative value for the servlet.

Advantage of load-on-startup element

As you know well, servlet is loaded at first request. That means it consumes more time at first request. If you specify the load-on-startup in web.xml, servlet will be loaded at project deployment time or server start. So, it will take **less time** for responding to first request.

Passing positive value

If you pass the positive value, the lower integer value servlet will be loaded before the higher integer value servlet. In other words, container loads the servlets in ascending integer value. The 0 value will be loaded first then 1, 2, 3 and so on.

Let's try to understand it by the example given below:

web.xml

1. `<web-app>`
2.
- 3.
4. `<servlet>`

```
5.  <servlet-name>servlet1</servlet-name>
6.  <servlet-class>com.javatpoint.FirstServlet</servlet-class>
7.  <load-on-startup>0</load-on-startup>
8.  </servlet>
9.
10. <servlet>
11.  <servlet-name>servlet2</servlet-name>
12.  <servlet-class>com.javatpoint.SecondServlet</servlet-class>
13.  <load-on-startup>1</load-on-startup>
14. </servlet>
15.
16. ...
17. </web-app>
```

There are defined 2 servlets, both servlets will be loaded at the time of project deployment or server start. But, servlet1 will be loaded first then servlet2.

Passing negative value

If you pass the negative value, servlet will be loaded at request time, at first request.

ServletRequest Interface

An object of ServletRequest is used to provide the client request information to a servlet such as content type, content length, parameter names and values, header informations, attributes etc.

Methods of ServletRequest interface

There are many methods defined in the ServletRequest interface. Some of them are as follows:

Method	Description
public String getParameter(String name)	is used to obtain the value of a parameter by name.
public String[] getParameterValues(String name)	returns an array of String containing all values of given parameter name. It is mainly used to obtain values of a Multi select list box.
java.util.Enumeration getParameterNames()	returns an enumeration of all of the request parameter names.
public int getContentLength()	Returns the size of the request entity data, or -1 if not known.
public String getCharacterEncoding()	Returns the character set encoding for the input of this request.
public String getContentType()	Returns the Internet Media Type of the request entity data, or null if not known.
public ServletInputStream getInputStream() throws IOException	Returns an input stream for reading binary data in the request body.
public abstract String getServerName()	Returns the host name of the server that received the request.
public int getServerPort()	Returns the port number on which this request was

	received.
--	-----------

Example of ServletRequest to display the name of the user

In this example, we are displaying the name of the user in the servlet. For this purpose, we have used the `getParameter` method that returns the value for the given request parameter name.

index.html

1. `<form action="welcome" method="get">`
2. Enter your name

3. `<input type="submit" value="login">`
4. `</form>`

DemoServ.java

1. **import** javax.servlet.http.*;
2. **import** javax.servlet.*;
3. **import** java.io.*;
4. **public class** DemoServ **extends** HttpServlet{
5. **public void** doGet(HttpServletRequest req, HttpServletResponse res)
6. **throws** ServletException, IOException
7. {
8. res.setContentType("text/html");
9. PrintWriter pw=res.getWriter();
- 10.
11. String name=req.getParameter("name");//will return value
12. pw.println("Welcome " +name);
- 13.
14. pw.close();
15. }}

RequestDispatcher in Servlet

The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the way of servlet collaboration.

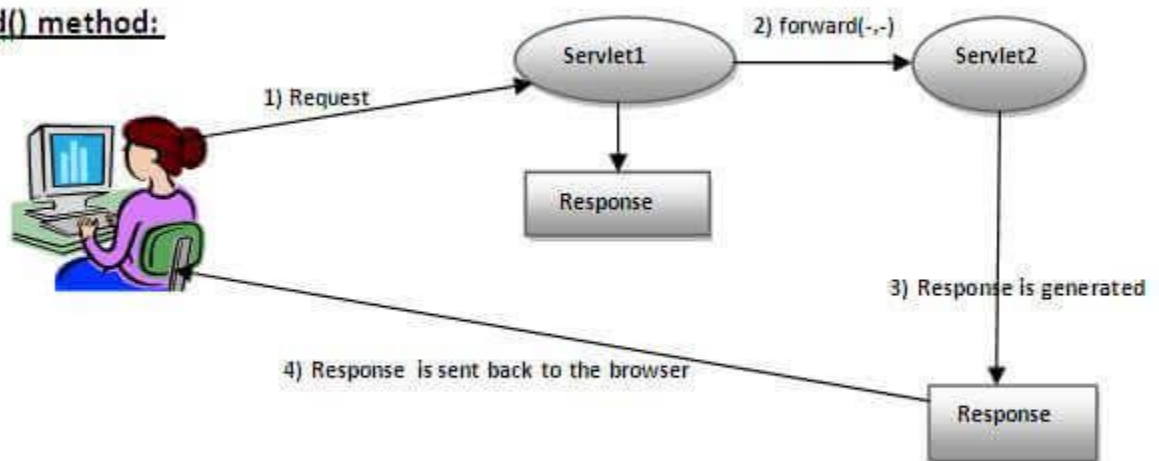
There are two methods defined in the RequestDispatcher interface.

Methods of RequestDispatcher interface

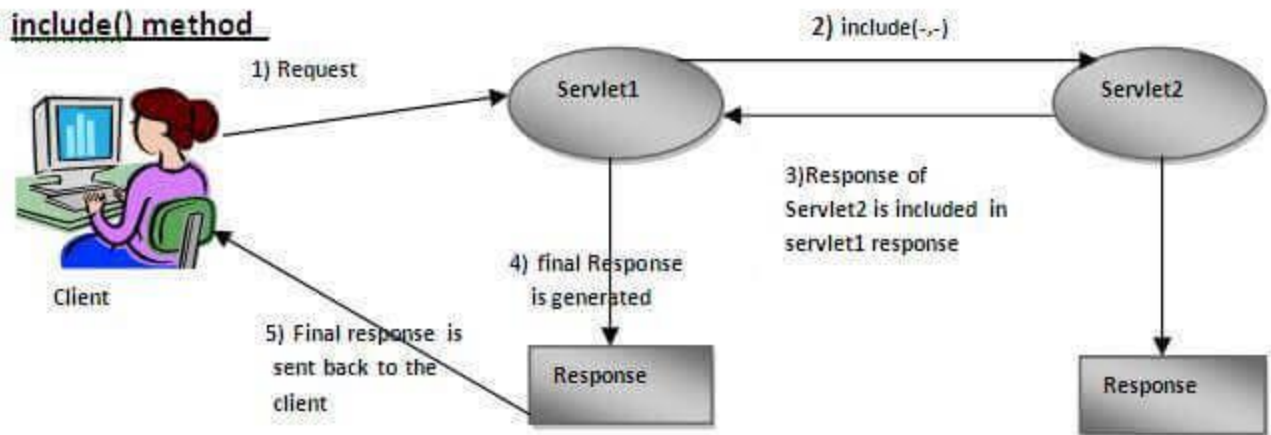
The RequestDispatcher interface provides two methods. They are:

1. **public void forward(ServletRequest request, ServletResponse response) throws ServletException, java.io.IOException:** Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.
2. **public void include(ServletRequest request, ServletResponse response) throws ServletException, java.io.IOException:** Includes the content of a resource (servlet, JSP page, or HTML file) in the response.

forward() method:



As you see in the above figure, response of second servlet is sent to the client. Response of the first servlet is not displayed to the user.



As you can see in the above figure, response of second servlet is included in the response of the first servlet that is being sent to the client.

How to get the object of RequestDispatcher

The `getRequestDispatcher()` method of `ServletRequest` interface returns the object of `RequestDispatcher`. Syntax:

Syntax of `getRequestDispatcher` method

1. **public** `RequestDispatcher` `getRequestDispatcher`(String resource);

Example of using `getRequestDispatcher` method

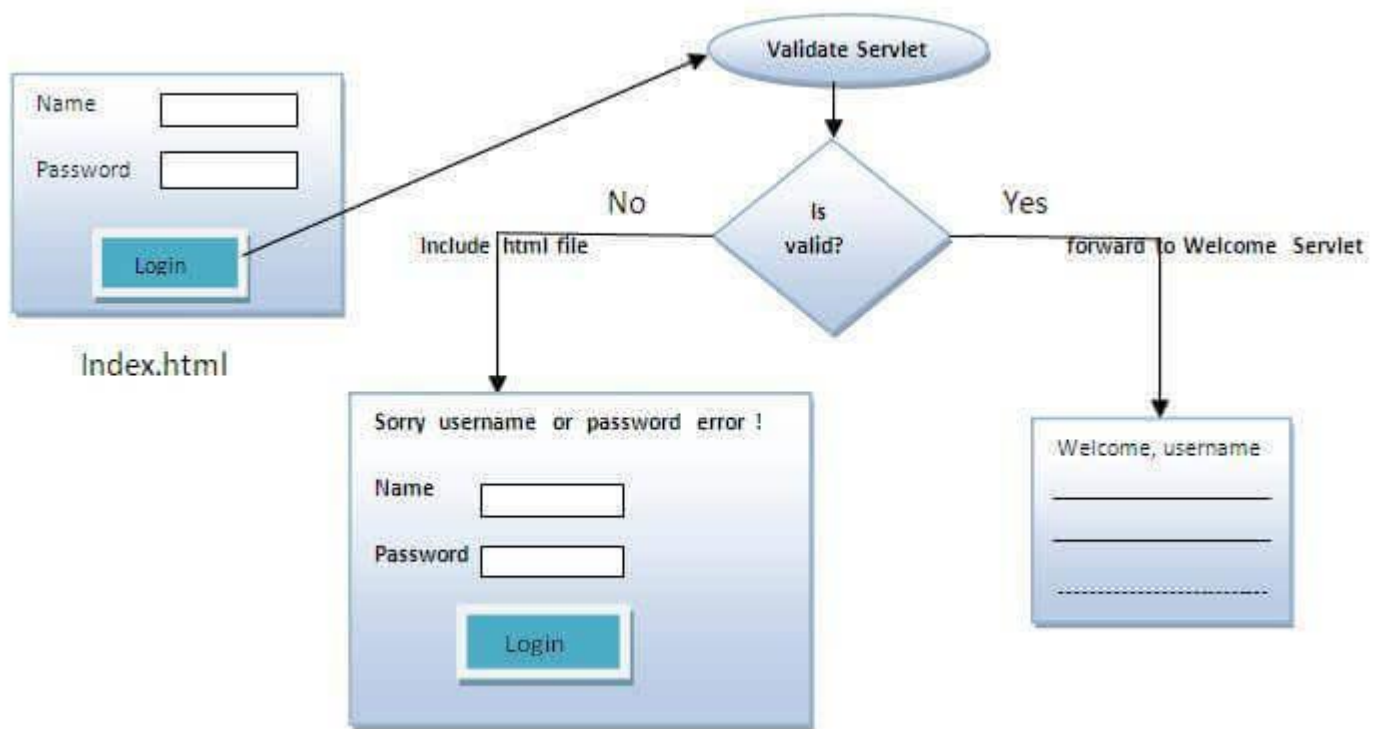
1. `RequestDispatcher rd=request.getRequestDispatcher("servlet2");`
2. *//servlet2 is the url-pattern of the second servlet*
- 3.
4. `rd.forward(request, response);`*//method may be include or forward*

Example of RequestDispatcher interface

In this example, we are validating the password entered by the user. If password is servlet, it will forward the request to the `WelcomeServlet`, otherwise will show an error message: sorry

username or password error!. In this program, we are checking for hardcoded information. But you can check it to the database also that we will see in the development chapter. In this example, we have created following files:

- **index.html file:** for getting input from the user.
- **Login.java file:** a servlet class for processing the response. If password is correct, it will forward the request to the welcome servlet.
- **WelcomeServlet.java file:** a servlet class for displaying the welcome message.
- **web.xml file:** a deployment descriptor file that contains the information about the servlet.



index.html

1. `<form action="servlet1" method="post">`
 2. `Name:<input type="text" name="userName"/> <br/>`
 3. `Password:<input type="password" name="userPass"/> <br/>`
 4. `<input type="submit" value="login"/>`
 5. `</form>`
-

Login.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class Login extends HttpServlet {
7.
8.     public void doPost(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.
11.         response.setContentType("text/html");
12.         PrintWriter out = response.getWriter();
13.
14.         String n=request.getParameter("userName");
15.         String p=request.getParameter("userPass");
16.
17.         if(p.equals("servlet")){
18.             RequestDispatcher rd=request.getRequestDispatcher("servlet2");
19.             rd.forward(request, response);
20.         }
21.         else{
22.             out.print("Sorry UserName or Password Error!");
23.             RequestDispatcher rd=request.getRequestDispatcher("/index.html");
24.             rd.include(request, response);
25.
26.         }
27.     }
28.
29. }
```

WelcomeServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5. public class WelcomeServlet extends HttpServlet {
```

```
6.
7.  public void doPost(HttpServletRequest request, HttpServletResponse response)
8.      throws ServletException, IOException {
9.
10.     response.setContentType("text/html");
11.     PrintWriter out = response.getWriter();
12.
13.     String n=request.getParameter("userName");
14.     out.print("Welcome "+n);
15. }
16.
17. }
```

web.xml

```
1. <web-app>
2.   <servlet>
3.     <servlet-name>Login</servlet-name>
4.     <servlet-class>Login</servlet-class>
5.   </servlet>
6.   <servlet>
7.     <servlet-name>WelcomeServlet</servlet-name>
8.     <servlet-class>WelcomeServlet</servlet-class>
9.   </servlet>
10.
11.
12. <servlet-mapping>
13.   <servlet-name>Login</servlet-name>
14.   <url-pattern>/servlet1</url-pattern>
15. </servlet-mapping>
16. <servlet-mapping>
17.   <servlet-name>WelcomeServlet</servlet-name>
18.   <url-pattern>/servlet2</url-pattern>
19. </servlet-mapping>
20.
21. <welcome-file-list>
22.   <welcome-file>index.html</welcome-file>
23. </welcome-file-list>
```

SendRedirect in servlet

The **sendRedirect()** method of **HttpServletResponse** interface can be used to redirect response to another resource, it may be servlet, jsp or html file.

It accepts relative as well as absolute URL.

It works at client side because it uses the url bar of the browser to make another request. So, it can work inside and outside the server.

Difference between forward() and sendRedirect() method

There are many differences between the forward() method of RequestDispatcher and sendRedirect() method of HttpServletResponse interface. They are given below:

forward() method	sendRedirect() method
The forward() method works at server side.	The sendRedirect() method works at client side.
It sends the same request and response objects to another servlet.	It always sends a new request.
It can work within the server only.	It can be used within and outside the server.
Example: request.getRequestDispatcher("servlet2").forward(request,response);	Example: response.sendRedirect("servlet2");

Syntax of sendRedirect() method

1. **public void** sendRedirect(String URL)**throws** IOException;

Example of sendRedirect() method

1. `response.sendRedirect("http://www.javatpoint.com");`
-

Full example of sendRedirect method in servlet

In this example, we are redirecting the request to the google server. Notice that sendRedirect method works client side, that is why we can our request to anywhere. We can send our request within and outside the server.

DemoServlet.java

1. **import** java.io.*;
 2. **import** javax.servlet.*;
 3. **import** javax.servlet.http.*;
 - 4.
 5. **public class** DemoServlet **extends** HttpServlet{
 6. **public void** doGet(HttpServletRequest req,HttpServletResponse res)
 7. **throws** ServletException,IOException
 8. {
 9. res.setContentType("text/html");
 10. PrintWriter pw=res.getWriter();
 - 11.
 12. response.sendRedirect("http://www.google.com");
 - 13.
 14. pw.close();
 15. }}
-

Creating custom google search using sendRedirect

In this example, we are using sendRedirect method to send request to google server with the request data.

index.html

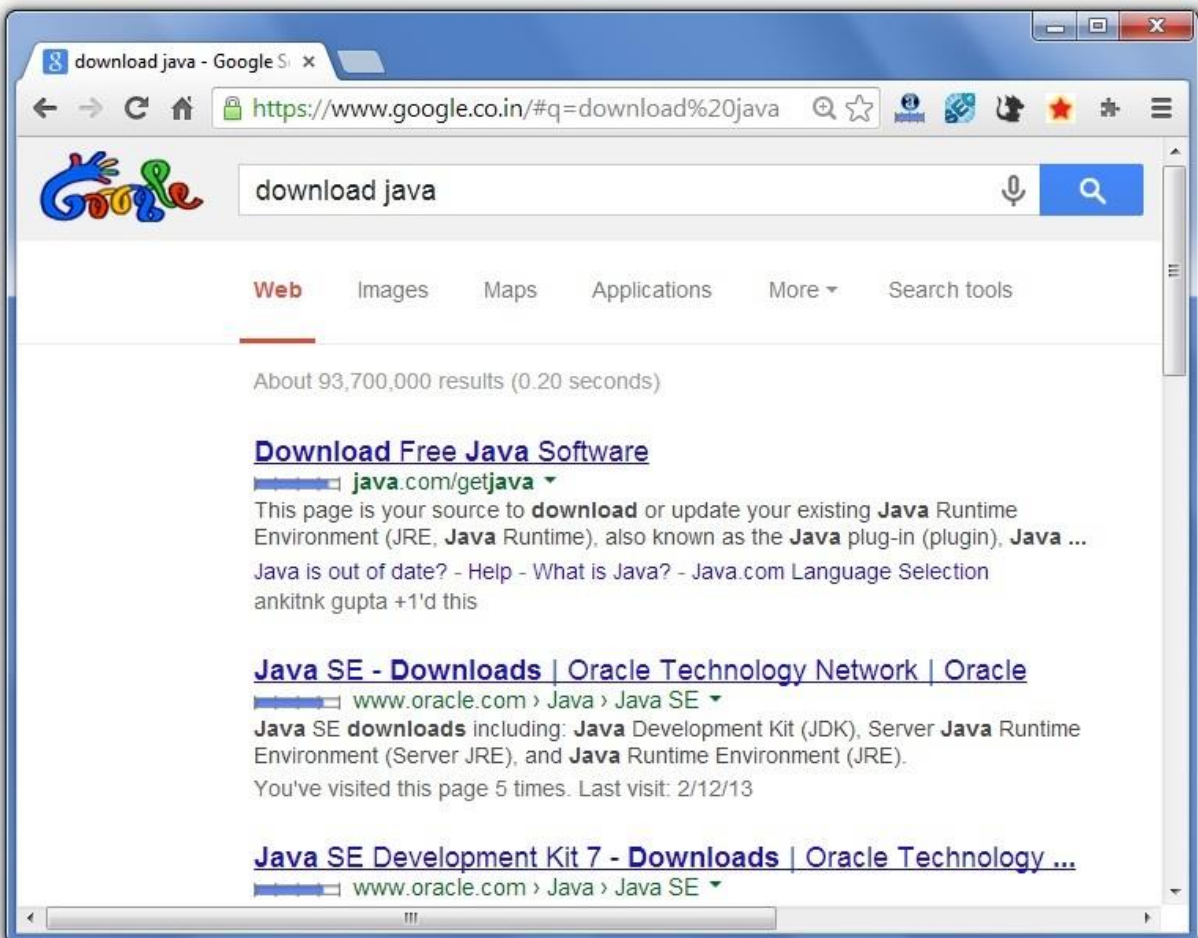
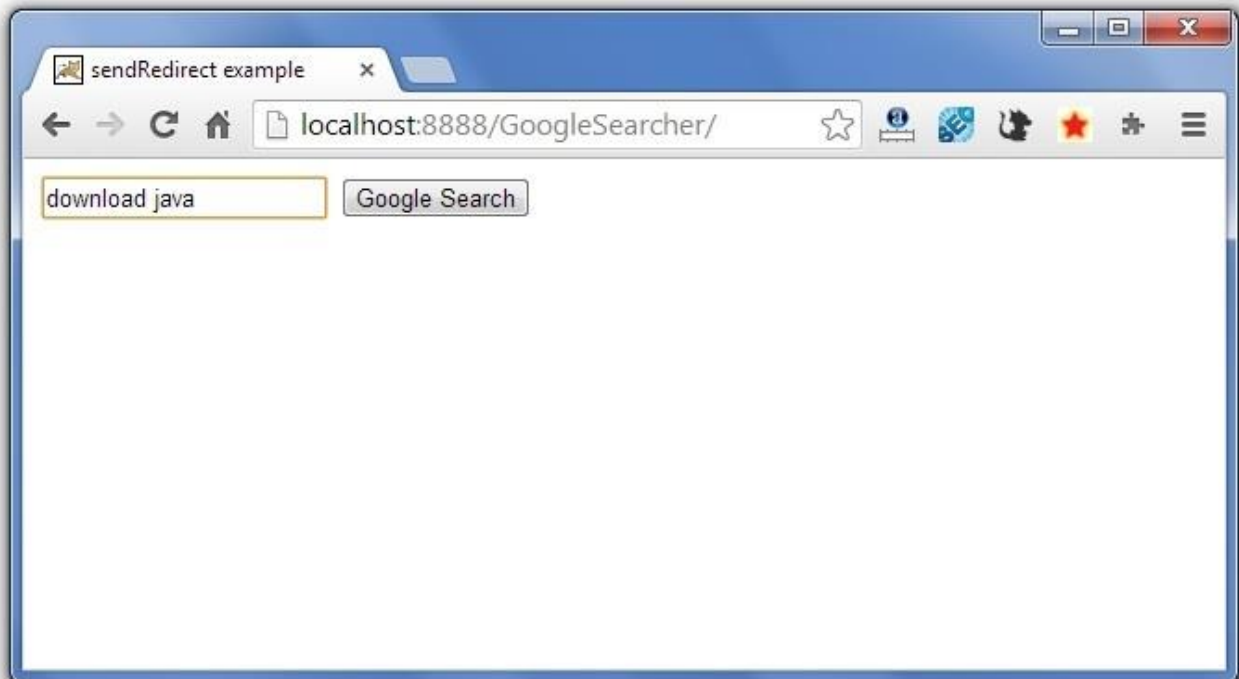
1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<meta charset="ISO-8859-1">`
5. `<title>sendRedirect example</title>`

```
6. </head>
7. <body>
8.
9.
10. <form action="MySearcher">
11. <input type="text" name="name">
12. <input type="submit" value="Google Search">
13. </form>
14.
15. </body>
16. </html>
```

MySearcher.java

```
1. import java.io.IOException;
2. import javax.servlet.ServletException;
3. import javax.servlet.http.HttpServlet;
4. import javax.servlet.http.HttpServletRequest;
5. import javax.servlet.http.HttpServletResponse;
6.
7. public class MySearcher extends HttpServlet {
8.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.
11.         String name=request.getParameter("name");
12.         response.sendRedirect("https://www.google.co.in/#q="+name);
13.     }
14. }
```

Output



ServletConfig Interface

An object of ServletConfig is created by the web container for each servlet. This object can be used to get configuration information from web.xml file.

If the configuration information is modified from the web.xml file, we don't need to change the servlet. So it is easier to manage the web application if any specific content is modified from time to time.

Advantage of ServletConfig

The core advantage of ServletConfig is that you don't need to edit the servlet file if information is modified from the web.xml file.

Methods of ServletConfig interface

1. **public String getInitParameter(String name):**Returns the parameter value for the specified parameter name.
2. **public Enumeration getInitParameterNames():**Returns an enumeration of all the initialization parameter names.
3. **public String getServletName():**Returns the name of the servlet.
4. **public ServletContext getServletContext():**Returns an object of ServletContext.

How to get the object of ServletConfig

1. **getServletConfig() method** of Servlet interface returns the object of ServletConfig.

Syntax of getServletConfig() method

1. **public** ServletConfig getServletConfig();

Example of getServletConfig() method

1. ServletConfig config=getServletConfig();
2. *//Now we can call the methods of ServletConfig interface*

Syntax to provide the initialization parameter for a servlet

The init-param sub-element of servlet is used to specify the initialization parameter for a servlet.

1. <web-app>
2. <servlet>
3.
- 4.
5. <init-param>
6. <param-name>parametername</param-name>
7. <param-value>parametervalue</param-value>
8. </init-param>
9.
10. </servlet>
11. </web-app>

Example of ServletConfig to get initialization parameter

In this example, we are getting the one initialization parameter from the web.xml file and printing this information in the servlet.

DemoServlet.java

1. **import** java.io.*;
2. **import** javax.servlet.*;
3. **import** javax.servlet.http.*;
- 4.
5. **public class** DemoServlet **extends** HttpServlet {
6. **public void** doGet(HttpServletRequest request, HttpServletResponse response)
7. **throws** ServletException, IOException {
- 8.
9. response.setContentType("text/html");
10. PrintWriter out = response.getWriter();
- 11.
12. ServletConfig config=getServletConfig();
13. String driver=config.getInitParameter("driver");
14. out.print("Driver is: "+driver);
- 15.
16. out.close();
17. }

- 18.
19. }

web.xml

1. <web-app>
- 2.
3. <servlet>
4. <servlet-name>DemoServlet</servlet-name>
5. <servlet-**class**>DemoServlet</servlet-**class**>
- 6.
7. <init-param>
8. <param-name>driver</param-name>
9. <param-value>sun.jdbc.odbc.JdbcOdbcDriver</param-value>
10. </init-param>
- 11.
12. </servlet>
- 13.
14. <servlet-mapping>
15. <servlet-name>DemoServlet</servlet-name>
16. <url-pattern>/servlet1</url-pattern>
17. </servlet-mapping>
- 18.
19. </web-app>

Example of ServletConfig to get all the initialization parameters

In this example, we are getting all the initialization parameter from the web.xml file and printing this information in the servlet.

DemoServlet.java

1. **import** java.io.IOException;
2. **import** java.io.PrintWriter;
3. **import** java.util.Enumeration;
- 4.

```

5. import javax.servlet.ServletConfig;
6. import javax.servlet.ServletException;
7. import javax.servlet.http.HttpServlet;
8. import javax.servlet.http.HttpServletRequest;
9. import javax.servlet.http.HttpServletResponse;
10.
11.
12. public class DemoServlet extends HttpServlet {
13. public void doGet(HttpServletRequest request, HttpServletResponse response)
14.     throws ServletException, IOException {
15.
16.     response.setContentType("text/html");
17.     PrintWriter out = response.getWriter();
18.
19.     ServletConfig config=getServletConfig();
20.     Enumeration<String> e=config.getInitParameterNames();
21.
22.     String str="";
23.     while(e.hasMoreElements()){
24.         str=e.nextElement();
25.         out.print("<br>Name: "+str);
26.         out.print(" value: "+config.getInitParameter(str));
27.     }
28.
29.     out.close();
30. }
31.
32. }

```

web.xml

```

1. <web-app>
2.
3. <servlet>
4. <servlet-name>DemoServlet</servlet-name>
5. <servlet-class>DemoServlet</servlet-class>

```

```
6.
7. <init-param>
8. <param-name>username</param-name>
9. <param-value>system</param-value>
10. </init-param>
11.
12. <init-param>
13. <param-name>password</param-name>
14. <param-value>oracle</param-value>
15. </init-param>
16.
17. </servlet>
18.
19. <servlet-mapping>
20. <servlet-name>DemoServlet</servlet-name>
21. <url-pattern>/servlet1</url-pattern>
22. </servlet-mapping>
23.
24. </web-app>
```

ServletContext Interface

An object of ServletContext is created by the web container at time of deploying the project. This object can be used to get configuration information from web.xml file. There is only one ServletContext object per web application.

If any information is shared to many servlet, it is better to provide it from the web.xml file using the **<context-param>** element.

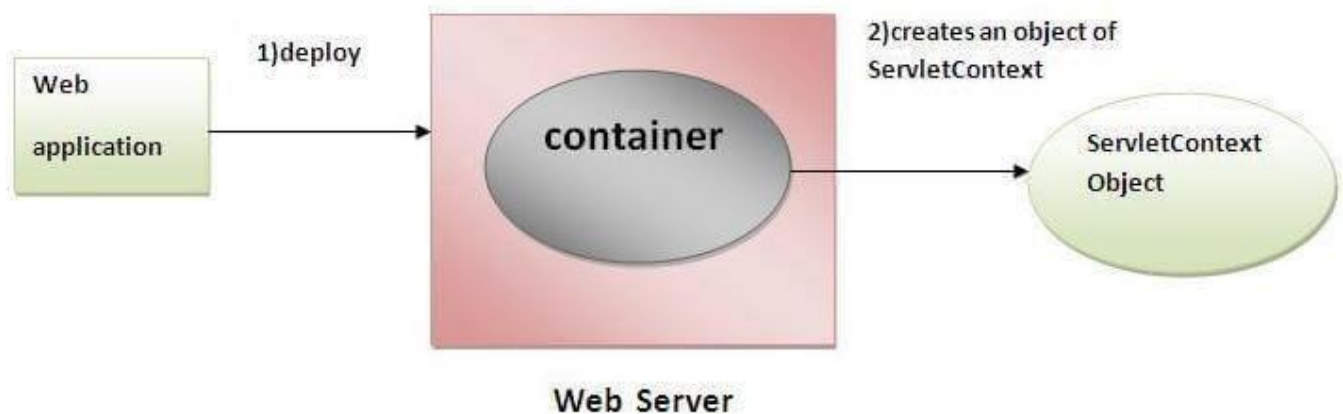
Advantage of ServletContext

Easy to maintain if any information is shared to all the servlet, it is better to make it available for all the servlet. We provide this information from the web.xml file, so if the information is changed, we don't need to modify the servlet. Thus it removes maintenance problem.

Usage of ServletContext Interface

There can be a lot of usage of ServletContext object. Some of them are as follows:

1. The object of ServletContext provides an interface between the container and servlet.
2. The ServletContext object can be used to get configuration information from the web.xml file.
3. The ServletContext object can be used to set, get or remove attribute from the web.xml file.
4. The ServletContext object can be used to provide inter-application communication.



Commonly used methods of ServletContext interface

There is given some commonly used methods of ServletContext interface.

1. **public String getInitParameter(String name):**Returns the parameter value for the specified parameter name.
2. **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters.
3. **public void setAttribute(String name, Object object):**sets the given object in the application scope.
4. **public Object getAttribute(String name):**Returns the attribute for the specified name.
5. **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters as an Enumeration of String objects.

6. **public void removeAttribute(String name):**Removes the attribute with the given name from the servlet context.

How to get the object of ServletContext interface

1. **getServletContext() method** of ServletConfig interface returns the object of ServletContext.
2. **getServletContext() method** of GenericServlet class returns the object of ServletContext.

Syntax of getServletContext() method

1. **public** ServletContext getServletContext()

Example of getServletContext() method

1. *//We can get the ServletContext object from ServletConfig object*
2. ServletContext application=getServletConfig().getServletContext();
- 3.
4. *//Another convenient way to get the ServletContext object*
5. ServletContext application=getServletContext();

Syntax to provide the initialization parameter in Context scope

The **context-param** element, subelement of web-app, is used to define the initialization parameter in the application scope. The param-name and param-value are the sub-elements of the context-param. The param-name element defines parameter name and param-value defines its value.

1. <web-app>
 2.
 - 3.
 4. <context-param>
 5. <param-name>parametername</param-name>
 6. <param-value>parametervalue</param-value>
 7. </context-param>
 8.
 9. </web-app>
-

Example of ServletContext to get the initialization parameter

In this example, we are getting the initialization parameter from the web.xml file and printing the value of the initialization parameter. Notice that the object of ServletContext represents the application scope. So if we change the value of the parameter from the web.xml file, all the servlet classes will get the changed value. So we don't need to modify the servlet. So it is better to have the common information for most of the servlets in the web.xml file by context-param element. Let's see the simple example:

DemoServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class DemoServlet extends HttpServlet{
7.     public void doGet(HttpServletRequest req,HttpServletResponse res)
8.     throws ServletException,IOException
9.     {
10. res.setContentType("text/html");
11. PrintWriter pw=res.getWriter();
12.
13. //creating ServletContext object
14. ServletContext context=getServletContext();
15.
16. //Getting the value of the initialization parameter and printing it
17. String driverName=context.getInitParameter("dname");
18. pw.println("driver name is="+driverName);
19.
20. pw.close();
21.
22. }}
```

web.xml

```
1. <web-app>
2.
```

```
3. <servlet>
4. <servlet-name>sonoojaiswal</servlet-name>
5. <servlet-class>DemoServlet</servlet-class>
6. </servlet>
7.
8. <context-param>
9. <param-name>dname</param-name>
10. <param-value>sun.jdbc.odbc.JdbcOdbcDriver</param-value>
11. </context-param>
12.
13. <servlet-mapping>
14. <servlet-name>sonoojaiswal</servlet-name>
15. <url-pattern>/context</url-pattern>
16. </servlet-mapping>
17.
18. </web-app>
```

Example of ServletContext to get all the initialization parameters

In this example, we are getting all the initialization parameter from the web.xml file. For getting all the parameters, we have used the `getInitParameterNames()` method in the servlet class.

DemoServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5. public class DemoServlet extends HttpServlet{
6. public void doGet(HttpServletRequest req,HttpServletResponse res)
7. throws ServletException,IOException
8. {
9. res.setContentType("text/html");
10. PrintWriter out=res.getWriter();
11.
```

```
12. ServletContext context=getServletContext();
13. Enumeration<String> e=context.getInitParameterNames();
14.
15. String str="";
16. while(e.hasMoreElements()){
17.     str=e.nextElement();
18.     out.print("<br> "+context.getInitParameter(str));
19. }
20. }}
```

web.xml

```
1. <web-app>
2.
3. <servlet>
4. <servlet-name>sonoojaiswal</servlet-name>
5. <servlet-class>DemoServlet</servlet-class>
6. </servlet>
7.
8. <context-param>
9. <param-name>dname</param-name>
10. <param-value>sun.jdbc.odbc.JdbcOdbcDriver</param-value>
11. </context-param>
12.
13. <context-param>
14. <param-name>username</param-name>
15. <param-value>system</param-value>
16. </context-param>
17.
18. <context-param>
19. <param-name>password</param-name>
20. <param-value>oracle</param-value>
21. </context-param>
22.
23. <servlet-mapping>
24. <servlet-name>sonoojaiswal</servlet-name>
25. <url-pattern>/context</url-pattern>
```

26. `</servlet-mapping>`
- 27.
28. `</web-app>`

Attribute in Servlet

An **attribute in servlet** is an object that can be set, get or removed from one of the following scopes:

1. request scope
2. session scope
3. application scope

The servlet programmer can pass informations from one servlet to another using attributes. It is just like passing object from one class to another so that we can reuse the same object again and again.

Attribute specific methods of ServletRequest, HttpSession and ServletContext interface

There are following 4 attribute specific methods. They are as follows:

1. **public void setAttribute(String name, Object object):** sets the given object in the application scope.
2. **public Object getAttribute(String name):** Returns the attribute for the specified name.
3. **public Enumeration getInitParameterNames():** Returns the names of the context's initialization parameters as an Enumeration of String objects.
4. **public void removeAttribute(String name):** Removes the attribute with the given name from the servlet context.

Example of ServletContext to set and get attribute

In this example, we are setting the attribute in the application scope and getting that value from another servlet.

DemoServlet1.java

1. **import** java.io.*;

```

2. import javax.servlet.*;
3. import javax.servlet.http.*;
4. public class DemoServlet1 extends HttpServlet{
5. public void doGet(HttpServletRequest req,HttpServletResponse res)
6. {
7. try{
8.
9. res.setContentType("text/html");
10. PrintWriter out=res.getWriter();
11.
12. ServletContext context=getServletContext();
13. context.setAttribute("company","IBM");
14.
15. out.println("Welcome to first servlet");
16. out.println("<a href='servlet2'>visit</a>");
17. out.close();
18.
19. }catch(Exception e){out.println(e);}
20.
21. }}

```

DemoServlet2.java

```

1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4. public class DemoServlet2 extends HttpServlet{
5. public void doGet(HttpServletRequest req,HttpServletResponse res)
6. {
7. try{
8.
9. res.setContentType("text/html");
10. PrintWriter out=res.getWriter();
11.
12. ServletContext context=getServletContext();
13. String n=(String)context.getAttribute("company");
14.

```

```
15. out.println("Welcome to "+n);
16. out.close();
17.
18. }catch(Exception e){out.println(e);}
19. }}
```

web.xml

```
1. <web-app>
2.
3. <servlet>
4. <servlet-name>s1</servlet-name>
5. <servlet-class>DemoServlet1</servlet-class>
6. </servlet>
7.
8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-class>DemoServlet2</servlet-class>
16. </servlet>
17.
18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
22.
23. </web-app>
```

Difference between ServletConfig and ServletContext

The servletconfig object refers to the single servlet whereas servletcontext object refers to the whole web application.

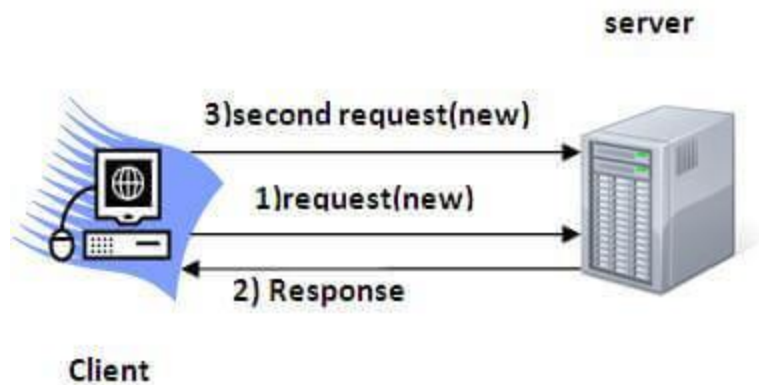
Session Tracking in Servlets

Session simply means a particular interval of time.

Session Tracking is a way to maintain state (data) of an user. It is also known as **session management** in servlet.

Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize to particular user.

HTTP is stateless that means each request is considered as the new request. It is shown in the figure given below:



Why use Session Tracking?

To recognize the user It is used to recognize the particular user.

Session Tracking Techniques

There are four techniques used in Session tracking:

1. **Cookies**
2. **Hidden Form Field**
3. **URL Rewriting**
4. **HttpSession**

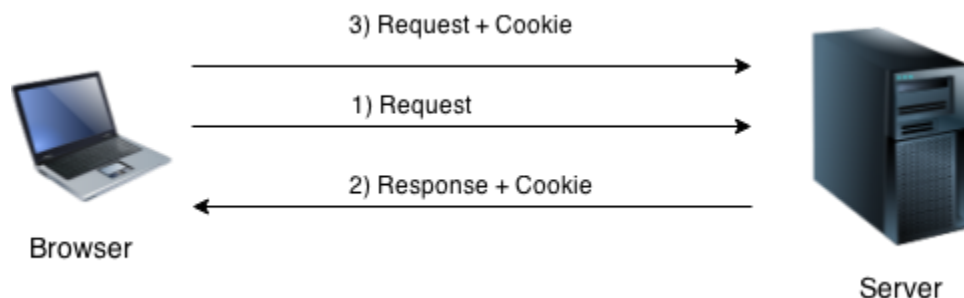
Cookies in Servlet

A **cookie** is a small piece of information that is persisted between the multiple client requests.

A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

How Cookie works

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.



Types of Cookie

There are 2 types of cookies in servlets.

1. Non-persistent cookie
2. Persistent cookie

Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie

It is **valid for multiple session**. It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if cookie is disabled from the browser.
2. Only textual information can be set in Cookie object.

Note: Gmail uses cookie technique for login. If you disable the cookie, gmail won't work.

Cookie class

javax.servlet.http.Cookie class provides the functionality of using cookies. It provides a lot of useful methods for cookies.

Constructor of Cookie class

Constructor	Description
Cookie()	constructs a cookie.
Cookie(String name, String value)	constructs a cookie with a specified name and value.

Useful Methods of Cookie class

There are given some commonly used methods of the Cookie class.

Method	Description
public void setMaxAge(int expiry)	Sets the maximum age of the cookie in seconds.
public String getName()	Returns the name of the cookie. The name cannot be changed after creation.
public String getValue()	Returns the value of the cookie.

public void setName(String name)	changes the name of the cookie.
public void setValue(String value)	changes the value of the cookie.

Other methods required for using Cookies

For adding cookie or getting the value from the cookie, we need some methods provided by other interfaces. They are:

1. **public void addCookie(Cookie ck):**method of HttpServletResponse interface is used to add cookie in response object.
2. **public Cookie[] getCookies():**method of HttpServletRequest interface is used to return all the cookies from the browser.

How to create Cookie?

Let's see the simple code to create cookie.

1. Cookie ck=**new** Cookie("user","sonoo jaiswal");//creating cookie object
2. response.addCookie(ck);//adding cookie in the response

How to delete Cookie?

Let's see the simple code to delete cookie. It is mainly used to logout or signout the user.

1. Cookie ck=**new** Cookie("user","");//deleting value of cookie
2. ck.setMaxAge(0);//changing the maximum age to 0 seconds
3. response.addCookie(ck);//adding cookie in the response

How to get Cookies?

Let's see the simple code to get all the cookies.

1. Cookie ck[]=request.getCookies();

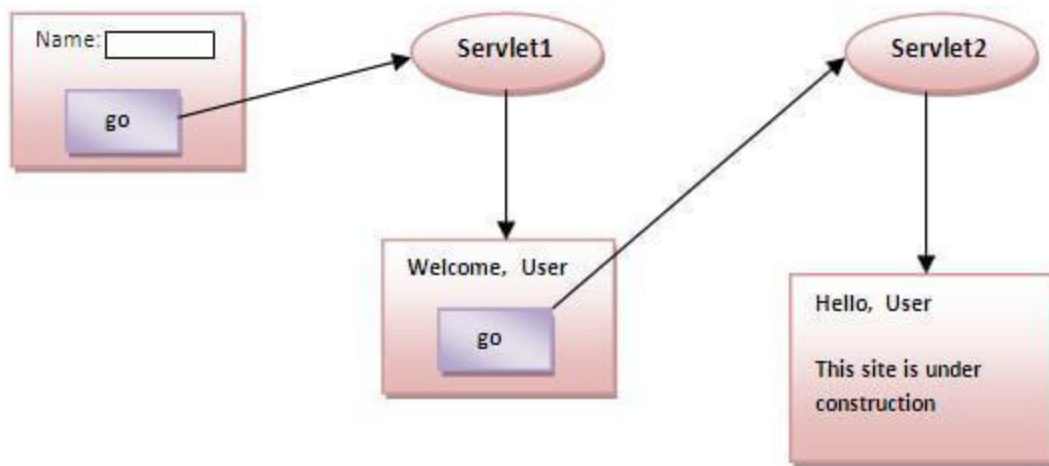
```

2. for(int i=0;i<ck.length;i++){
3.   out.print("<br>" + ck[i].getName() + " " + ck[i].getValue()); //printing name and value of cookie
4. }

```

Simple example of Servlet Cookies

In this example, we are storing the name of the user in the cookie object and accessing it in another servlet. As we know well that session corresponds to the particular user. So if you access it from too many browsers with different values, you will get the different value.



index.html

```

1. <form action="servlet1" method="post">
2. Name:<input type="text" name="userName"/> <br/>
3. <input type="submit" value="go"/>
4. </form>

```

FirstServlet.java

```

1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class FirstServlet extends HttpServlet {
7.
8.   public void doPost(HttpServletRequest request, HttpServletResponse response){
9.     try{

```

```

10.
11. response.setContentType("text/html");
12. PrintWriter out = response.getWriter();
13.
14. String n=request.getParameter("userName");
15. out.print("Welcome "+n);
16.
17. Cookie ck=new Cookie("uname",n);//creating cookie object
18. response.addCookie(ck);//adding cookie in the response
19.
20. //creating submit button
21. out.print("<form action='servlet2'>");
22. out.print("<input type='submit' value='go'>");
23. out.print("</form>");
24.
25. out.close();
26.
27.     }catch(Exception e){System.out.println(e);}
28. }
29. }

```

SecondServlet.java

```

1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5. public class SecondServlet extends HttpServlet {
6.
7.     public void doPost(HttpServletRequest request, HttpServletResponse response){
8.         try{
9.
10.            response.setContentType("text/html");
11.            PrintWriter out = response.getWriter();
12.
13.            Cookie ck[]=request.getCookies();
14.            out.print("Hello "+ck[0].getValue());
15.
16.            out.close();
17.

```

```
18.     }catch(Exception e){System.out.println(e);}
19.   }
20.
21.
22. }
```

web.xml

```
1. <web-app>
2.
3. <servlet>
4. <servlet-name>s1</servlet-name>
5. <servlet-class>FirstServlet</servlet-class>
6. </servlet>
7.
8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-class>SecondServlet</servlet-class>
16. </servlet>
17.
18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
22.
23. </web-app>
```

Servlet Login and Logout Example using Cookies

A **cookie** is a kind of information that is stored at client side.

In the previous page, we learned a lot about cookie e.g. how to create cookie, how to delete cookie, how to get cookie etc.

Here, we are going to create a login and logout example using servlet cookies.

In this example, we are creating 3 links: login, logout and profile. User can't go to profile page until he/she is logged in. If user is logged out, he need to login again to visit profile.

In this application, we have created following files.

1. index.html
2. link.html
3. login.html
4. LoginServlet.java
5. LogoutServlet.java
6. ProfileServlet.java
7. web.xml

1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<meta charset="ISO-8859-1">`
5. `<title>Servlet Login Example</title>`
6. `</head>`
7. `<body>`
- 8.
9. `<h1>Welcome to Login App by Cookie</h1>`
10. `<a href="login.html">Login</a> |`
11. `<a href="LogoutServlet">Logout</a> |`
12. `<a href="ProfileServlet">Profile</a>`
- 13.
14. `</body>`
15. `</html>`

File: link.html

1. `<a href="login.html">Login</a> |`
 2. `<a href="LogoutServlet">Logout</a> |`
 3. `<a href="ProfileServlet">Profile</a>`
 4. `<hr>`
-

File: login.html

1. `<form action="LoginServlet" method="post">`
2. Name: `<input type="text" name="name"> <br>`
3. Password: `<input type="password" name="password"> <br>`
4. `<input type="submit" value="login">`
5. `</form>`

File: LoginServlet.java

1. **package** com.javatpoint;
- 2.
3. **import** java.io.IOException;
4. **import** java.io.PrintWriter;
5. **import** javax.servlet.ServletException;
6. **import** javax.servlet.http.Cookie;
7. **import** javax.servlet.http.HttpServlet;
8. **import** javax.servlet.http.HttpServletRequest;
9. **import** javax.servlet.http.HttpServletResponse;
10. **public class** LoginServlet **extends** HttpServlet {
11. **protected void** doPost(HttpServletRequest request, HttpServletResponse response)
12. **throws** ServletException, IOException {
13. response.setContentType("text/html");
14. PrintWriter out=response.getWriter();
- 15.
16. request.getRequestDispatcher("link.html").include(request, response);
- 17.
18. String name=request.getParameter("name");
19. String password=request.getParameter("password");
- 20.
21. **if**(password.equals("admin123")){
22. out.print("You are successfully logged in!");
23. out.print("
Welcome, " + name);
- 24.
25. Cookie ck=**new** Cookie("name",name);
26. response.addCookie(ck);
27. }**else**{
28. out.print("sorry, username or password error!");
29. request.getRequestDispatcher("login.html").include(request, response);
30. }

```
31.  
32.     out.close();  
33. }  
34.  
35. }
```

File: LogoutServlet.java

```
1. package com.javatpoint;  
2.  
3. import java.io.IOException;  
4. import java.io.PrintWriter;  
5. import javax.servlet.ServletException;  
6. import javax.servlet.http.Cookie;  
7. import javax.servlet.http.HttpServlet;  
8. import javax.servlet.http.HttpServletRequest;  
9. import javax.servlet.http.HttpServletResponse;  
10. public class LogoutServlet extends HttpServlet {  
11.     protected void doGet(HttpServletRequest request, HttpServletResponse response)  
12.         throws ServletException, IOException {  
13.         response.setContentType("text/html");  
14.         PrintWriter out=response.getWriter();  
15.  
16.  
17.         request.getRequestDispatcher("link.html").include(request, response);  
18.  
19.         Cookie ck=new Cookie("name","");  
20.         ck.setMaxAge(0);  
21.         response.addCookie(ck);  
22.  
23.         out.print("you are successfully logged out!");  
24.     }  
25. }
```

File: ProfileServlet.java

```
1. package com.javatpoint;  
2.  
3. import java.io.IOException;  
4. import java.io.PrintWriter;
```

```

5. import javax.servlet.ServletException;
6. import javax.servlet.http.Cookie;
7. import javax.servlet.http.HttpServlet;
8. import javax.servlet.http.HttpServletRequest;
9. import javax.servlet.http.HttpServletResponse;
10. public class ProfileServlet extends HttpServlet {
11.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
12.         throws ServletException, IOException {
13.         response.setContentType("text/html");
14.         PrintWriter out=response.getWriter();
15.
16.         request.getRequestDispatcher("link.html").include(request, response);
17.
18.         Cookie ck[]=request.getCookies();
19.         if(ck!=null){
20.             String name=ck[0].getValue();
21.             if(!name.equals("")||name!=null){
22.                 out.print("<b>Welcome to Profile</b>");
23.                 out.print("<br>Welcome, " + name);
24.             }
25.         }else{
26.             out.print("Please login first");
27.             request.getRequestDispatcher("login.html").include(request, response);
28.         }
29.         out.close();
30.     }
31. }

```

File: web.xml

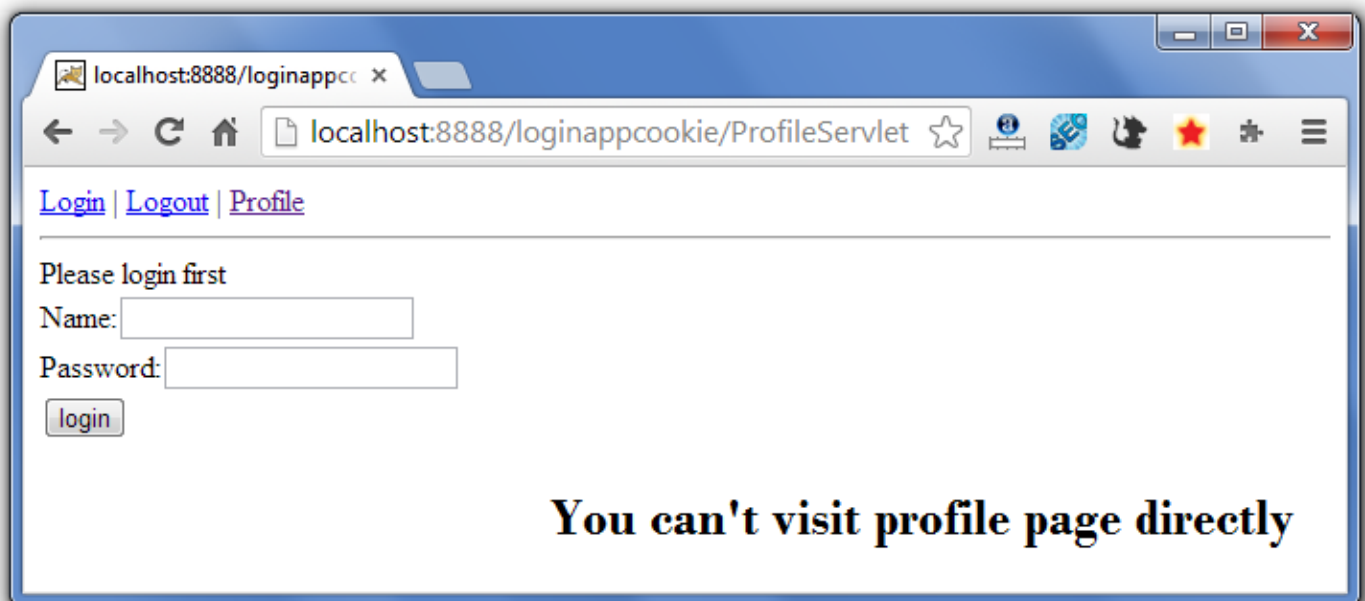
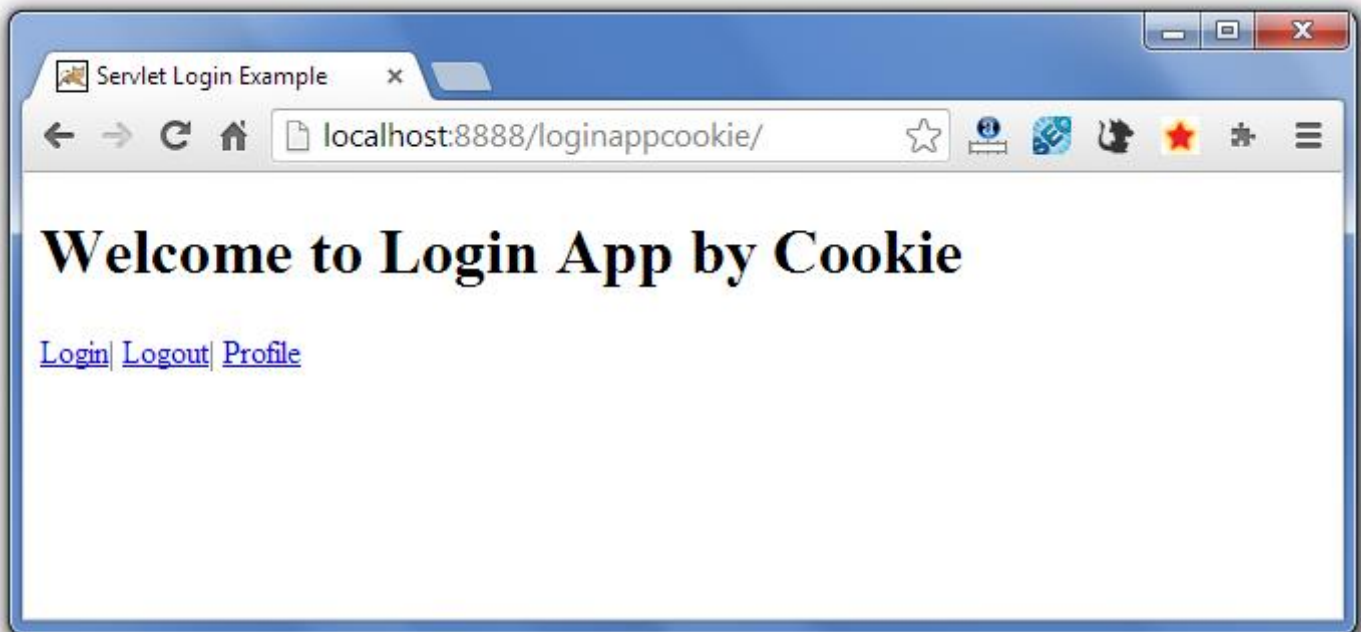
```

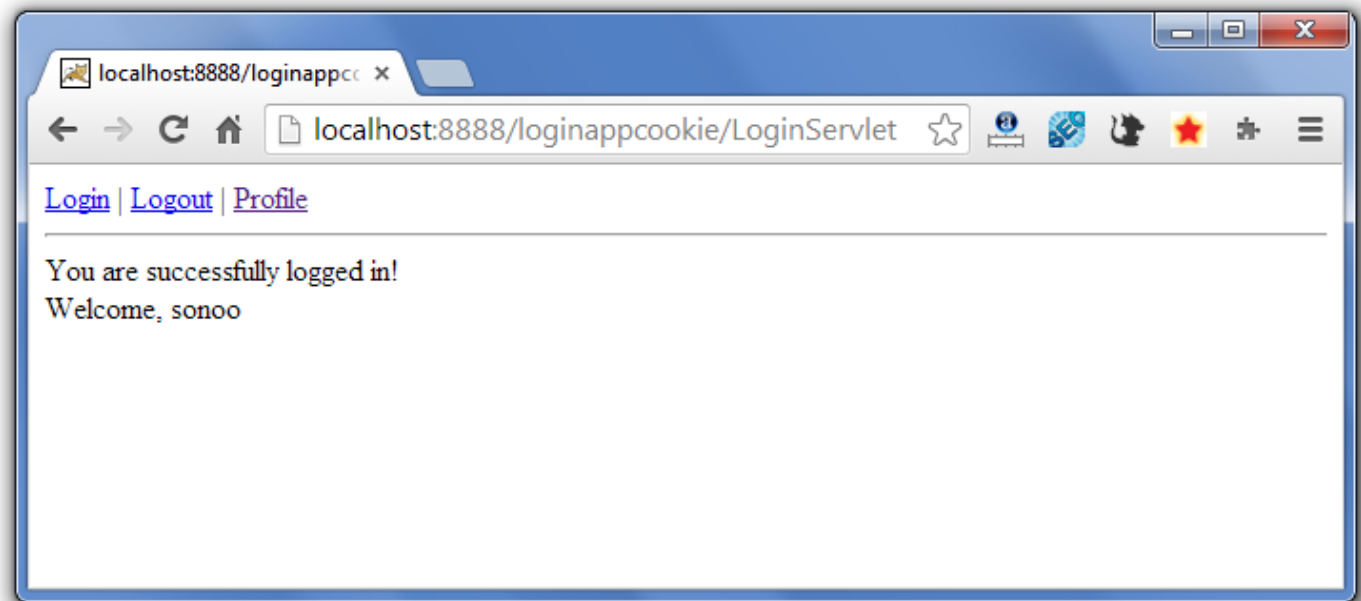
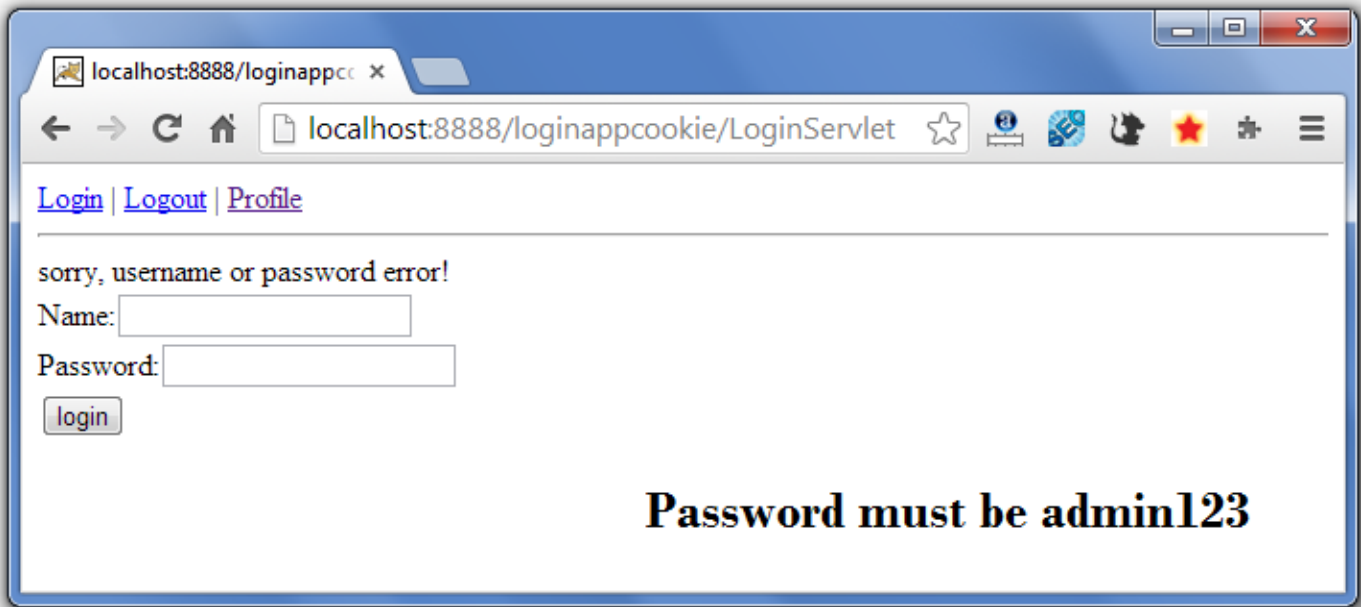
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3. xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/jav
   aee
4. http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd" id="WebApp_ID" version="2.5">
5.
6.     <servlet>
7.         <description></description>
8.         <display-name>LoginServlet</display-name>

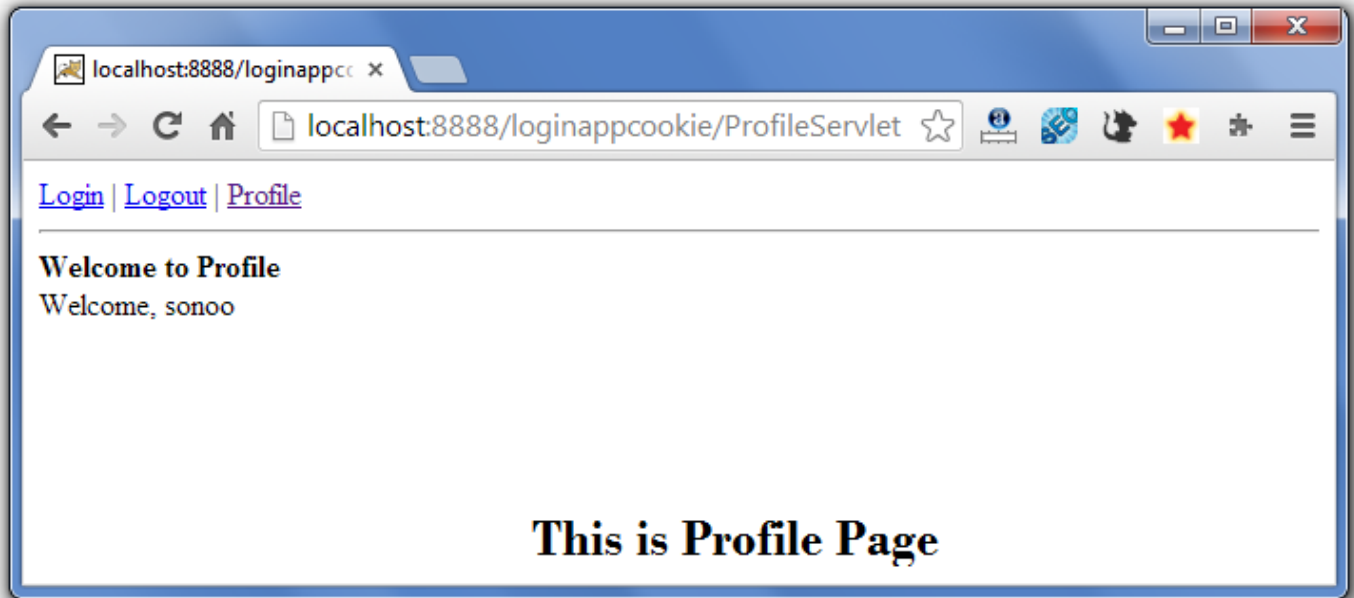
```

```
9.     <servlet-name>LoginServlet</servlet-name>
10.    <servlet-class>com.javatpoint.LoginServlet</servlet-class>
11.  </servlet>
12.  <servlet-mapping>
13.    <servlet-name>LoginServlet</servlet-name>
14.    <url-pattern>/LoginServlet</url-pattern>
15.  </servlet-mapping>
16.  <servlet>
17.    <description></description>
18.    <display-name>ProfileServlet</display-name>
19.    <servlet-name>ProfileServlet</servlet-name>
20.    <servlet-class>com.javatpoint.ProfileServlet</servlet-class>
21.  </servlet>
22.  <servlet-mapping>
23.    <servlet-name>ProfileServlet</servlet-name>
24.    <url-pattern>/ProfileServlet</url-pattern>
25.  </servlet-mapping>
26.  <servlet>
27.    <description></description>
28.    <display-name>LogoutServlet</display-name>
29.    <servlet-name>LogoutServlet</servlet-name>
30.    <servlet-class>com.javatpoint.LogoutServlet</servlet-class>
31.  </servlet>
32.  <servlet-mapping>
33.    <servlet-name>LogoutServlet</servlet-name>
34.    <url-pattern>/LogoutServlet</url-pattern>
35.  </servlet-mapping>
36. </web-app>
```

Output







If again you click on the profile link, you need to login first.

2) Hidden Form Field

In case of Hidden Form Field a **hidden (invisible) textfield** is used for maintaining the state of an user.

In such case, we store the information in the hidden field and get it from another servlet. This approach is better if we have to submit form in all the pages and we don't want to depend on the browser.

Let's see the code to store value in hidden field.

1. `<input type="hidden" name="uname" value="Vimal Jaiswal">`

Here, uname is the hidden field name and Vimal Jaiswal is the hidden field value.

Real application of hidden form field

It is widely used in comment form of a website. In such case, we store page id or page name in the hidden field so that each page can be uniquely identified.

Advantage of Hidden Form Field

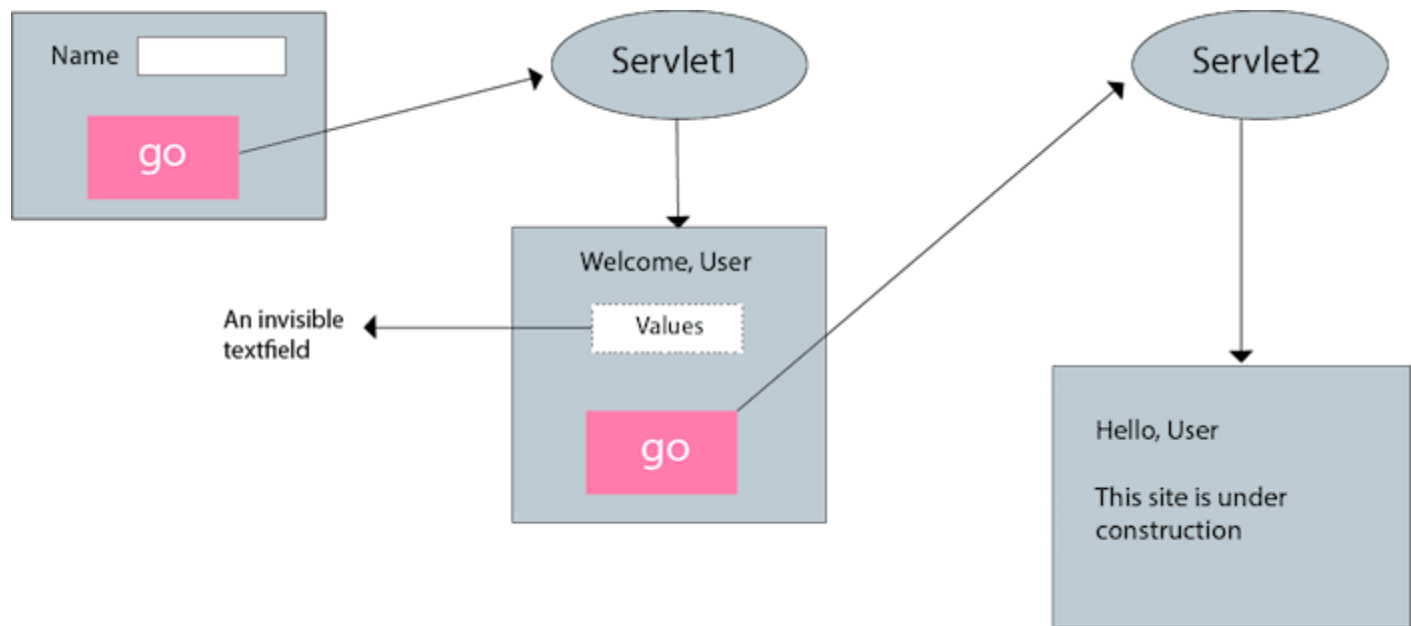
1. It will always work whether cookie is disabled or not.

Disadvantage of Hidden Form Field:

1. It is maintained at server side.
2. Extra form submission is required on each pages.
3. Only textual information can be used.

Example of using Hidden Form Field

In this example, we are storing the name of the user in a hidden textfield and getting that value from another servlet.



index.html

1. <form action="servlet1">
2. Name:<input type="text" name="userName"/>

3. <input type="submit" value="go"/>
4. </form>

FirstServlet.java

1. **import** java.io.*;
2. **import** javax.servlet.*;
3. **import** javax.servlet.http.*;
- 4.
5. **public class** FirstServlet **extends** HttpServlet {
6. **public void** doGet(HttpServletRequest request, HttpServletResponse response){
7. **try**{
- 8.
9. response.setContentType("text/html");
10. PrintWriter out = response.getWriter();
- 11.
12. String n=request.getParameter("userName");
13. out.print("Welcome "+n);
- 14.
15. //creating form that have invisible textfield
16. out.print("<form action='servlet2'>");
17. out.print("<input type='hidden' name='uname' value='"+n+"'>");
18. out.print("<input type='submit' value='go'>");

```

19.    out.print("</form>");
20.    out.close();
21.
22.        }catch(Exception e){System.out.println(e);}
23.    }
24.
25. }

```

SecondServlet.java

```

1.  import java.io.*;
2.  import javax.servlet.*;
3.  import javax.servlet.http.*;
4.  public class SecondServlet extends HttpServlet {
5.      public void doGet(HttpServletRequest request, HttpServletResponse response)
6.          try{
7.              response.setContentType("text/html");
8.              PrintWriter out = response.getWriter();
9.
10.             //Getting the value from the hidden field
11.             String n=request.getParameter("uname");
12.             out.print("Hello "+n);
13.
14.             out.close();
15.             }catch(Exception e){System.out.println(e);}
16.     }
17. }

```

web.xml

```

1. <web-app>
2.
3. <servlet>
4. <servlet-name>s1</servlet-name>
5. <servlet-class>FirstServlet</servlet-class>
6. </servlet>
7.
8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>

```

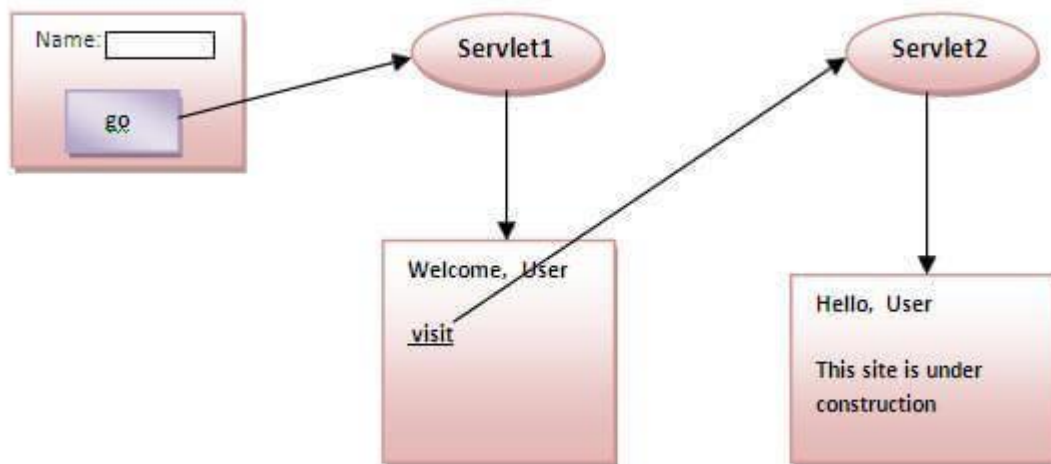
```
12.  
13. <servlet>  
14. <servlet-name>s2</servlet-name>  
15. <servlet-class>SecondServlet</servlet-class>  
16. </servlet>  
17.  
18. <servlet-mapping>  
19. <servlet-name>s2</servlet-name>  
20. <url-pattern>/servlet2</url-pattern>  
21. </servlet-mapping>  
22.  
23. </web-app>
```

3)URL Rewriting

In URL rewriting, we append a token or identifier to the URL of the next Servlet or the next resource. We can send parameter name/value pairs using the following format:

url?name1=value1&name2=value2&??

A name and a value is separated using an equal = sign, a parameter name/value pair is separated from another parameter using the ampersand(&). When the user clicks the hyperlink, the parameter name/value pairs will be passed to the server. From a Servlet, we can use `getParameter()` method to obtain a parameter value.



Advantage of URL Rewriting

1. It will always work whether cookie is disabled or not (browser independent).
2. Extra form submission is not required on each pages.

Disadvantage of URL Rewriting

1. It will work only with links.
2. It can send Only textual information.

Example of using URL Rewriting

In this example, we are maintaining the state of the user using link. For this purpose, we are appending the name of the user in the query string and getting the value from the query string in another page.

index.html

1. `<form action="servlet1">`
2. Name: `<input type="text" name="userName"/>` `<br/>`
3. `<input type="submit" value="go"/>`
4. `</form>`

FirstServlet.java

1. `import java.io.*;`
2. `import javax.servlet.*;`
3. `import javax.servlet.http.*;`
- 4.
- 5.
6. `public class FirstServlet extends HttpServlet {`
- 7.
8. `public void doGet(HttpServletRequest request, HttpServletResponse response){`
9. `try{`
- 10.
11. `response.setContentType("text/html");`
12. `PrintWriter out = response.getWriter();`
- 13.
14. `String n=request.getParameter("userName");`
15. `out.print("Welcome "+n);`
- 16.
17. `//appending the username in the query string`
18. `out.print("<a href='servlet2?uname="+n+"'>visit</a>");`

```
19.  
20.     out.close();  
21.  
22.         }catch(Exception e){System.out.println(e);}  
23.     }  
24.  
25. }
```

SecondServlet.java

```
1.  import java.io.*;  
2.  import javax.servlet.*;  
3.  import javax.servlet.http.*;  
4.  
5.  public class SecondServlet extends HttpServlet {  
6.  
7.      public void doGet(HttpServletRequest request, HttpServletResponse response)  
8.          try{  
9.  
10.         response.setContentType("text/html");  
11.         PrintWriter out = response.getWriter();  
12.  
13.         //getting value from the query string  
14.         String n=request.getParameter("uname");  
15.         out.print("Hello " +n);  
16.  
17.         out.close();  
18.  
19.         }catch(Exception e){System.out.println(e);}  
20.     }  
21.  
22.  
23. }
```

web.xml

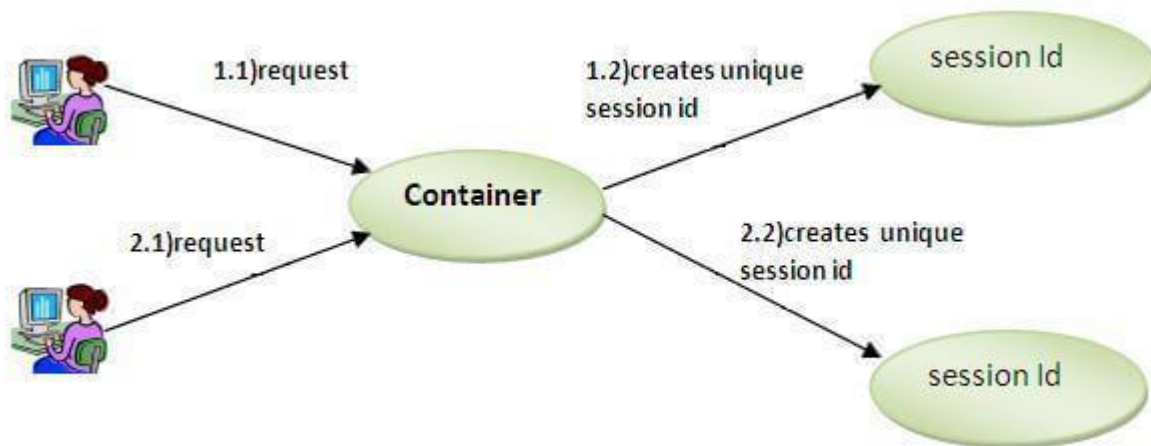
```
1. <web-app>  
2.  
3. <servlet>  
4. <servlet-name>s1</servlet-name>  
5. <servlet-class>FirstServlet</servlet-class>
```

6. </servlet>
- 7.
8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
- 12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-**class**>SecondServlet</servlet-**class**>
16. </servlet>
- 17.
18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
- 22.
23. </web-app>

4) HttpSession interface

In such case, container creates a session id for each user. The container uses this id to identify the particular user. An object of HttpSession can be used to perform two tasks:

1. bind objects
2. view and manipulate information about a session, such as the session identifier, creation time, and last accessed time.



How to get the HttpSession object ?

The `HttpServletRequest` interface provides two methods to get the object of `HttpSession`:

1. **`public HttpSession getSession():`**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **`public HttpSession getSession(boolean create):`**Returns the current `HttpSession` associated with this request or, if there is no current session and `create` is true, returns a new session.

Commonly used methods of HttpSession interface

1. **`public String getId():`**Returns a string containing the unique identifier value.
2. **`public long getCreationTime():`**Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
3. **`public long getLastAccessedTime():`**Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.
4. **`public void invalidate():`**Invalidates this session then unbinds any objects bound to it.

Example of using HttpSession

In this example, we are setting the attribute in the session scope in one servlet and getting that value from the session scope in another servlet. To set the attribute in the session scope, we have used the `setAttribute()` method of `HttpSession` interface and to get the attribute, we have used the `getAttribute` method.

index.html

```
1. <form action="servlet1">
2. Name:<input type="text" name="userName"/> <br/>
3. <input type="submit" value="go"/>
4. </form>
```

FirstServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class FirstServlet extends HttpServlet {
7.
8.     public void doGet(HttpServletRequest request, HttpServletResponse response){
9.         try{
10.
11.             response.setContentType("text/html");
12.             PrintWriter out = response.getWriter();
13.
14.             String n=request.getParameter("userName");
15.             out.print("Welcome " +n);
16.
17.             HttpSession session=request.getSession();
18.             session.setAttribute("uname",n);
19.
20.             out.print("<a href='servlet2'>visit</a>");
21.
22.             out.close();
23.
24.         }catch(Exception e){System.out.println(e);}
25.     }
26.
27. }
```

SecondServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
```



```

4.
5. public class SecondServlet extends HttpServlet {
6.
7. public void doGet(HttpServletRequest request, HttpServletResponse response)
8.     try{
9.
10.         response.setContentType("text/html");
11.         PrintWriter out = response.getWriter();
12.
13.         HttpSession session=request.getSession(false);
14.         String n=(String)session.getAttribute("uname");
15.         out.print("Hello "+n);
16.
17.         out.close();
18.
19.     }catch(Exception e){System.out.println(e);}
20. }
21.
22.
23. }

```

web.xml

```

1. <web-app>
2.
3. <servlet>
4. <servlet-name>s1</servlet-name>
5. <servlet-class>FirstServlet</servlet-class>
6. </servlet>
7.
8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-class>SecondServlet</servlet-class>
16. </servlet>
17.

```

18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
- 22.
23. </web-app>

Servlet HttpSession Login and Logout Example

We can bind the objects on HttpSession instance and get the objects by using setAttribute and getAttribute methods.

In the previous page, we have learnt about what is HttpSession, How to store and get data from session object etc.

Here, we are going to create a real world login and logout application without using database code. We are assuming that password is admin123.

Visit here for login and logout application using cookies only [servlet login and logout example using cookies](#)

In this example, we are creating 3 links: login, logout and profile. User can't go to profile page until he/she is logged in. If user is logged out, he need to login again to visit profile.

In this application, we have created following files.

1. index.html
2. link.html
3. login.html
4. LoginServlet.java
5. LogoutServlet.java
6. ProfileServlet.java
7. web.xml

1. <!DOCTYPE html>
2. <html>
3. <head>
4. <meta charset="ISO-8859-1">

5. `<title>Servlet Login Example</title>`
 6. `</head>`
 7. `<body>`
 - 8.
 9. `<h1>Login App using HttpSession</h1>`
 10. `<a href="login.html">Login</a>|`
 11. `<a href="LogoutServlet">Logout</a>|`
 12. `<a href="ProfileServlet">Profile</a>`
 - 13.
 14. `</body>`
 15. `</html>`
-

File: link.html

1. `<a href="login.html">Login</a> |`
 2. `<a href="LogoutServlet">Logout</a> |`
 3. `<a href="ProfileServlet">Profile</a>`
 4. `<hr>`
-

File: login.html

1. `<form action="LoginServlet" method="post">`
 2. Name: `<input type="text" name="name"> <br>`
 3. Password: `<input type="password" name="password"> <br>`
 4. `<input type="submit" value="login">`
 5. `</form>`
-

File: LoginServlet.java

1. `import java.io.IOException;`
2. `import java.io.PrintWriter;`
- 3.
4. `import javax.servlet.ServletException;`
5. `import javax.servlet.http.HttpServlet;`
6. `import javax.servlet.http.HttpServletRequest;`
7. `import javax.servlet.http.HttpServletResponse;`
8. `import javax.servlet.http.HttpSession;`
9. `public class LoginServlet extends HttpServlet {`
10. `protected void doPost(HttpServletRequest request, HttpServletResponse response)`

```

11.         throws ServletException, IOException {
12.     response.setContentType("text/html");
13.     PrintWriter out=response.getWriter();
14.     request.getRequestDispatcher("link.html").include(request, response);
15.
16.     String name=request.getParameter("name");
17.     String password=request.getParameter("password");
18.
19.     if(password.equals("admin123")){
20.         out.print("Welcome, "+name);
21.         HttpSession session=request.getSession();
22.         session.setAttribute("name",name);
23.     }
24.     else{
25.         out.print("Sorry, username or password error!");
26.         request.getRequestDispatcher("login.html").include(request, response);
27.     }
28.     out.close();
29. }
30. }

```

File: LogoutServlet.java

```

1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.http.HttpServlet;
6. import javax.servlet.http.HttpServletRequest;
7. import javax.servlet.http.HttpServletResponse;
8. import javax.servlet.http.HttpSession;
9. public class LogoutServlet extends HttpServlet {
10.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
11.         throws ServletException, IOException {
12.         response.setContentType("text/html");
13.         PrintWriter out=response.getWriter();
14.
15.         request.getRequestDispatcher("link.html").include(request, response);

```

```
16.
17.     HttpSession session=request.getSession();
18.     session.invalidate();
19.
20.     out.print("You are successfully logged out!");
21.
22.     out.close();
23. }
24. }
```

File: ProfileServlet.java

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3. import javax.servlet.ServletException;
4. import javax.servlet.http.HttpServlet;
5. import javax.servlet.http.HttpServletRequest;
6. import javax.servlet.http.HttpServletResponse;
7. import javax.servlet.http.HttpSession;
8. public class ProfileServlet extends HttpServlet {
9.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
10.         throws ServletException, IOException {
11.         response.setContentType("text/html");
12.         PrintWriter out=response.getWriter();
13.         request.getRequestDispatcher("link.html").include(request, response);
14.
15.         HttpSession session=request.getSession(false);
16.         if(session!=null){
17.             String name=(String)session.getAttribute("name");
18.
19.             out.print("Hello, " +name+ " Welcome to Profile");
20.         }
21.         else{
22.             out.print("Please login first");
23.             request.getRequestDispatcher("login.html").include(request, response);
24.         }
25.         out.close();
26.     }
```

27. }

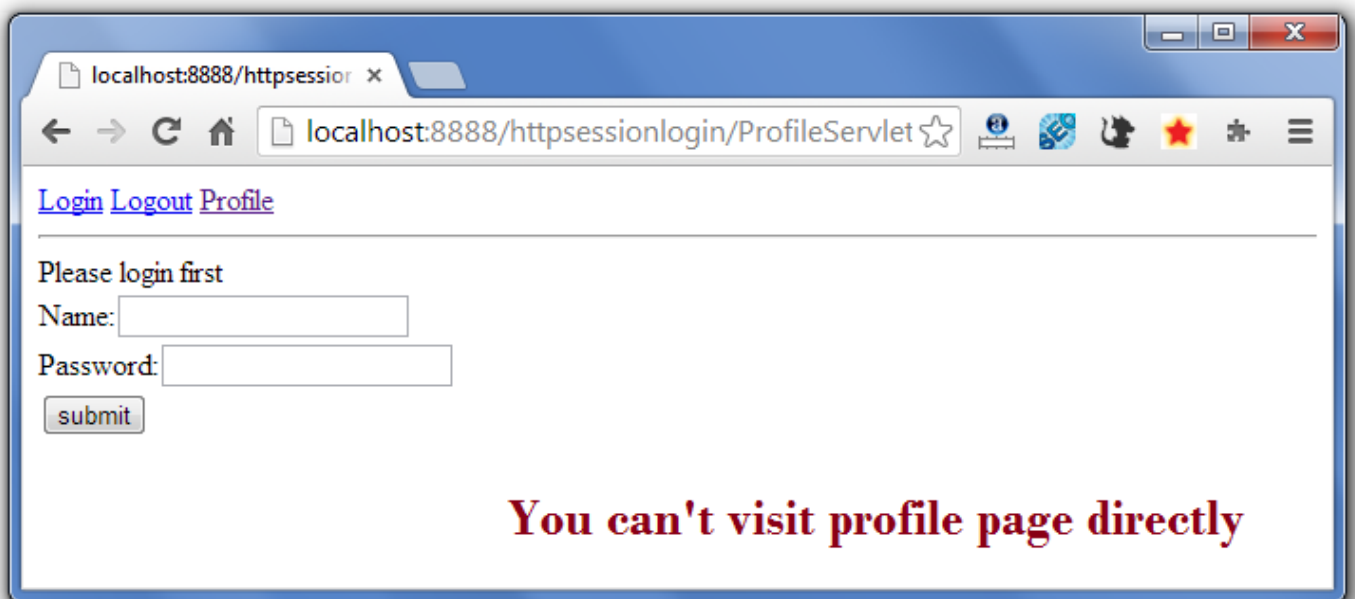
File: web.xml

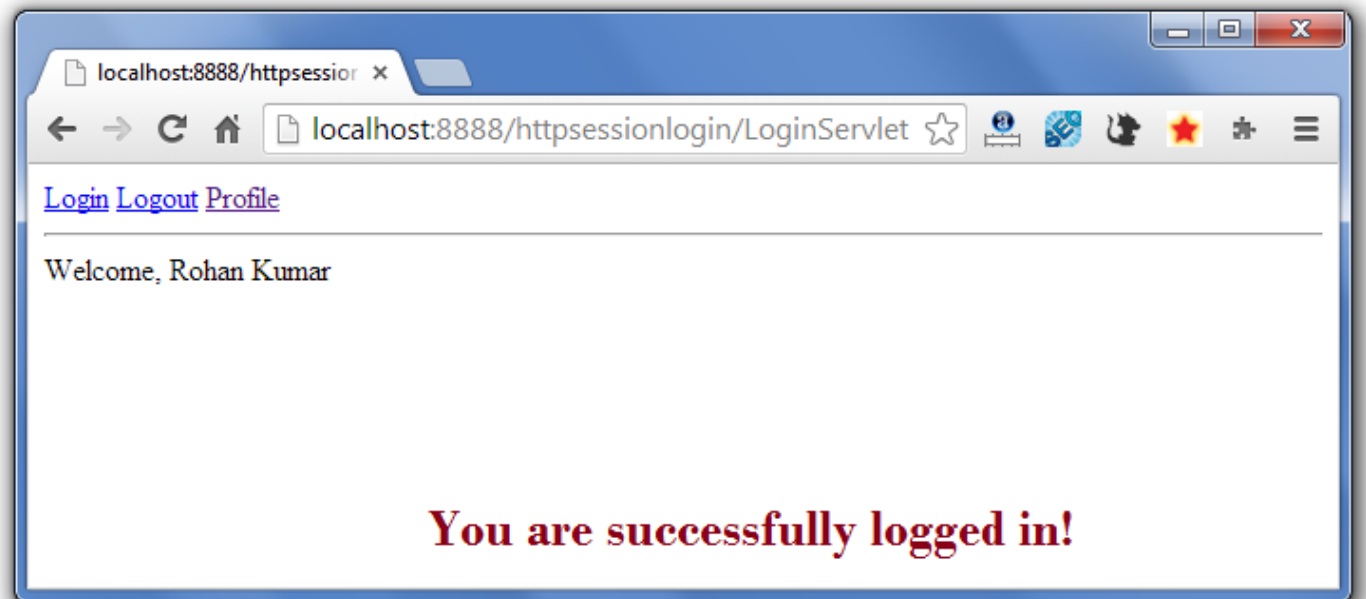
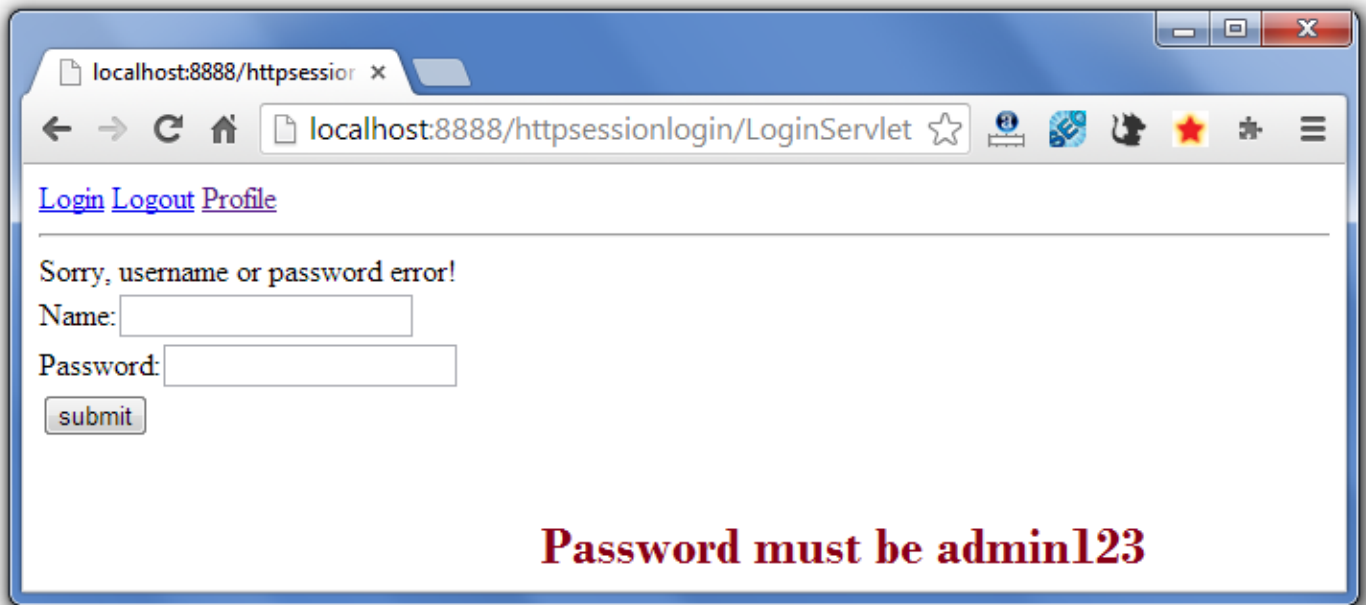
```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3. xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/jav
   aee
4. http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd" id="WebApp_ID" version="2.5">
5.
6.   <servlet>
7.     <description></description>
8.     <display-name>LoginServlet</display-name>
9.     <servlet-name>LoginServlet</servlet-name>
10.    <servlet-class>LoginServlet</servlet-class>
11.  </servlet>
12.  <servlet-mapping>
13.    <servlet-name>LoginServlet</servlet-name>
14.    <url-pattern>/LoginServlet</url-pattern>
15.  </servlet-mapping>
16.  <servlet>
17.    <description></description>
18.    <display-name>ProfileServlet</display-name>
19.    <servlet-name>ProfileServlet</servlet-name>
20.    <servlet-class>ProfileServlet</servlet-class>
21.  </servlet>
22.  <servlet-mapping>
23.    <servlet-name>ProfileServlet</servlet-name>
24.    <url-pattern>/ProfileServlet</url-pattern>
25.  </servlet-mapping>
26.  <servlet>
27.    <description></description>
28.    <display-name>LogoutServlet</display-name>
29.    <servlet-name>LogoutServlet</servlet-name>
30.    <servlet-class>LogoutServlet</servlet-class>
31.  </servlet>
32.  <servlet-mapping>
33.    <servlet-name>LogoutServlet</servlet-name>
```

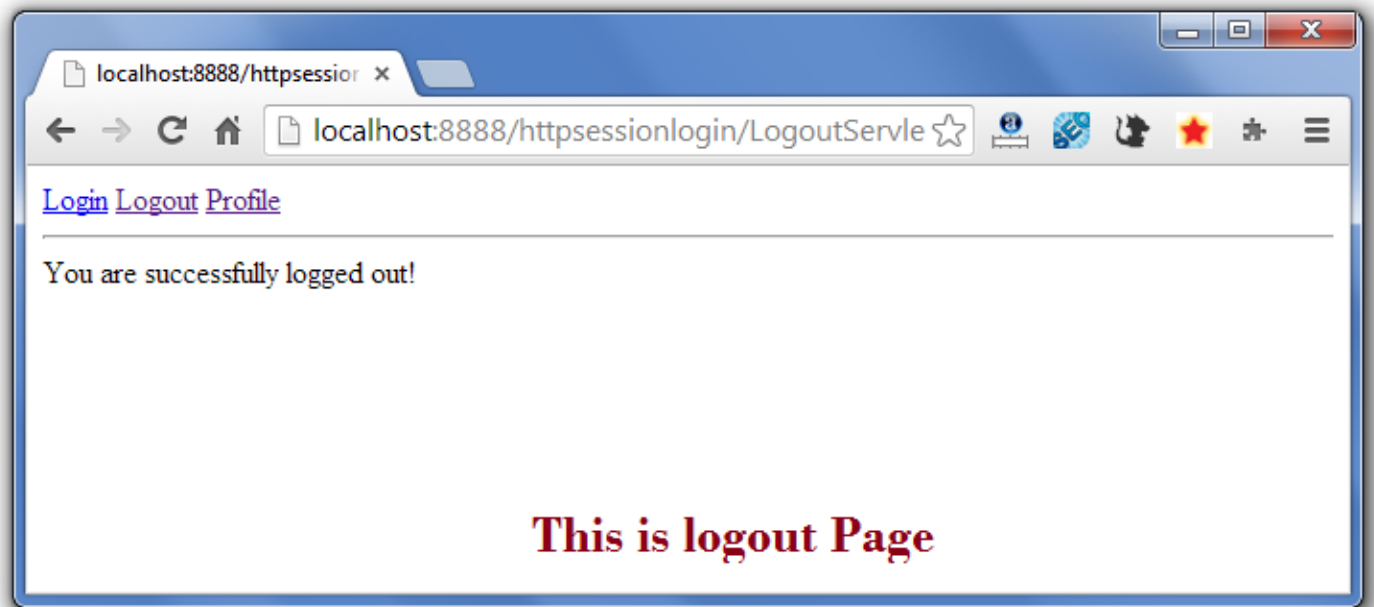
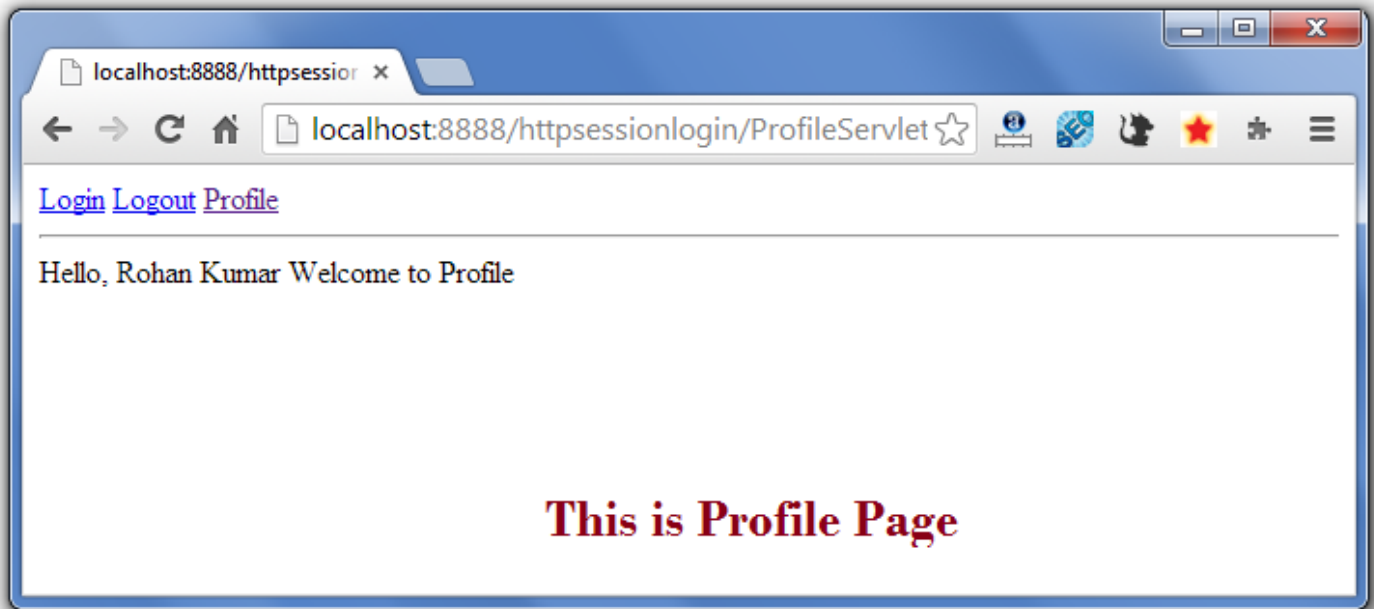
- 34. `<url-pattern>/LogoutServlet</url-pattern>`
 - 35. `</servlet-mapping>`
 - 36. `</web-app>`
-

[download this example \(developed using Eclipse IDE\)](#)

Output







If again you click on the profile link, you need to login first

Event and Listener in Servlet

Events are basically occurrence of something. Changing the state of an object is known as an event.

We can perform some important tasks at the occurrence of these exceptions, such as counting total and current logged-in users, creating tables of the database at time of deploying the project, creating database connection object etc.

There are many Event classes and Listener interfaces in the javax.servlet and javax.servlet.http packages.

Event classes

The event classes are as follows:

1. ServletRequestEvent
2. ServletContextEvent
3. ServletRequestAttributeEvent
4. ServletContextAttributeEvent
5. HttpSessionEvent
6. HttpSessionBindingEvent

Event interfaces

The event interfaces are as follows:

1. ServletRequestListener
2. ServletRequestAttributeListener
3. ServletContextListener
4. ServletContextAttributeListener
5. HttpSessionListener
6. HttpSessionAttributeListener
7. HttpSessionBindingListener
8. HttpSessionActivationListener

Upcoming topics in Servlet Events and Listeners

[ServletContextEvent](#)

Let's see the simple example of ServletContextEvent and ServletContextListener

[HttpSessionEvent](#)

Let's see the simple example of HttpSessionEvent and HttpSessionListener

[ServletRequestEvent](#)

Let's see the simple example of ServletRequestEvent and ServletRequestListener

[ServletContextAttributeEvent](#)

Let's see the simple example of ServletContextAttributeEvent and ServletContextAttributeListener

[HttpSessionBindingEvent](#)

Let's see the simple example of HttpSessionBindingEvent and HttpSessionAttributeListener

[ServletRequestAttributeEvent](#)

Let's see the simple example of ServletRequestAttributeEvent and ServletRequestAttributeEvent

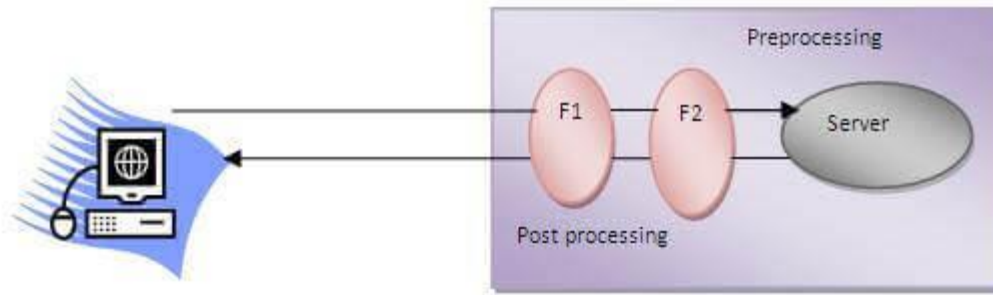
Servlet Filter

A **filter** is an object that is invoked at the preprocessing and postprocessing of a request.

It is mainly used to perform filtering tasks such as conversion, logging, compression, encryption and decryption, input validation etc.

The **servlet filter is pluggable**, i.e. its entry is defined in the web.xml file, if we remove the entry of filter from the web.xml file, filter will be removed automatically and we don't need to change the servlet.

So maintenance cost will be less.



Note: Unlike Servlet, One filter doesn't have dependency on another filter.

Usage of Filter

- recording all incoming requests
- logs the IP addresses of the computers from which the requests originate
- conversion
- data compression
- encryption and decryption
- input validation etc.

Advantage of Filter

1. Filter is pluggable.
2. One filter don't have dependency onto another resource.
3. Less Maintenance

Filter API

Like servlet filter have its own API. The javax.servlet package contains the three interfaces of Filter API.

1. Filter
2. FilterChain
3. FilterConfig

1) Filter interface

For creating any filter, you must implement the Filter interface. Filter interface provides the life cycle methods for a filter.

Method	Description
public void init(FilterConfig config)	init() method is invoked only once. It is used to initialize the filter.
public void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)	doFilter() method is invoked every time when user request to any resource, to which the filter is mapped. It is used to perform filtering tasks.
public void destroy()	This is invoked only once when filter is taken out of the service.

2) FilterChain interface

The object of FilterChain is responsible to invoke the next filter or resource in the chain. This object is passed in the doFilter method of Filter interface. The FilterChain interface contains only one method:

1. **public void doFilter(HttpServletRequest request, HttpServletResponse response):** it passes the control to the next filter or resource.

How to define Filter

We can define filter same as servlet. Let's see the elements of filter and filter-mapping.

1. <web-app>
- 2.
3. <filter>
4. <filter-name> ... </filter-name>
5. <filter-class> ... </filter-class>
6. </filter>

- 7.
8. <filter-mapping>
9. <filter-name>...</filter-name>
10. <url-pattern>...</url-pattern>
11. </filter-mapping>
- 12.
13. </web-app>

For mapping filter we can use, either url-pattern or servlet-name. The url-pattern elements has an advantage over servlet-name element i.e. it can be applied on servlet, JSP or HTML.

Simple Example of Filter

In this example, we are simply displaying information that filter is invoked automatically after the post processing of the request.

index.html

1. click here

MyFilter.java

1. **import** java.io.IOException;
2. **import** java.io.PrintWriter;
- 3.
4. **import** javax.servlet.*;
- 5.
6. **public class** MyFilter **implements** Filter{
- 7.
8. **public void** init(FilterConfig arg0) **throws** ServletException {}
- 9.
10. **public void** doFilter(ServletRequest req, ServletResponse resp,
11. FilterChain chain) **throws** IOException, ServletException {
- 12.
13. PrintWriter out=resp.getWriter();
14. out.print("filter is invoked before");
- 15.

```

16. chain.doFilter(req, resp); //sends request to next resource
17.
18. out.print("filter is invoked after");
19. }
20. public void destroy() {}
21. }

```

HelloServlet.java

```

1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.http.*;
6.
7. public class HelloServlet extends HttpServlet {
8.     public void doGet(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.
11.         response.setContentType("text/html");
12.         PrintWriter out = response.getWriter();
13.
14.         out.print("<br>welcome to servlet<br>");
15.
16.     }
17.
18. }

```

web.xml

For defining the filter, filter element of web-app must be defined just like servlet.

```

1. <web-app>
2.
3. <servlet>
4. <servlet-name>s1</servlet-name>
5. <servlet-class>HelloServlet</servlet-class>
6. </servlet>
7.

```

8. <servlet-mapping>
9. <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
- 12.
13. <filter>
14. <filter-name>f1</filter-name>
15. <filter-**class**>MyFilter</filter-**class**>
16. </filter>
- 17.
18. <filter-mapping>
19. <filter-name>f1</filter-name>
20. <url-pattern>/servlet1</url-pattern>
21. </filter-mapping>
- 22.
- 23.
24. </web-app>

Authentication Filter

We can perform authentication in filter. Here, we are going to check to password given by the user in filter class, if given password is admin, it will forward the request to the WelcomeAdmin servlet otherwise it will display error message.

Example of authenticating user using filter

Let's see the simple example of authenticating user using filter.

- Here, we have created 4 files: index.html
- MyFilter.java
- AdminServlet.java
- web.xml

index.html

1. <form action="**servlet1**">
2. Name:<input type="**text**" name="**name**"/>

3. Password:<input type="**password**" name="**password**"/>

4.
5. <input type="submit" value="login">
6.
7. </form>

MyFilter.java

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3. import javax.servlet.*;
4.
5. public class MyFilter implements Filter{
6.
7.     public void init(FilterConfig arg0) throws ServletException {}
8.
9.     public void doFilter(ServletRequest req, ServletResponse resp,
10.         FilterChain chain) throws IOException, ServletException {
11.
12.         PrintWriter out=resp.getWriter();
13.
14.         String password=req.getParameter("password");
15.         if(password.equals("admin")){
16.             chain.doFilter(req, resp); //sends request to next resource
17.         }
18.         else{
19.             out.print("username or password error!");
20.             RequestDispatcher rd=req.getRequestDispatcher("index.html");
21.             rd.include(req, resp);
22.         }
23.
24.     }
25.     public void destroy() {}
26.
27. }
```

AdminServlet.java

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.http.*;
6.
7. public class AdminServlet extends HttpServlet {
8.     public void doGet(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.
11.         response.setContentType("text/html");
12.         PrintWriter out = response.getWriter();
13.
14.         out.print("welcome ADMIN");
15.         out.close();
16.     }
17. }
```

web.xml

```
1. <web-app>
2.     <servlet>
3.         <servlet-name>AdminServlet</servlet-name>
4.         <servlet-class>AdminServlet</servlet-class>
5.     </servlet>
6.
7.     <servlet-mapping>
8.         <servlet-name>AdminServlet</servlet-name>
9.         <url-pattern>/servlet1</url-pattern>
10.     </servlet-mapping>
11.
12.     <filter>
13.         <filter-name>f1</filter-name>
14.         <filter-class>MyFilter</filter-class>
15.     </filter>
16.     <filter-mapping>
```

17. `<filter-name>f1</filter-name>`
18. `<url-pattern>/servlet1</url-pattern>`
19. `</filter-mapping>`
- 20.
21. `</web-app>`

FilterConfig

An object of FilterConfig is created by the web container. This object can be used to get the configuration information from the web.xml file.

Methods of FilterConfig interface

There are following 4 methods in the FilterConfig interface.

1. **public void init(FilterConfig config):** init() method is invoked only once it is used to initialize the filter.
2. **public String getInitParameter(String parameterName):** Returns the parameter value for the specified parameter name.
3. **public java.util.Enumeration getInitParameterNames():** Returns an enumeration containing all the parameter names.
4. **public ServletContext getServletContext():** Returns the ServletContext object.

Example of FilterConfig

In this example, if you change the param-value to no, request will be forwarded to the servlet otherwise filter will create the response with the message: this page is underprocessing. Let's see the simple example of FilterConfig. Here, we have created 4 files:

- index.html
- MyFilter.java
- HelloServlet.java
- web.xml

index.html

1. click here

MyFilter.java

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.*;
5.
6. public class MyFilter implements Filter{
7.     FilterConfig config;
8.
9.     public void init(FilterConfig config) throws ServletException {
10.         this.config=config;
11.     }
12.
13.     public void doFilter(ServletRequest req, ServletResponse resp,
14.         FilterChain chain) throws IOException, ServletException {
15.
16.         PrintWriter out=resp.getWriter();
17.
18.         String s=config.getInitParameter("construction");
19.
20.         if(s.equals("yes")){
21.             out.print("This page is under construction");
22.         }
23.         else{
24.             chain.doFilter(req, resp); //sends request to next resource
25.         }
26.
27.     }
28.     public void destroy() {}
29. }
```

HelloServlet.java

```
1. import java.io.IOException;
```

```
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.http.*;
6.
7. public class HelloServlet extends HttpServlet {
8.     public void doGet(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.
11.         response.setContentType("text/html");
12.         PrintWriter out = response.getWriter();
13.
14.         out.print("<br>welcome to servlet<br>");
15.
16.     }
17.
18. }
```

web.xml

```
1. <web-app>
2.
3.     <servlet>
4.         <servlet-name>HelloServlet</servlet-name>
5.         <servlet-class>HelloServlet</servlet-class>
6.     </servlet>
7.
8.     <servlet-mapping>
9.         <servlet-name>HelloServlet</servlet-name>
10.        <url-pattern>/servlet1</url-pattern>
11.    </servlet-mapping>
12.
13.    <filter>
14.        <filter-name>f1</filter-name>
15.        <filter-class>MyFilter</filter-class>
16.    </filter>
```

17. <param-name>construction</param-name>
18. <param-value>no</param-value>
19. </init-param>
20. </filter>
21. <filter-mapping>
22. <filter-name>f1</filter-name>
23. <url-pattern>/servlet1</url-pattern>
24. </filter-mapping>
- 25.
- 26.
27. </web-app>

ServletInputStream class

ServletInputStream class provides stream to read binary data such as image etc. from the request object. It is an abstract class.

The **getInputStream()** method of **ServletRequest** interface returns the instance of ServletInputStream class. So can be get as:

1. ServletInputStream sin=request.getInputStream();

Method of ServletInputStream class

There are only one method defined in the ServletInputStream class.

1. **int readLine(byte[] b, int off, int len)** it reads the input stream.

ServletOutputStream class

ServletOutputStream class provides a stream to write binary data into the response. It is an abstract class.

The **getOutputStream()** method of **ServletResponse** interface returns the instance of ServletOutputStream class. It may be get as:

1. ServletOutputStream out=response.getOutputStream();

Methods of ServletOutputStream class

The ServletOutputStream class provides print() and println() methods that are overloaded.

1. void print(boolean b){}
2. void print(char c){}
3. void print(int i){}
4. void print(long l){}
5. void print(float f){}
6. void print(double d){}
7. void print(String s){}
8. void println{}
9. void println(boolean b){}
10. void println(char c){}
11. void println(int i){}
12. void println(long l){}
13. void println(float f){}
14. void println(double d){}
15. void println(String s){}

CRUD in Servlet

A CRUD (Create, Read, Update and Delete) application is the most important application for any project development. In Servlet, we can easily create CRUD application.

Servlet CRUD example

Create "user905" table in Oracle Database with auto incrementing id using sequence. There are 5 fields in it: id, name, password, email and country.

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
NAME	VARCHAR2(4000)	Yes	-	-
PASSWORD	VARCHAR2(4000)	Yes	-	-
EMAIL	VARCHAR2(4000)	Yes	-	-
COUNTRY	VARCHAR2(4000)	Yes	-	-
				1 - 5

File: index.html

1. <!DOCTYPE html>
2. <html>
3. <head>
4. <meta charset="ISO-8859-1">
5. <title>Insert title here</title>
6. </head>
7. <body>
- 8.
9. <h1>Add New Employee</h1>
10. <form action="SaveServlet" method="post">
11. <table>
12. <tr><td>Name:</td><td><input type="text" name="name"/></td></tr>
13. <tr><td>Password:</td><td><input type="password" name="password"/></td></tr>
14. <tr><td>Email:</td><td><input type="email" name="email"/></td></tr>
15. <tr><td>Country:</td><td>
16. <select name="country" style="width:150px">
17. <option>India</option>
18. <option>USA</option>
19. <option>UK</option>
20. <option>Other</option>
21. </select>
22. </td></tr>
23. <tr><td colspan="2"><input type="submit" value="Save Employee"/></td></tr>
24. </table>
25. </form>
- 26.


```
27. <br/>
28. <a href="ViewServlet">view employees</a>
29.
30. </body>
31. </html>

1. File: Emp.java public class Emp {
2.   private int id;
3.   private String name,password,email,country;
4.   public int getId() {
5.     return id;
6.   }
7.   public void setId(int id) {
8.     this.id = id;
9.   }
10. public String getName() {
11.   return name;
12. }
13. public void setName(String name) {
14.   this.name = name;
15. }
16. public String getPassword() {
17.   return password;
18. }
19. public void setPassword(String password) {
20.   this.password = password;
21. }
22. public String getEmail() {
23.   return email;
24. }
25. public void setEmail(String email) {
26.   this.email = email;
27. }
28. public String getCountry() {
29.   return country;
30. }
```

```
31. public void setCountry(String country) {  
32.     this.country = country;  
33. }  
34.  
35. }
```

File: EmpDao.java

```
1. import java.util.*;  
2. import java.sql.*;  
3.  
4. public class EmpDao {  
5.  
6.     public static Connection getConnection(){  
7.         Connection con=null;  
8.         try{  
9.             Class.forName("oracle.jdbc.driver.OracleDriver");  
10.            con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");  
11.        }catch(Exception e){System.out.println(e);}  
12.        return con;  
13.    }  
14.    public static int save(Emp e){  
15.        int status=0;  
16.        try{  
17.            Connection con=EmpDao.getConnection();  
18.            PreparedStatement ps=con.prepareStatement(  
19.                "insert into user905(name,password,email,country) values (?,?,,?)");  
20.            ps.setString(1,e.getName());  
21.            ps.setString(2,e.getPassword());  
22.            ps.setString(3,e.getEmail());  
23.            ps.setString(4,e.getCountry());  
24.  
25.            status=ps.executeUpdate();  
26.  
27.            con.close();
```

```
28.     }catch(Exception ex){ex.printStackTrace();}
29.
30.     return status;
31. }
32. public static int update(Emp e){
33.     int status=0;
34.     try{
35.         Connection con=EmpDao.getConnection();
36.         PreparedStatement ps=con.prepareStatement(
37.             "update user905 set name=?,password=?,email=?,country=? where id=?
38. ");
39.         ps.setString(1,e.getName());
40.         ps.setString(2,e.getPassword());
41.         ps.setString(3,e.getEmail());
42.         ps.setString(4,e.getCountry());
43.         ps.setInt(5,e.getId());
44.
45.         status=ps.executeUpdate();
46.
47.         con.close();
48.     }catch(Exception ex){ex.printStackTrace();}
49.
50.     return status;
51. }
52. public static int delete(int id){
53.     int status=0;
54.     try{
55.         Connection con=EmpDao.getConnection();
56.         PreparedStatement ps=con.prepareStatement("delete from user905 where id=?"
57. );
58.         ps.setInt(1,id);
59.         status=ps.executeUpdate();
60.
61.         con.close();
62.     }catch(Exception e){e.printStackTrace();}
```

```
61.
62.     return status;
63. }
64. public static Emp getEmployeeById(int id){
65.     Emp e=new Emp();
66.
67.     try{
68.         Connection con=EmpDao.getConnection();
69.         PreparedStatement ps=con.prepareStatement("select * from user905 where id=?
70. ");
71.         ps.setInt(1,id);
72.         ResultSet rs=ps.executeQuery();
73.         if(rs.next()){
74.             e.setId(rs.getInt(1));
75.             e.setName(rs.getString(2));
76.             e.setPassword(rs.getString(3));
77.             e.setEmail(rs.getString(4));
78.             e.setCountry(rs.getString(5));
79.         }
80.         con.close();
81.     }catch(Exception ex){ex.printStackTrace();}
82.     return e;
83. }
84. public static List<Emp> getAllEmployees(){
85.     List<Emp> list=new ArrayList<Emp>();
86.
87.     try{
88.         Connection con=EmpDao.getConnection();
89.         PreparedStatement ps=con.prepareStatement("select * from user905");
90.         ResultSet rs=ps.executeQuery();
91.         while(rs.next()){
92.             Emp e=new Emp();
93.             e.setId(rs.getInt(1));
94.             e.setName(rs.getString(2));
```

```

95.         e.setPassword(rs.getString(3));
96.         e.setEmail(rs.getString(4));
97.         e.setCountry(rs.getString(5));
98.         list.add(e);
99.     }
100.         con.close();
101.     }catch(Exception e){e.printStackTrace();}
102.
103.         return list;
104.     }
105. }

```

File: SaveServlet.java

```

1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.annotation.WebServlet;
6. import javax.servlet.http.HttpServlet;
7. import javax.servlet.http.HttpServletRequest;
8. import javax.servlet.http.HttpServletResponse;
9. @WebServlet("/SaveServlet")
10. public class SaveServlet extends HttpServlet {
11.     protected void doPost(HttpServletRequest request, HttpServletResponse response)
12.         throws ServletException, IOException {
13.         response.setContentType("text/html");
14.         PrintWriter out=response.getWriter();
15.
16.         String name=request.getParameter("name");
17.         String password=request.getParameter("password");
18.         String email=request.getParameter("email");
19.         String country=request.getParameter("country");
20.
21.         Emp e=new Emp();
22.         e.setName(name);

```

```

23.     e.setPassword(password);
24.     e.setEmail(email);
25.     e.setCountry(country);
26.
27.     int status=EmpDao.save(e);
28.     if(status>0){
29.         out.print("<p>Record saved successfully!</p>");
30.         request.getRequestDispatcher("index.html").include(request, response);
31.     }else{
32.         out.println("Sorry! unable to save record");
33.     }
34.
35.     out.close();
36. }
37.
38. }

```

File: EditServlet.java

```

1. import java.io.IOException;
2. import java.io.PrintWriter;
3.
4. import javax.servlet.ServletException;
5. import javax.servlet.annotation.WebServlet;
6. import javax.servlet.http.HttpServlet;
7. import javax.servlet.http.HttpServletRequest;
8. import javax.servlet.http.HttpServletResponse;
9. @WebServlet("/EditServlet")
10. public class EditServlet extends HttpServlet {
11.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
12.         throws ServletException, IOException {
13.         response.setContentType("text/html");
14.         PrintWriter out=response.getWriter();
15.         out.println("<h1>Update Employee</h1>");
16.         String sid=request.getParameter("id");
17.         int id=Integer.parseInt(sid);

```

```

18.
19.     Emp e=EmpDao.getEmployeeById(id);
20.
21.     out.print("<form action='EditServlet2' method='post'>");
22.     out.print("<table>");
23.     out.print("<tr><td></td><td><input type='hidden' name='id' value='"+e.getId()
+ "'/></td></tr>");
24.     out.print("<tr><td>Name:</td><td><input type='text' name='name' value='"+e.
getName()+"'/></td></tr>");
25.     out.print("<tr><td>Password:</td><td><input type='password' name='password'
value='"+e.getPassword()+"'/>
26.         </td></tr>");
27.     out.print("<tr><td>Email:</td><td><input type='email' name='email' value='"+e.
getEmail()+"'/></td></tr>");
28.     out.print("<tr><td>Country:</td><td>");
29.     out.print("<select name='country' style='width:150px'>");
30.     out.print("<option>India</option>");
31.     out.print("<option>USA</option>");
32.     out.print("<option>UK</option>");
33.     out.print("<option>Other</option>");
34.     out.print("</select>");
35.     out.print("</td></tr>");
36.     out.print("<tr><td colspan='2'><input type='submit' value='Edit & Save' /></td>
</tr>");
37.     out.print("</table>");
38.     out.print("</form>");
39.
40.     out.close();
41. }
42. }

```

File: EditServlet2.java

```

1. import java.io.IOException;
2. import java.io.PrintWriter;
3.

```

```
4. import javax.servlet.ServletException;
5. import javax.servlet.annotation.WebServlet;
6. import javax.servlet.http.HttpServlet;
7. import javax.servlet.http.HttpServletRequest;
8. import javax.servlet.http.HttpServletResponse;
9. @WebServlet("/EditServlet2")
10. public class EditServlet2 extends HttpServlet {
11.     protected void doPost(HttpServletRequest request, HttpServletResponse response)
12.         throws ServletException, IOException {
13.         response.setContentType("text/html");
14.         PrintWriter out=response.getWriter();
15.
16.         String sid=request.getParameter("id");
17.         int id=Integer.parseInt(sid);
18.         String name=request.getParameter("name");
19.         String password=request.getParameter("password");
20.         String email=request.getParameter("email");
21.         String country=request.getParameter("country");
22.
23.         Emp e=new Emp();
24.         e.setld(id);
25.         e.setName(name);
26.         e.setPassword(password);
27.         e.setEmail(email);
28.         e.setCountry(country);
29.
30.         int status=EmpDao.update(e);
31.         if(status>0){
32.             response.sendRedirect("ViewServlet");
33.         }else{
34.             out.println("Sorry! unable to update record");
35.         }
36.
37.         out.close();
38.     }
```


39.
40. }

File: DeleteServlet.java

```
1. import java.io.IOException;
2. import javax.servlet.ServletException;
3. import javax.servlet.annotation.WebServlet;
4. import javax.servlet.http.HttpServlet;
5. import javax.servlet.http.HttpServletRequest;
6. import javax.servlet.http.HttpServletResponse;
7. @WebServlet("/DeleteServlet")
8. public class DeleteServlet extends HttpServlet {
9.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
10.         throws ServletException, IOException {
11.         String sid=request.getParameter("id");
12.         int id=Integer.parseInt(sid);
13.         EmpDao.delete(id);
14.         response.sendRedirect("ViewServlet");
15.     }
16. }
```

File: ViewServlet.java

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3. import java.util.List;
4.
5. import javax.servlet.ServletException;
6. import javax.servlet.annotation.WebServlet;
7. import javax.servlet.http.HttpServlet;
8. import javax.servlet.http.HttpServletRequest;
9. import javax.servlet.http.HttpServletResponse;
10. @WebServlet("/ViewServlet")
11. public class ViewServlet extends HttpServlet {
12.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
13.         throws ServletException, IOException {
```

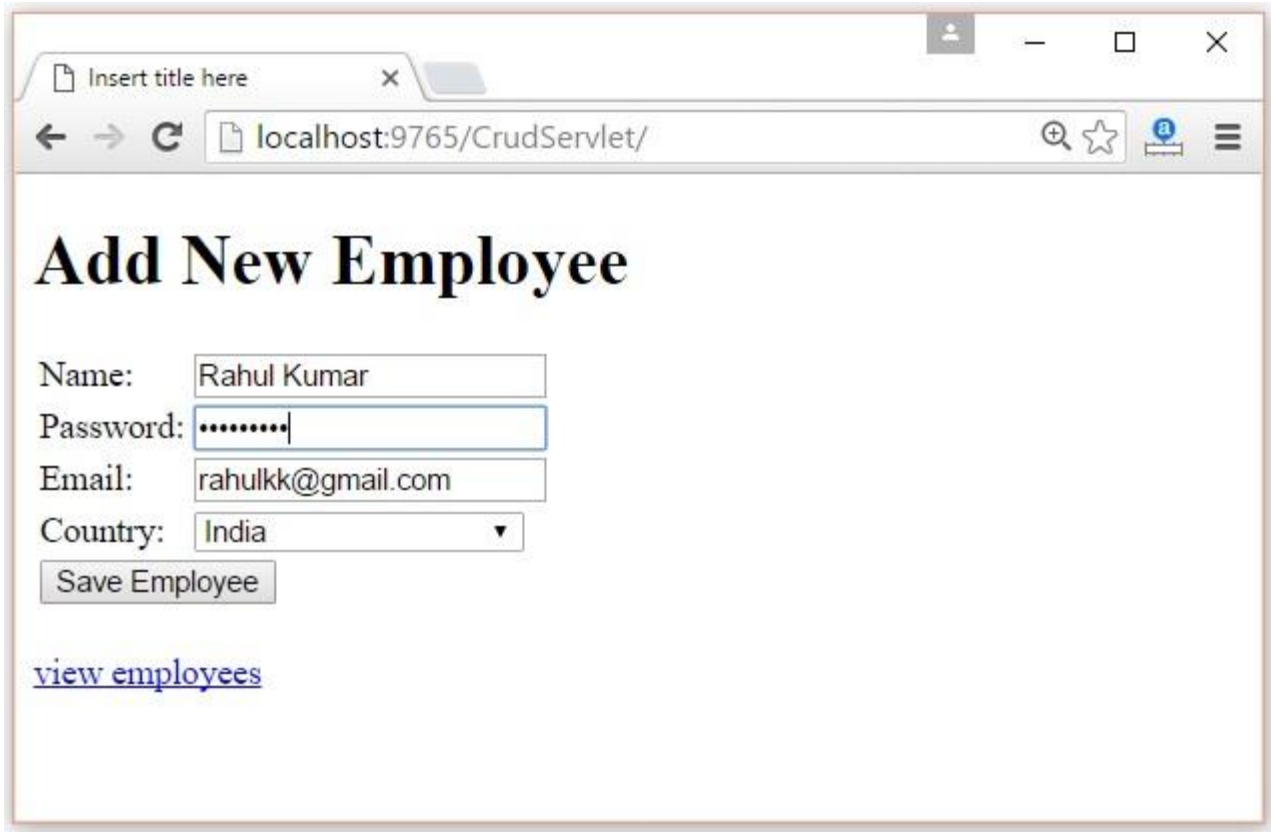
```

14. response.setContentType("text/html");
15. PrintWriter out=response.getWriter();
16. out.println("<a href='index.html'>Add New Employee</a>");
17. out.println("<h1>Employees List</h1>");
18.
19. List<Emp> list=EmpDao.getAllEmployees();
20.
21. out.print("<table border='1' width='100%'");
22. out.print("<tr><th>Id</th><th>Name</th><th>Password</th><th>Email</th><th>Cou
    ntry</th>
23.         <th>Edit</th><th>Delete</th></tr>");
24. for(Emp e:list){
25.     out.print("<tr><td>" +e.getId()+"</td><td>" +e.getName()+"</td><td>" +e.getPassword(
    )+"</td>
26.         <td>" +e.getEmail()+"</td><td>" +e.getCountry()+"</td><td><a href='EditServlet?i
    d="+e.getId()+">edit</a></td>
27.         <td><a href='DeleteServlet?id="+e.getId()+">delete</a></td></tr>");
28.     }
29. out.print("</table>");
30.
31. out.close();
32. }
33. }

```

Output

First page will look like this, fill the form and submit it.



The screenshot shows a web browser window with the address bar displaying "localhost:9765/CrudServlet/". The page title is "Insert title here". The main heading is "Add New Employee". Below the heading, there is a form with the following fields:

- Name:
- Password:
- Email:
- Country:

Below the form fields, there is a button labeled "Save Employee". At the bottom of the form, there is a link labeled [view employees](#).

You will get a message "Record successfully saved!".

Record saved successfully!

Add New Employee

Name:

Password:

Email:

Country:

[view employees](#)

Click on the View Employees link to see the total employees list.

localhost:9765/CrudServle X

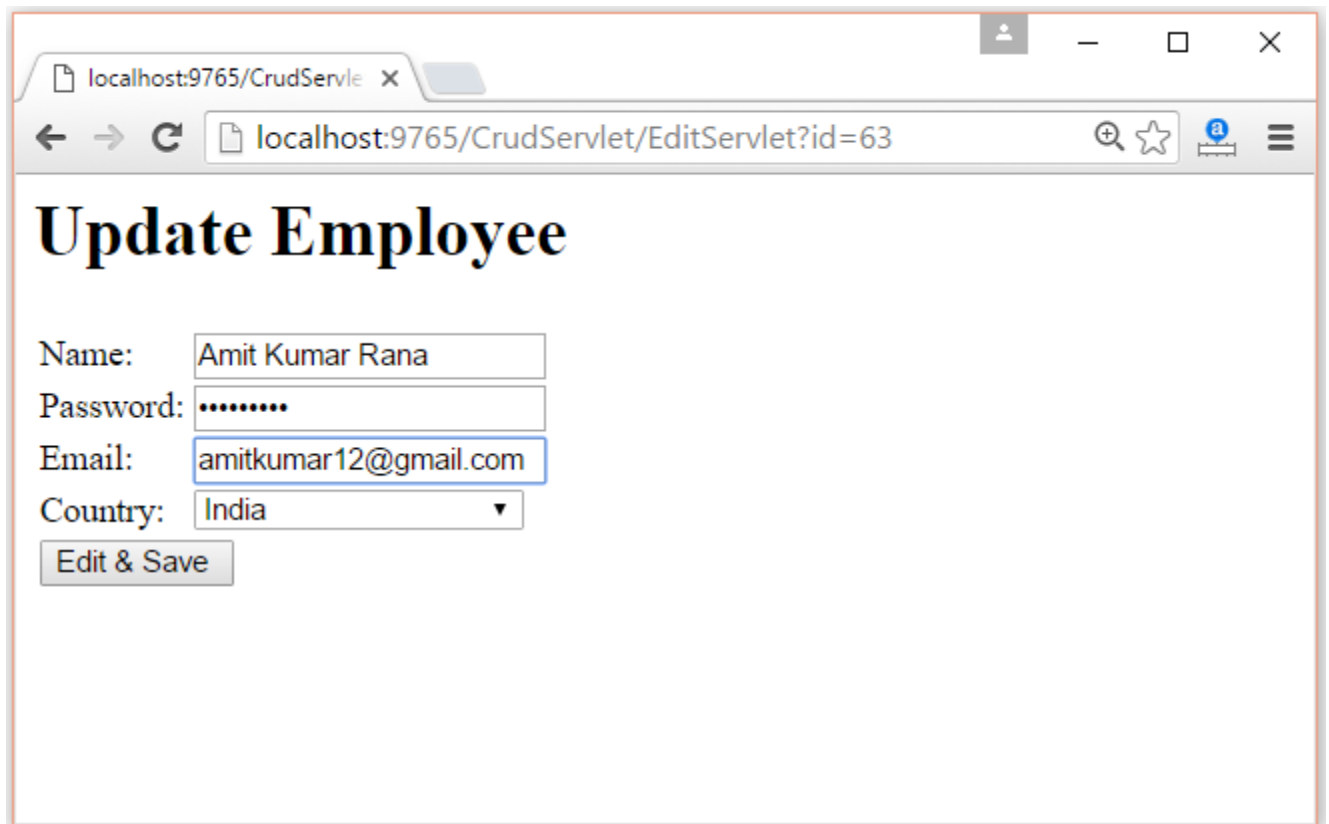
localhost:9765/CrudServlet/ViewServlet

[Add New Employee](#)

Employees List

ID	Name	Password	Email	Country	Edit	Delete
63	Amit Kumar	amtknj45	amitkumar@gmail.com	India	edit	delete
61	Rahul Kumar	rahul4000	rahulkk@gmail.com	India	edit	delete
62	Sonoo Jaiswal	sonoobsk	sonoojaiswal1987@gmail.com	India	edit	delete
44	adarsh kumar	kkkkk	adarsh232@gmail.com	India	edit	delete

Click on the update link, to change the data.



The screenshot shows a web browser window with the address bar displaying `localhost:9765/CrudServlet/EditServlet?id=63`. The page title is "Update Employee". The form contains the following fields:

- Name:
- Password:
- Email:
- Country:

Below the form is a button labeled "Edit & Save".

After changing information, submit button. You will see that information is changed.

localhost:9765/CrudServle x

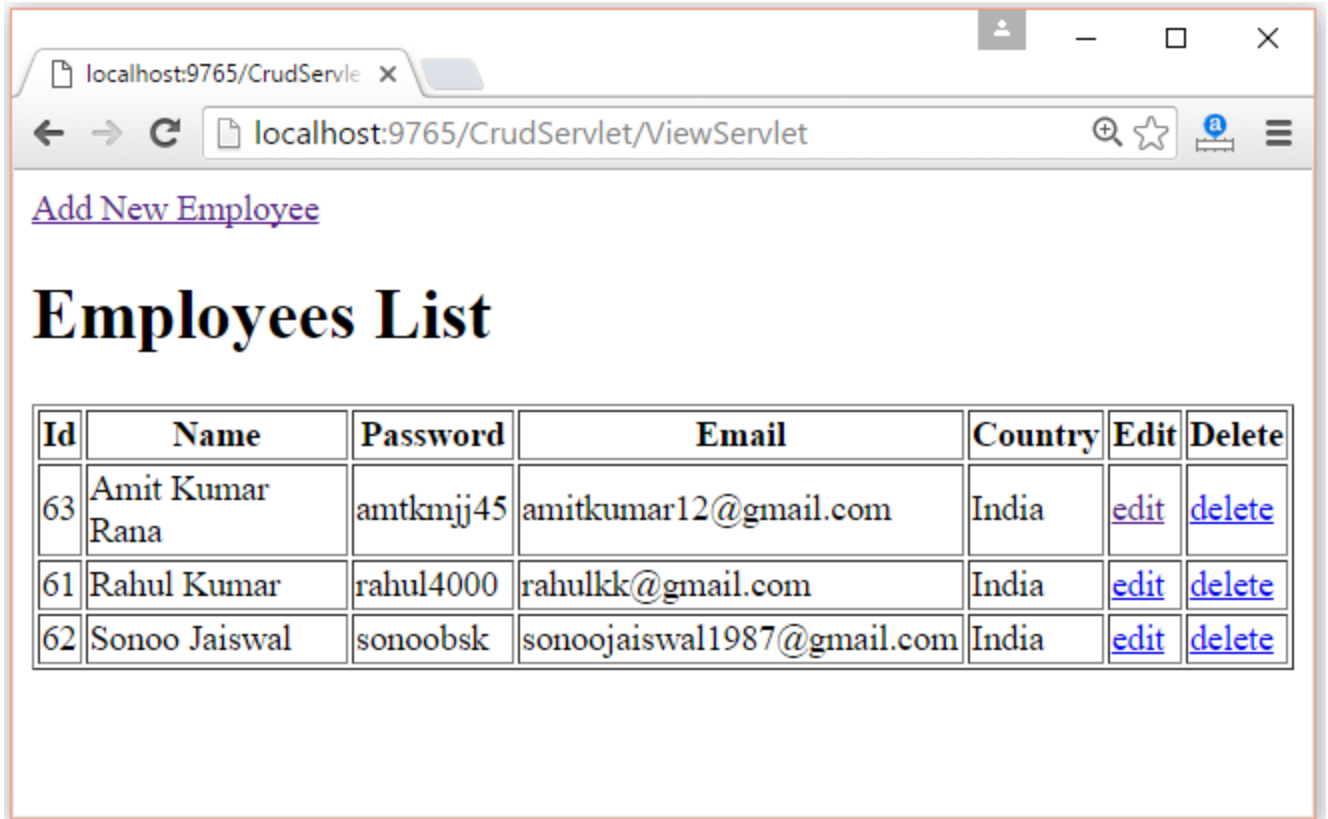
localhost:9765/CrudServlet/ViewServlet

[Add New Employee](#)

Employees List

Id	Name	Password	Email	Country	Edit	Delete
63	Amit Kumar Rana	amtkmj45	amitkumar12@gmail.com	India	edit	delete
61	Rahul Kumar	rahul4000	rahulkk@gmail.com	India	edit	delete
62	Sonoo Jaiswal	sonoobsk	sonoojaiswal1987@gmail.com	India	edit	delete
44	adarsh kumar	kkkkk	adarsh232@gmail.com	India	edit	delete

Now, click on the delete link to delete the record.



Pagination in Servlet

To divide large number of records into multiple parts, we use pagination. It allows user to display a part of records only. Loading all records in a single page may take time, so it is always recommended to create pagination. In servlet, we can develop pagination example easily.

In this servlet pagination example, we are using MySQL database to fetch records.

Here, we have created "emp" table in "test" database. The emp table has three fields: id, name and salary. Either create table and insert records manually or import our sql file.

index.html

1. `<a href="ViewServlet?page=1">View Employees</a>`

ViewServlet.java

1. `package com.javatpoint.servlets;`
- 2.
3. `import java.io.IOException;`
4. `import java.io.PrintWriter;`


```

5. import java.util.List;
6. import javax.servlet.ServletException;
7. import javax.servlet.annotation.WebServlet;
8. import javax.servlet.http.HttpServlet;
9. import javax.servlet.http.HttpServletRequest;
10. import javax.servlet.http.HttpServletResponse;
11. import com.javatpoint.beans.Emp;
12. import com.javatpoint.dao.EmpDao;
13.
14. @WebServlet("/ViewServlet")
15. public class ViewServlet extends HttpServlet {
16.     private static final long serialVersionUID = 1L;
17.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
18.         throws ServletException, IOException {
19.         response.setContentType("text/html");
20.         PrintWriter out=response.getWriter();
21.
22.         String spageid=request.getParameter("page");
23.         int pageid=Integer.parseInt(spageid);
24.         int total=5;
25.         if(pageid==1){}
26.         else{
27.             pageid=pageid-1;
28.             pageid=pageid*total+1;
29.         }
30.         List<Emp> list=EmpDao.getRecords(pageid,total);
31.
32.         out.print("<h1>Page No: "+spageid+"</h1>");
33.         out.print("<table border='1' cellpadding='4' width='60%'>");
34.         out.print("<tr><th>Id</th><th>Name</th><th>Salary</th>");
35.         for(Emp e:list){
36.             out.print("<tr><td>" +e.getId()+"</td><td>" +e.getName()+"</td><td>" +e.getSalary
37.                 ()+"</td></tr>");
38.         }
39.         out.print("</table>");

```

```

39.
40.     out.print("<a href='ViewServlet?page=1'>1</a> ");
41.     out.print("<a href='ViewServlet?page=2'>2</a> ");
42.     out.print("<a href='ViewServlet?page=3'>3</a> ");
43.
44.     out.close();
45. }
46. }

```

Emp.java

```

1. package com.javatpoint.beans;
2.
3. public class Emp {
4.     private int id;
5.     private String name;
6.     private float salary;
7.     //getters and setters
8. }

```

EmpDao.java

```

1. package com.javatpoint.dao;
2. import com.javatpoint.beans.*;
3. import java.sql.*;
4. import java.util.ArrayList;
5. import java.util.List;
6. public class EmpDao {
7.
8.     public static Connection getConnection(){
9.         Connection con=null;
10.        try{
11.            Class.forName("com.mysql.jdbc.Driver");
12.            con=DriverManager.getConnection("jdbc:mysql://localhost:3306/test","","");
13.        }catch(Exception e){System.out.println(e);}
14.        return con;
15.    }
16.

```

```

17. public static List<Emp> getRecords(int start,int total){
18.     List<Emp> list=new ArrayList<Emp>();
19.     try{
20.         Connection con=getConnection();
21.         PreparedStatement ps=con.prepareStatement("select * from emp limit "+(start-
1)+","+total);
22.         ResultSet rs=ps.executeQuery();
23.         while(rs.next()){
24.             Emp e=new Emp();
25.             e.setId(rs.getInt(1));
26.             e.setName(rs.getString(2));
27.             e.setSalary(rs.getFloat(3));
28.             list.add(e);
29.         }
30.         con.close();
31.     }catch(Exception e){System.out.println(e);}
32.     return list;
33. }
34. }

```

Output

